



Chip Hippo

User Guide

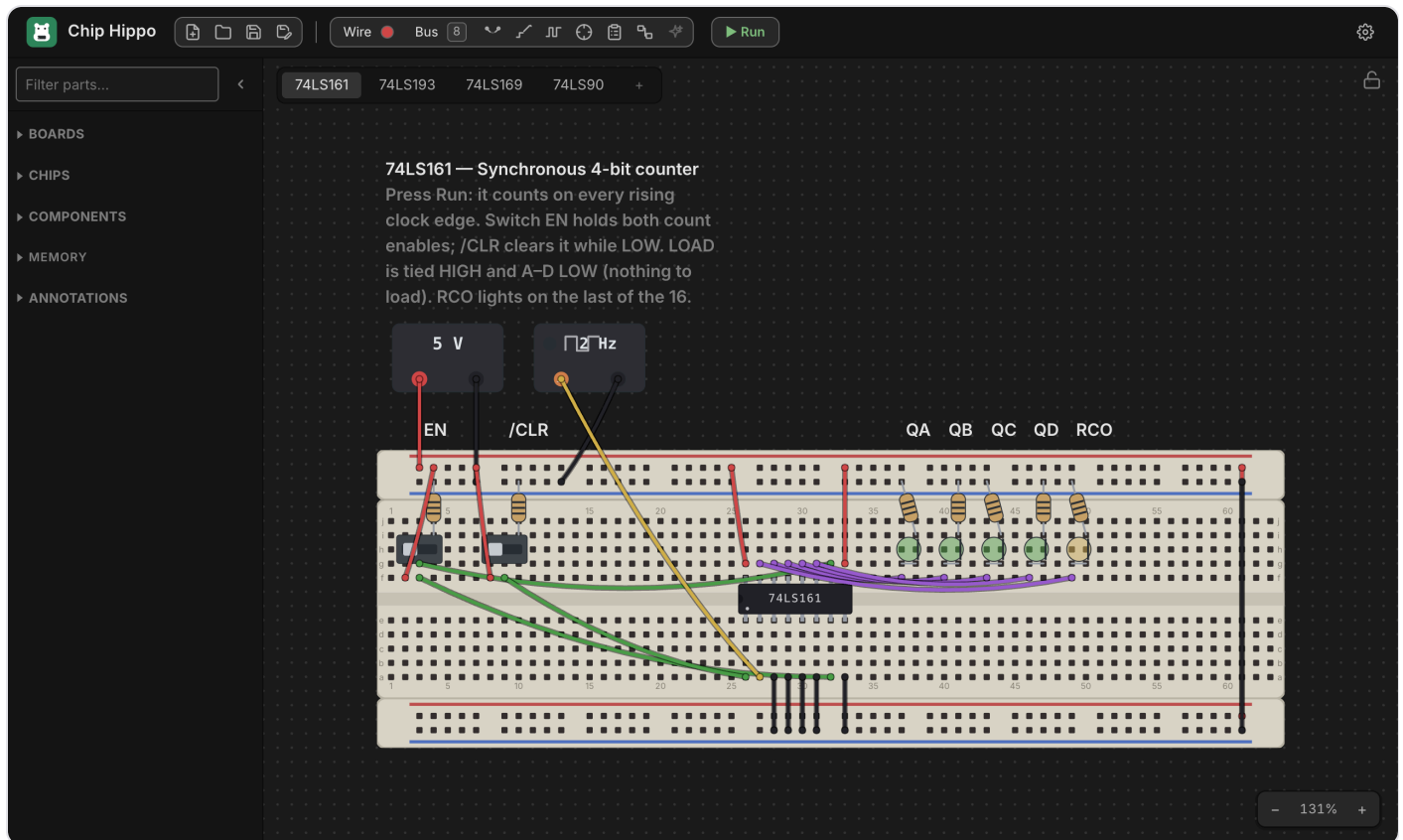
Version 1.0.2 · September 28, 2026

Contents

1	Overview
2	Getting Started
3	The Desk & Breadboards
4	Chips & Components
5	Wiring, Nets & Buses
6	Power & Clock Sources
7	The Chip Library
8	Running a Simulation
9	Probing & Net Names
10	Memory Chips & the Inspector
11	Logic Analyzer & Timing
12	Build Guide & BOM
13	Schematic View
14	AI Circuit Builder
15	Arduino Integration
16	Serial Protocol
17	Files, Saving & Undo
18	Projects & Desktops
19	Settings
20	Keyboard Shortcuts

Chip Hippo User Guide

Chip Hippo is a desktop app for designing and simulating **74xx TTL logic circuits** on virtual solderless breadboards. Place a breadboard on the infinite desk, populate it with 74xx DIP chips, wires, switches, LEDs, and power sources, then press **Run** — a simulation engine traces electricity from every power source, resolves each electrical net, and ripples changes through the circuit exactly like a real breadboard would.



What you can build

Anything a 74xx TTL breadboard bench can hold: combinational logic (gates, decoders, multiplexers), sequential logic (flip-flops, counters, shift registers), a free-running or manually stepped clock, and small memory circuits backed by ROM/RAM chips — all wired across Full/Half/Tiny breadboards, powered at 3 V / 5 V / 12 V, and watched settle live. Beyond the 74xx shelf there is a **PROCESSOR group** — a W65C02 and a Zilog Z80A, each a full instruction-set simulation — and an **Interface group** of 65xx peripherals (a PIA and a VIA), for building a small single-board computer out of the same glue logic.

Highlights

- **Breadboard-accurate placement** — chips, wires, and discretes snap to the real 0.1 in hole grid, with the same row-half/trench/rail tie-point rules as a physical board.
- **Live simulation** — Run the circuit and watch LEDs light, chip health badges report power/damage state, and switches drive the circuit live.
- **Deep inspection tools** — a connectivity **probe**, **net names/labels**, and a **logic analyzer** for capturing waveforms.
- **Memory chips with a hex inspector** — file-backed ROM/EEPROM images you program with an in-app programmer, plus a live hex/ASCII viewer.
- **A derived schematic view** — press `Tab` to flip between the physical breadboard and a logical diagram of chip symbols, routed named nets, and bus lines, kept in sync with the desk.
- **Build guide & BOM export** — including a numbered cutting list of every jumper — and **undo/redo** across every edit, per desktop of the open project.
- **Seven languages** — English, German, Spanish, French, Italian, Japanese and Chinese, chosen in Settings ▸ Appearance (this guide is English only).
- **An AI circuit builder** — describe a circuit in words, using your own AI connection, and get a wired design that has already been built and run before you're offered it.
- **A link to real hardware** — Outputs and Inputs carry values between the running circuit and an Arduino over USB, and Chip Hippo writes the sketch's side of the link for you.

Table of contents

Getting started

- [Getting Started](#) — install the app, place your first board and chip, and run your first circuit.

Building a circuit

- [The Desk & Breadboards](#) — pan/zoom, breadboard kits, strips and rails, snapping and grouping.
- [Chips & Components](#) — the parts palette, DIP chips, discretes, placement and rotation.
- [Wiring, Nets & Buses](#) — the wire tool, colors, cross-board wires, and multi-bit buses.
- [Power & Clock Sources](#) — PSU bricks, voltage and the 12 V damage rule, and clock sources.
- [The Chip Library](#) — the catalog: 74xx logic, memory, the 65xx interface parts and the CPUs, and the pin-assignments/datasheet window.

Simulating & inspecting

- [Running a Simulation](#) — Run/Stop/Pause/Step, the settle model, and live views.

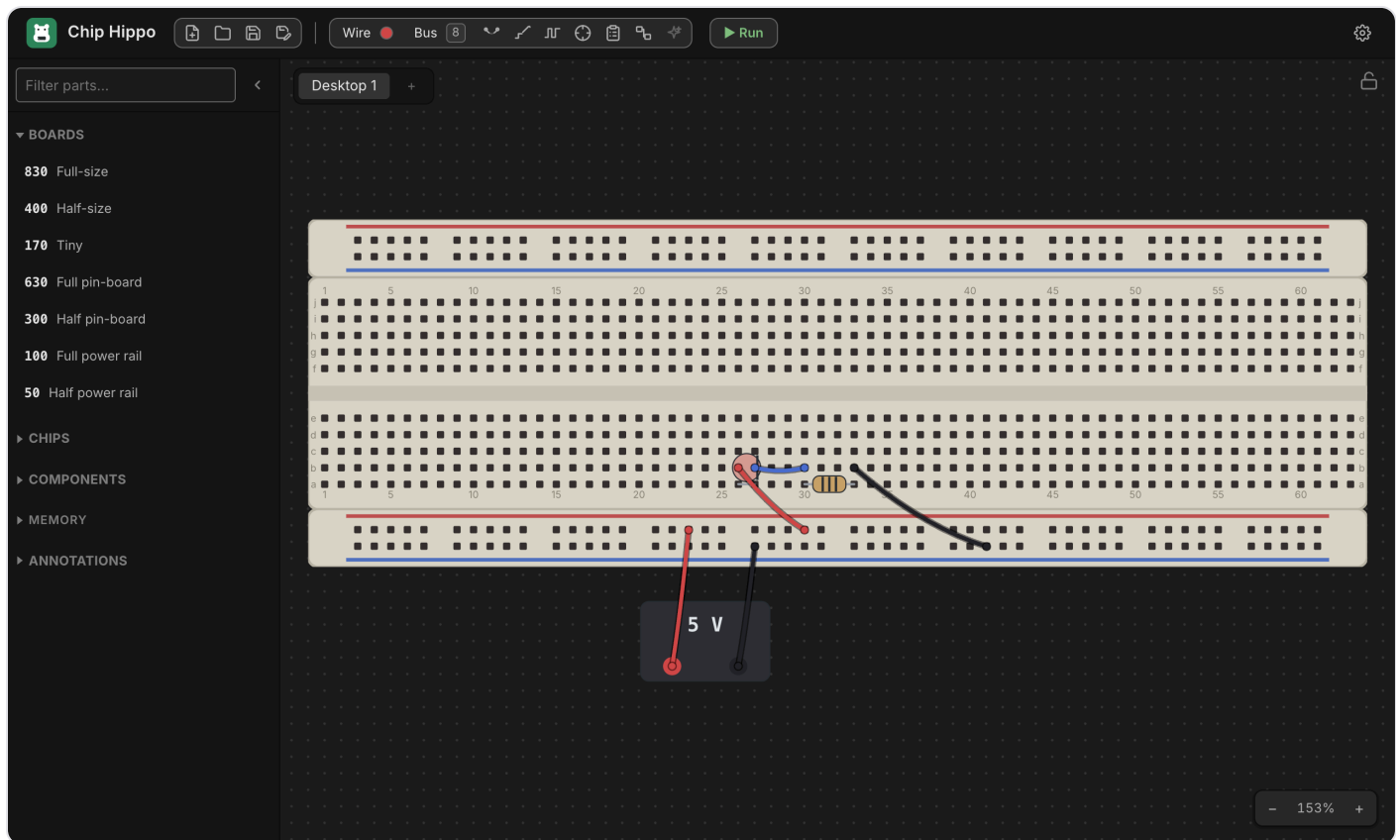
- [Probing & Net Names](#) — the connectivity probe and naming nets.
- [Memory Chips & the Inspector](#) — ROM/RAM, the programmer, and the hex/ASCII inspector.
- [Logic Analyzer & Timing](#) — capturing and reading waveforms.
- [Schematic View](#) — the derived logical diagram.
- [AI Circuit Builder](#) — describe a circuit in words and get a simulation-proven design, using your own AI connection.
- [Arduino Integration](#) — pass values between the running circuit and a real Arduino over USB, with the sketch's header generated for you.

Files & reference

- [Build Guide & BOM](#) — deriving a bill of materials, a jumper cutting list, and an ordered assembly checklist.
- [Files, Saving & Undo](#) — where a desktop and a project are kept, saving them, and undo/redo.
- [Projects & Desktops](#) — tabbed desktops, saving a project, and copying a design from one desktop onto another.
- [Settings](#) — the settings dialog and its options.
- [Keyboard Shortcuts](#) — every shortcut in one place.

Getting Started

This page walks through building the smallest possible circuit — a breadboard, a power supply, and an LED — from a completely empty desk to a lit LED. It takes about five minutes and introduces the tools you'll use for every larger circuit afterward.



Launch Chip Hippo

Open the installed app like any other desktop program (or, if you're running from source, `make debug`). Chip Hippo opens on an **infinitely pannable, zoomable desk** — drag to pan, scroll or pinch to zoom, and the header's zoom controls (bottom-right of the desk) reset or step the view. The first time you launch, the desk is empty and shows a hint: *"Open the parts tray and add a breadboard to get started."*

Open the parts palette

The palette is a tray docked to the left of the desk, and it carries its own open/close control rather than a toolbar button. The chevron in the tray's top-right corner shuts it down to a narrow strip along the desk's left edge; the chevron at the top of that strip (or `Ctrl+P` / `⌘P`) slides it back out.

Below that chevron the strip keeps an icon for each of the tray's sections — Boards, Chips, Components, Memory, Annotations and Signals — the same icon that sits beside that section's name in the open tray, and level with it. Clicking one opens the tray straight onto that section, with every other section folded away. The icons are sized from the text, so they grow and shrink with the font size you set in Settings.

At the top of the tray is the **board selector** (Full / Half / Tiny breadboards, plus loose strips); below that, chips and components grouped by function, with a filter box to search by name.

Drag the tray's **right edge** to make it wider or narrower — useful when a long part name is elided. The width you leave it at is remembered between launches, and across closing and reopening the tray.

Add a breadboard

In the palette's **BOARDS** section, click **Full-size** (830 tie points) — the default recommendation for a first circuit; it has the most room and both power rails already attached. A translucent ghost of the whole kit follows your cursor. Move it over open desk space and click to drop it.

The Full kit places three strips as one rigid group: a power rail, the pin-board, and a second power rail — exactly like a real solderless breadboard. You can drag the whole board later by its body, or reopen the palette any time to add more.

Add power

Still in the palette, open the **COMPONENTS ▶ Power** group and click **Power supply**. A small PSU brick ghost follows your cursor — click open desk space near the breadboard to place it. A PSU brick isn't seated on the board; it's a free-standing part with two wireable terminals, **+** and **−**.

Right-click the placed PSU and choose **Properties...** to confirm it's set to **5 V** (new PSUs default to 5 V, but 3 V and 12 V are also there — 12 V is enough to damage a chip, so save it for parts rated to take it).

Delete Component, further down the same context menu, removes the PSU if you want to start over.

Add an LED

Back in the palette, open **COMPONENTS ▶ LEDs** and click **LED**. A small LED ghost follows your cursor, already colored with your "Default LED color" setting (Settings ▶ Appearance — red out of the box). Move it over a free row of breadboard holes and click to seat it — an LED's two legs land in adjacent holes on the same row, anode first. (Press **F** while the ghost is in hand to flip which leg is the anode, or **R** to stand it up on two free ends instead of a footprint row.) To change an LED's color afterward, right-click it and choose **Properties...**

Give an LED a series resistor, just as you would on a bench. Wired straight between a supply rail and ground it has nothing to limit the current, and the simulation shows it **burnt** rather than lit. One resistor anywhere in the loop is enough — including a single one in a display's common leg, which protects all of its segments at once.

Wire it up

Press **W** (or click **Wire** in the toolbar) to arm the wire tool. Click a free hole or terminal to anchor the first end, then click a second free point to lay the wire — a rubber-band preview tracks your cursor between clicks, and it turns red over an illegal target. The tool stays armed, so you can lay several wires in a row without re-arming it; press **Esc** to cancel a pending wire, or **Esc** /click **Wire** again to disarm the tool.

Lay two wires:

1. **PSU +** → **the LED's anode hole** (or a rail hole in the same row/column as the anode, then the anode's row completes the path through the board).
2. **PSU -** → **the LED's cathode hole**, the same way.

The active wire color is shown as a dot on the **Wire** button; click the dot to open the color palette and pick a different one — the color you pick stays active for every wire you lay afterward, so use different colors for positive and return paths if you want them easy to tell apart at a glance.

Run it

Press **Run** in the header toolbar (or **Space**) to start the simulation. The engine traces power from the PSU, resolves the electrical net at every hole, and settles the circuit — the LED should light immediately, since anode-high through cathode-low is exactly what a driven LED needs.

While running, editing is locked (you can't move parts or lay new wires), but the connectivity **probe**, the **logic analyzer**, and the **build guide** all stay live. Press **Stop** (or **Space** again) to freeze the simulation and return to editing.

If the LED doesn't light, double-check both wires land on the correct terminal (**+** / **-**) and hole, and that the PSU is on and not showing a damage badge.

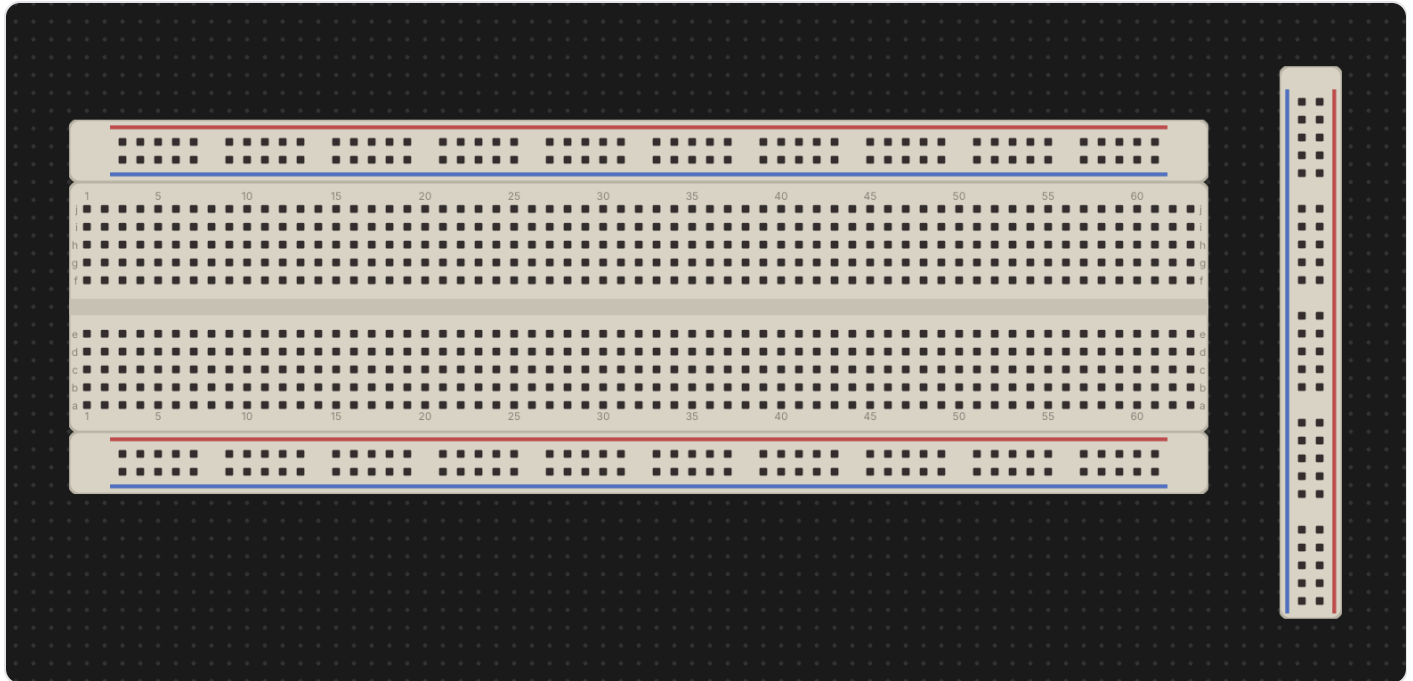
What next

- **The Desk & Breadboards** — pan/zoom, breadboard kits, strips and rails, snapping and grouping multiple boards together.
- **Chips & Components** — the parts palette in depth: DIP chips, discretes, placement, and rotation.

- **Wiring, Nets & Buses** — the wire tool in depth, cross-board wires, colors, and multi-bit buses.
- **Power & Clock Sources** — PSU voltage and the 12 V damage rule, plus clock sources for sequential circuits.
- **Running a Simulation** — Run/Pause/Step, the settle model, and how live views (LEDs, chip badges) work.

The Desk & Breadboards

The desk is Chip Hippo's infinite, pannable, zoomable workspace — the place every board, chip, wire, and power source lives. This page covers how to move around the desk and how breadboards actually work under the hood: they are not single parts but individual **strips** — a pin-board plus power rails — that snap and mate together the way real boards do on a bench.



Pan, zoom & fit to screen

The desk has no edges — pan and zoom to work at whatever scale suits the circuit in front of you.

- **Pan:** click-drag an empty patch of desk, or two-finger scroll on a trackpad.
- **Zoom:** scroll/pinch to zoom, centered on the pointer, or use the **+** / **–** buttons in the zoom control (bottom-right of the desk). Keyboard: `Option+Cmd/Ctrl+=` (zoom in), `Option+Cmd/Ctrl+–` (zoom out), `Option+Cmd/Ctrl+0` (reset to 100%). **Option is what makes these the desk's:** the same keys *without* it resize Chip Hippo's own text instead (see [Settings](#)).
- **Fit to screen:** `Cmd/Ctrl+F` frames everything currently on the desk in the viewport — the fastest way to find your circuit after panning away. It also slides the whole design back around the middle of the desk, keeping a layout that has crept a long way out from drifting further; that part is a document edit (one `Cmd/Ctrl+Z` puts it back) and is skipped while the circuit is running.
- **Zoom out full:** `Cmd/Ctrl+Shift+F` zooms all the way out, useful for locating a part on a very large or sprawling layout.

A faint **dot grid** marks the 0.1 in hole pitch under everything you place; it coarsens at low zoom and disappears entirely once you're zoomed out too far for individual holes to matter. Zoom and pan position are remembered between sessions.

The desk padlock

A padlock sits in the desk's **top-right corner**, across from the tab strip. Click it (or press `Cmd/Ctrl+L`) to shut it, and the **scroll wheel stops reaching the desk**.

It exists for one input in particular. A Magic Mouse — and some trackpads — reports a scroll from a finger merely resting on the surface, so the wheel is the one thing that moves the desk without being asked to; midway through seating a chip, that's a circuit that slides out from under you. Everything else still works while it's locked: drag-to-pan, the zoom cluster, the keyboard shortcuts above, and **Fit**. It's the wheel that's locked, not the camera.

The padlock's shape changes rather than just its colour — an open shackle when the wheel is live, a shut one when it isn't. It's **saved with the project**: shut it, save, and the project reopens with the wheel still locked. Because it's part of the file, changing it counts as an unsaved change, like renaming a desktop. A new project starts with it open. The schematic view has a wheel of its own and no padlock over it.

Breadboards are strips

A real solderless breadboard is not one molded part — it's a centre **pin-board** with one or two **power-rail** strips dovetailed onto its top and bottom edges. Chip Hippo models this literally: each strip is its own object on the desk, and what looks like "a breadboard" is a small **kit** of strips placed together in one action.

The palette offers three assembled kits:

- **Full 830** — a full-length pin-board with a rail strip above and below.
- **Half 400** — the same layout at half length.
- **Tiny 170** — a bare pin-board only; the real 170-point part ships with no rails at all, so this kit is a single strip.

On a pin-board, rows run top to bottom as `j i h g f`, then the **trench**, then `e d c b a`. The trench is the gap down the centre that isolates the top half of each column from the bottom half electrically — a DIP chip **straddles the trench**, with one row of pins in `f` and the other in `e`, exactly as it would seat on a real board. A power-rail strip carries both a `+` and a `-` line, each one continuous connection along its whole length, independent of every other strip.

See [Chips & Components](#) for how chips and discretes seat into the grid, and [Power & Clock Sources](#) for feeding a rail from a PSU.

Kits vs. loose strips

Below the assembled kits, the palette also offers the individual strips on their own — a bare **Full pin-board**, a bare **Half pin-board**, a spare **Full power rail**, and a spare **Half power rail**. Reach for these when a board shipped without enough rails, when you want a rail somewhere a kit wouldn't put one, or when you're building up a custom layout strip by strip. Placing, ghosting, and overlap checking all work identically whether you're dropping a whole kit or a single loose strip.

Snapping & mating

Drop a strip within a couple of tenths of an inch of another strip it can dovetail with — matching width when stacked, matching height side by side, meeting flush with no gap — and it snaps into place automatically. This is the same **magnetic pull** whether you're dragging a placed strip or holding a fresh one from the palette; a whole kit snaps as one piece, and the pull only ever engages when the snapped position is still legal (it will never pull a strip into an illegal overlap).

Two strips that end up flush **mate**: they join a shared **group** and from then on drag together as one rigid unit, just like a real breadboard's rails stay attached to its pin-board when you nudge it on the bench. A kit arrives pre-grouped; anything you drop flush against an existing board joins its group (or merges two groups into one, if it bridges a gap). When several strips are mated, clicking any one of them highlights the whole set — the outline traces the union of the group's strips, so flush neighbours read as one board rather than showing a seam between them.

Selecting things

Clicking a board, part, wire, bus or note selects that one thing, and clicking empty desk clears the selection. There are two ways to select **several** at once, and they are complementary:

- **Shift-drag a marquee** anywhere on the desk. It *replaces* the selection with everything the box encloses **completely** — any board wholly inside, any part whose every pin is inside, and any wire with **both** ends inside. Half-covered things are left out on purpose: a wire with one end outside the box would have to be cut to come along.
- **Cmd -click** (**Ctrl** -click on Windows and Linux) an individual item to *add* it to the selection, or to take it back out if it's already in. This is how you pick up the four chips you actually want without drawing a box around the eight that are next to each other, and how you drop the one thing a marquee caught that you didn't mean.

`Cmd/Ctrl` -clicking a **board** takes its whole snapped group, the same set a plain click selects. Clicking a **bus** takes all of its member wires — a bus is bookkeeping over wires, and the wires are what a delete or a copy acts on. Clicking **empty desk** with the modifier held leaves the selection alone rather than clearing it: an add that landed on nothing has nothing to add.

Notes and labels are the one thing a multi-selection can't hold, so `Cmd/Ctrl` -clicking one does nothing; a plain click still selects it.

On macOS this is `Cmd`, not `Ctrl` — the modifier the rest of the platform's lists use. `Ctrl` -click there is the system's secondary click, and it keeps that job: it still opens an item's context menu.

Either way the result is one selection: `Delete` removes all of it in a single step, `Cmd/Ctrl+C` copies it (as a whole design, if it includes boards — see [Projects & Desktops](#)), and `Esc` clears it. `Cmd/Ctrl+A` selects the entire desk.

A selection also moves as one. Grab any part in it and the whole set travels by the same amount, keeping the arrangement you built; hold **Option** as you press and the wiring comes too — every wire end, and every resistor or LED leg, plugged into any member. The drop is all or nothing, and the move is one `Cmd/Ctrl+Z`. See [Moving several parts at once](#) for the whole of it.

Boards are the exception, and it's the one thing to know before you drag a mixed selection. A **board** in the set makes a press on a part do nothing at all — strips have their own drag, which carries everything seated on them under mating and overlap rules a part re-seat knows nothing about — and the selection is left untouched so you can `Cmd/Ctrl` -click the board back out. Grabbing the **board** instead drags that board's own group (and everything seated on it) in the usual way, replacing the selection as any plain click does.

Nothing can be selected while the simulation is **running** — the topology is frozen, so a selection would be one you couldn't act on.

Groups & breaking a snap

Grabbing a mated strip normally drags the **whole group** together. To pull just part of a group apart, hold a modifier while you start the drag:

- **Option-drag** — moves the **reachable run forward** from the strip you grabbed: itself plus every strip mated to it through a below/right edge.
- **Option+Shift-drag** — moves the run **backward** instead, following above/left edges.

Either way, only strips still reachable through the group come along; a strip that merely happens to sit flush elsewhere in the layout is left behind. When you drop a torn-off run, both the piece you moved and

what's left of the original group are re-evaluated — each is split into fresh groups based on what's still actually mated within it, so the two halves never end up sharing a group id after the break.

Rotating a rail

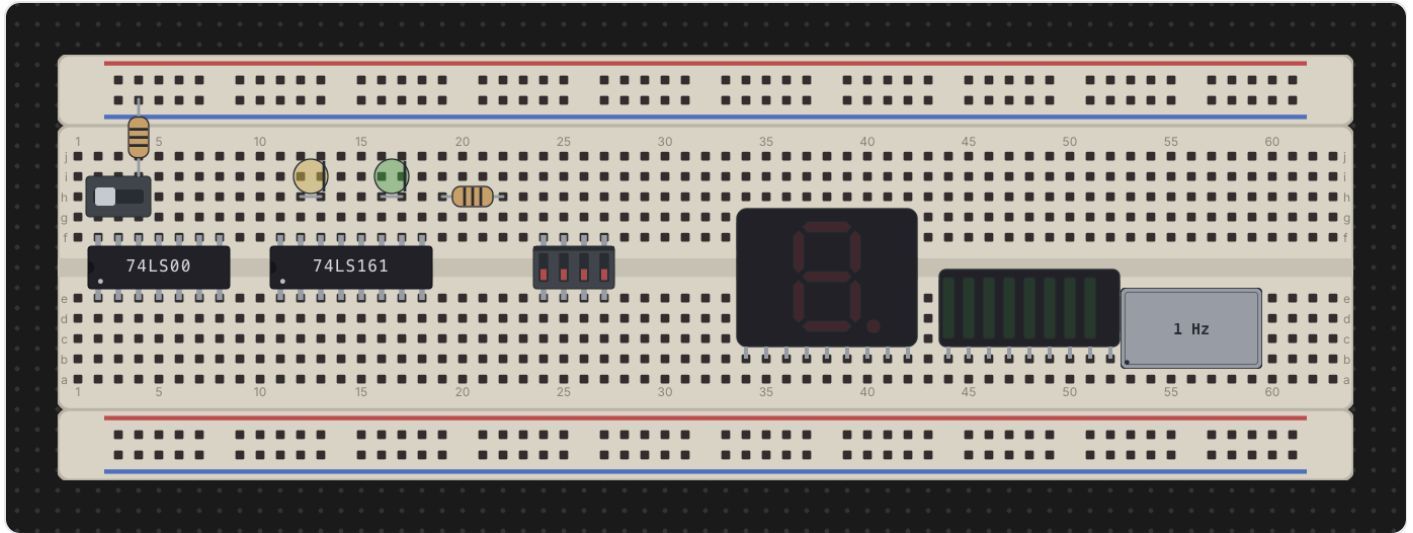
Power rails can rotate; pin-boards cannot. A pin-board's trench (and every DIP seated across it) only makes sense in one orientation, so it's locked at 0°. A rail strip is just two parallel lines of holes, so it reads the same standing on end — turned 90° it becomes a vertical **signal bus** you can run alongside a board and tap into at any point along its length.

Press **R** while a rail is in hand (mid-placement, before you click it down) to cycle its rotation through 0°/90°/180°/270°. Once a strip is placed its angle is fixed; to change it, pick it up again. Rotating doesn't change anything electrically — a rail is one continuous node however it's turned — it only changes which way the strip's footprint runs on the desk.

Next: [Chips & Components](#) for populating a board, or [Wiring, Nets & Buses](#) for connecting everything up.

Chips & Components

Everything that isn't a breadboard strip lives in the **Parts** palette on the left: 74xx logic chips, CPUs, 65xx interface chips, memory chips, switches, LEDs, displays, resistors, oscillators, and power/clock bricks. This page covers finding a part, seating it on a board, and the (surprisingly varied) ways different parts rotate and flip once they're down.



The parts palette

The palette opens with every section collapsed, grouped by function:

- **BOARDS** — the breadboard kits and loose strips, pinned at the top (see [The Desk & Breadboards](#)).
- **CHIPS** — every 74xx logic family, folder-grouped, ending with the **Interface** group (the 65xx PIA/VIA) and the **PROCESSOR** group (the W65C02 and Z80A CPUs).
- **COMPONENTS** — **Switches**, **Resistors**, **LEDs**, **Displays**, **Oscillators**, and **Power**, in that shelf order.
- **Memory** — the ROM/RAM chips, pulled out of CHIPS into a top-level group of their own.
- **ANNOTATIONS** — labels and notes (see [Probing & Net Names](#)).
- **SIGNALS** — bench stimulus buttons, pinned at the bottom (see [External signals](#)).

With the tray shut, the strip it leaves on the desk's edge carries one icon per section in the same order — click one to open the tray on that section alone. To keep the tray down to one section all the time, turn on **Auto-close tray folders** in [Settings](#): opening any folder then closes the rest.

The tray's right edge is a drag handle: pull it out to give long part names more room, and the width you leave it at is remembered.

Type in the **Filter parts...** box to search by id, title, or description — matching, it forces every group open so results aren't hidden behind a collapsed folder. Click any entry to arm placement — a ghost of the part follows your pointer until you click it down on a board, or press **Esc** to cancel.

Placing a DIP chip

A DIP chip always straddles the trench: half its pins seat in row **e**, the other half in row **f**, running the standard counterclockwise DIP numbering — pin 1 at the anchor hole in row e, pins continuing left to right along e, then wrapping back right to left along f. The **notch** end of the chip (or the dot beside pin 1) marks pin 1 and always faces left. Move the ghost over a pin-board and it snaps to the nearest legal seat; it turns red if the seat is already occupied or falls off the edge of the board.

Because a chip's footprint always occupies rows e/f no matter how it's turned, placing one is a matter of picking the column — there's no click-to-rotate step while placing a chip the way there is for a rail or a resistor; instead, rotation happens afterward (see below).

Placing a discrete

Most discretely — slide switches, push buttons, toggle buttons, LEDs (in their default horizontal form), single-digit 7/8-segment displays, and LED bars (bar8) — are **linear**: they seat along a run of adjacent holes in any single grid row (any of **a – j**), not just rows e/f. Drop one anywhere its footprint fits and every free hole underneath it is available.

A few parts don't fit that linear model:

- **bar8iso** — the isolated 8-segment LED bar — is packaged as a 16-pin DIP, so it straddles the trench exactly like a chip: anodes A1–A8 in row e, cathodes K1–K8 in row f.
- **DIP switch banks** (**sw-dip1**, **sw-dip2**, **sw-dip4**, **sw-dip8**) are likewise DIP-packaged (2/4/8/16 pins for 1/2/4/8 switch positions), straddling the trench the same way: each position's two facing pins — one in row e, one in row f — are its own independent SPST switch.
- **Oscillator cans** (**osc-full**, **osc-half**) are rigid four-cornered shapes rather than a line of pins — a full can is 7 holes by 4, a half can 4 holes square, with legs only at the four corners. A can can seat anywhere on the grid, including straddling the trench, since its shape (not a row) determines its footprint.
- **Character LCD modules** (**lcd16x2**, **lcd20x4**) are linear — a 16-way header along 16 adjacent holes in one row — but the module itself is much bigger than that row. See below.

Character LCD modules

The **Displays** group holds two HD44780 character-LCD modules: a 16×2 (the standard 1602A) and a 20×4 (the 2004A). They're drawn to their real sizes — 80 × 36 mm and 98 × 60 mm of PCB — and their screens are **live**: run the circuit, drive the module, and the characters appear on the glass.

Both plug in through a **16-way header along one row**, and the pin assignment is identical between them, so what you learn wiring one applies to the other. The header runs along the module's **top** edge, which means the body hangs **below** the row it plugs into — so seat one on a **bottom row (a)** and it clears the board it's plugged into rather than covering it. The placement ghost shows you the whole module, so you can see this before you click.

Driving one is the ordinary HD44780 parallel bus: put a command or character code on **DB0 – DB7**, set **RS** (0 = instruction, 1 = data) and **R/W** (0 = write), and pulse **E** — the byte latches on **E**'s falling edge. Wire **VDD / VSS** to a 5 V rail. **V0** (contrast) and **A / K** (backlight) are present on the pinout but cosmetic here. During a *read* the module drives **DB0 – DB7** itself, so tri-state anything else sharing that bus.

Both modules show the same controller datasheet in their pin-assignments window — it's one document, because **RS / R/W / E**, the bus and the address maps are the controller's and are identical across the two sizes.

Rotating & flipping

Rotation behavior is **not one rule for every part** — it depends on what kind of part it is. This is the part worth reading carefully.

Chips (R , mid-drag or while selected). A DIP chip's footprint maps onto itself when flipped — same two rows, same columns — so flipping only reverses which physical pin sits where; the chip never has to move. Select a placed chip and press **R** to flip it 180° in place, or press **R** while mid-drag to flip it before you drop it. Either way its pin-assignments window updates to show the new numbering.

bar8iso and DIP switch banks (R while selected). The isolated LED bar and every DIP switch bank are DIP-packaged, so they flip exactly like a chip: **R** turns the part 180° in place, the same holes, only the pin numbering per position reverses (a switch bank's own position states don't move — position 1 is still position 1, just wired to the opposite pins now). Neither has the turn-in-hand behavior of a plain LED — treat them as chips for rotation purposes.

Resistor and LED (R , both while placing and once placed). These two-lead parts start in a horizontal **footprint** form (pin 1 and pin 2 a fixed span apart along one row). Press **R** while the ghost is armed to turn it into a vertical **two-free-ends** form instead: pin 1 stays at the anchor hole, and pin 2 becomes a free lead that can land on any other free hole — including a hole on a different strip entirely, such as reaching up to a power rail. Each **R** press while placing steps the ghost a further quarter turn, cycling

through all four compass directions before repeating. Once the part is placed, select it and press **R** to rotate it 90° at a time — pin 1 stays put and pin 2's lead swings around it, hunting for the next free hole to land in.

Oscillator cans (**R , but the exact step differs by state).** A can spins around its own centre, not around one pin, and the step size changes depending on whether you're still placing it:

- **While placing**, **R** steps the ghost a full 90° quarter-turn each press, so you can hunt through every orientation for one that fits.
- **Once seated and selected**, **R** behaves differently for the two sizes: the square **osc-half** can still steps 90° at a time, but the rectangular **osc-full** can jumps straight from 0° to 180° (and 90° to 270°) — because a non-square footprint only has two genuinely distinct orientations once it's down (rotating it a further 90° would just retrace the same two footprints it already swept through while placing).

LED polarity (**F , while placing only).** Independent of rotation, press **F** while an LED's placement ghost is armed to flip which lead is the anode and which is the cathode, before you click it down.

Moving a seated part — with or without its wiring

Dragging a placed part re-seats the part and **leaves every wire where it was**. That is often what you want while a circuit is still bare, but once it's wired it quietly changes the circuit: the pins land on different column-halves, the wires stay in the holes you laid them in, and what was pin 1's input is now pin 3's.

So, exactly as with pulling a mated strip out of a board group, hold a modifier while you start the drag:

- **Option-drag** — moves the part **and everything plugged into it**. Every wire end sitting in a column-half one of the part's pins occupies comes along, keeping its own row and its offset from the part, so the circuit after the move is the circuit before it. The far end of each wire stays put.

A resistor or LED plugged into the part comes too — by the leg that is connected to it. A leg in one of the part's column-halves is attached to it exactly as a jumper in the next hole along is, so it travels; the other leg stays exactly where it is and the part simply **bends** around it, the way a resistor's legs bend on a real bench. Plugged in at *both* legs, it travels whole instead. (A chip's pins can't bend, so a chip next door is never dragged along — it isn't a lead, it's a body.)

Only what is actually connected to the part comes — a wire or a leg in a column the part merely spans without having a pin there (a push button reaches two holes three columns apart, not the one between them) is left alone, because it was never connected to it. And nothing chases further than one step: what travels lands in the same column-half its pin lands in, so a rider never leaves anything of its own stranded behind it.

To see what would come, hold Option. With a part selected, holding Option rings every wire end and every leg that would travel with it, and releasing Option puts the rings away — so you can check before you commit to the drag rather than discover it during one. Something connected at *both* ends gets two rings, so a resistor that will travel whole reads differently from one that will bend; something connected at one end gets one ring, on the end that moves. A part with nothing attached simply shows nothing.

The drop is **all or nothing**. If any of those has nowhere to go — its hole is taken by something that isn't moving, it would run off the end of the strip, or the bend would squeeze a resistor's legs closer than its body is long — the part *and* everything it would have carried turn red, and releasing puts everything back. Half a move would silently cut the connections it left behind, which is the very thing the gesture exists to avoid.

A resistor or LED is dragged by its two ends rather than by a footprint, and that makes one distinction: dragging its **body** moves both legs together, so Option carries its wiring exactly as for any other part; dragging **one leg** is a re-bend, and carries nothing — that leg can land in any hole, at any angle, on any strip, so there is nothing for a wire to follow. (An LED lying flat is only one hole wide, so there is no body to grab between its legs; stand it up with `R`, or move it as part of a selection.)

Two details worth knowing:

- Whether the wiring comes is decided when you **press**, not when you let go. Once the part is in hand, the set is fixed.
- Moving **across the trench** takes the wiring over with it, keeping the arrangement. Rows `a – e` and `f – j` are separate nodes, so a wire that stayed in its row would be left in the half its pin had just left — it travels the same number of holes the part did instead, so a wire two holes from the part is still two holes from it, on the same side. Drag far enough that the wiring would run off the edge of the board and the drop is refused; a row nearer the trench fits.
- Moving to **another board** works the same way — the wiring lands in the matching holes over there, and anything the wires reach back to stays where it is. While the part is over the gap between two boards there is nowhere for it to land, so the wiring sits where it really is and turns red until the part is over holes again.

`Shift` is not this modifier: Shift-drag always rubber-bands a selection, including when the press lands on top of a part.

Moving several parts at once

A selection drags as one unit. Marquee a group of parts (Shift-drag), or add them one at a time with `Cmd / Ctrl`-click, then drag any one of them: every selected part travels by the same amount, keeping the arrangement exactly as you built it. Power and clock bricks count as parts here, and come too.

Everything above applies unchanged, just to the whole group:

- **Option** takes the wiring with it — every wire, and every resistor or LED leg, riding *any* member. A wire between two selected parts travels at both ends; one that leaves the group keeps its far end where it is; a resistor plugged into a member bends after it exactly as above.
- Holding Option **rings** every end and leg that would travel, across all of them.
- The drop is **all or nothing**. If one part, or one wire end, or one leg has nowhere to land, every member and everything riding it turns red and releasing puts the lot back — a group that dropped some of its members would be a rearrangement you never asked for.
- The whole move is **one undo step**.

A **board** in the selection is the one case that declines: strips have their own drag, which carries everything seated on them under rules a part re-seat knows nothing about. Pressing a part then does nothing and leaves the selection alone — `Cmd / Ctrl` -click the board back out and drag the parts, or grab the board itself, which drags its own snapped group with everything seated on it and replaces the selection the way any plain click does. And a plain click inside a selection narrows it to the part you clicked, exactly as clicking a part outside one does.

One thing a selection does **not** carry: a **wire** you selected that isn't plugged into any of the parts moving. Wires travel because they ride a pin, not because they are highlighted — so a wire selected on its own (to delete or recolor it) stays exactly where it is while the parts move around it.

Occupancy — one hole, one lead

Every hole on a breadboard — and every terminal on a power/clock brick — holds **at most one lead**, whether that's a chip pin or a wire end. Placing a part checks every one of its derived pin positions against every other part and every wire already on the desk; if any pin would land on an occupied hole, or off the edge of a board entirely, the ghost turns red and the drop is refused. This is the same rule a wire's endpoints follow (see [Wiring, Nets & Buses](#)) — chips, discretes, and wires all compete for the same holes, with no separate bookkeeping for any of them.

A rotated resistor or LED's free lead is the one case where a pin can legally resolve to **nothing** — if you later move or delete the strip under that lead, the part stays exactly where it is and that leg simply floats, unconnected, just as it would on a real bench.

The pin-assignments window

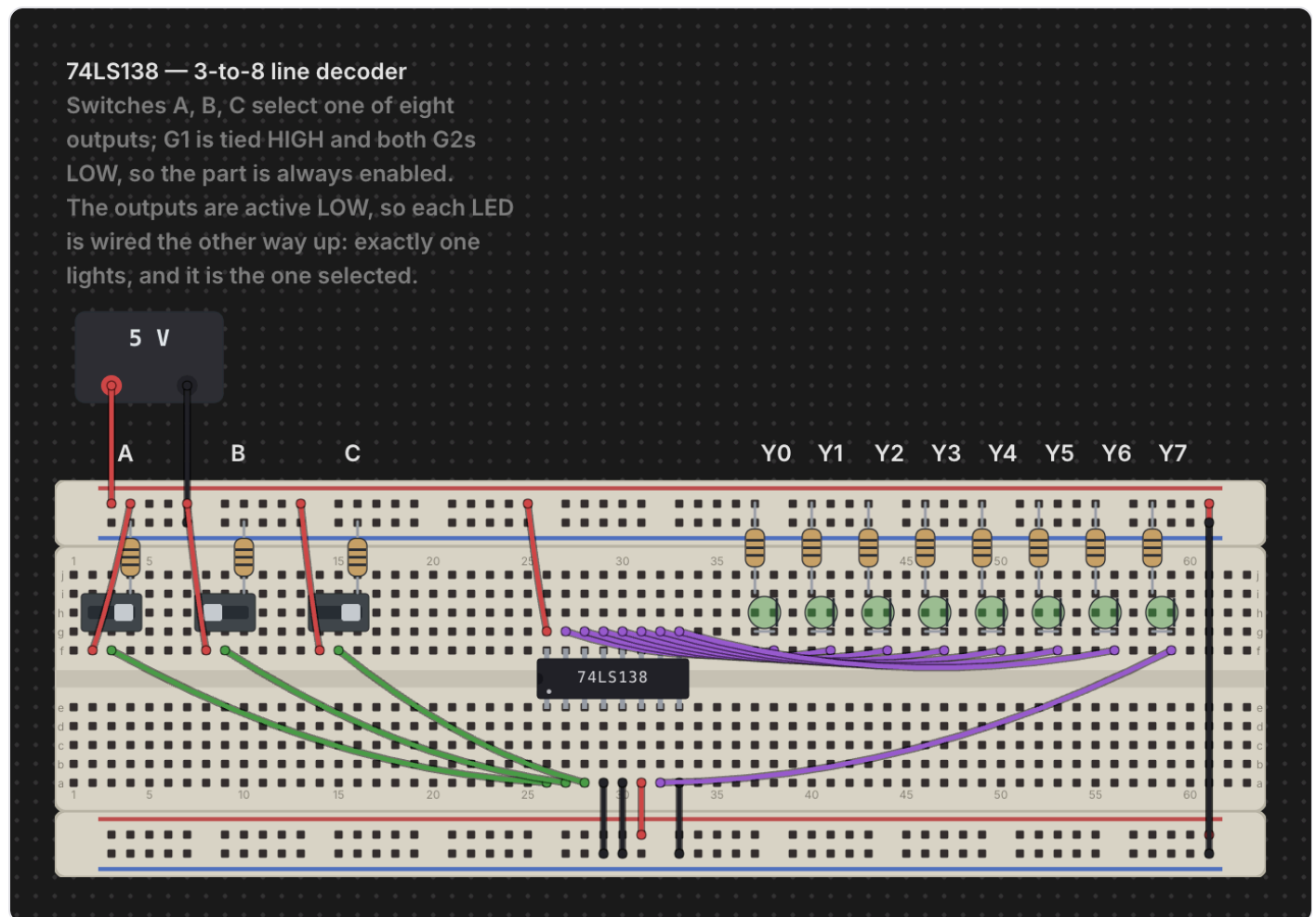
Right-click any placed part — chip, discrete, or brick — and choose **Pin Assignment**, at the top of its context menu, to open its floating pin-assignments window: a diagram of every pin/terminal and, for most chips, a cropped datasheet excerpt below it. A real chip's diagram stays fixed at its canonical layout no matter how you've flipped it on the desk (it matches the physical part, not the placement); a DIP-packaged discrete — `bar8iso` or any DIP switch bank — is the exception: its diagram reflects its

current **R** flip, since it has no real notch of its own. See [The Chip Library](#) for the full detail on what the window shows and how it sources its datasheet crops.

See also: [The Desk & Breadboards](#) for how boards and strips work, and [Wiring, Nets & Buses](#) for connecting components together once they're placed.

Wiring, Nets & Buses

Wires are how you connect the holes, rails, and terminals you've already populated with boards and parts into a working circuit. Chip Hippo's wire tool is click-click (no drag-to-draw), every wire remembers the two *addresses* it connects rather than raw screen positions, and a bus lets you lay a whole marching group of wires — a data or address bus — as one gesture. This page covers the wire tool, colors, cross-board wiring, re-routing and deleting a wire, addresses, and buses.



The wire tool

Press **W** (or click **Wire** in the toolbar) to arm the wire tool. With the tool armed:

1. Click a free hole or terminal to anchor the first end — a rubber-band preview then tracks your cursor.
2. Click a second free point to drop the wire between them.

The tool stays armed after a wire is committed, so you can lay a whole chain of wires without re-arming — each new wire starts fresh, so it does **not** automatically continue from the last endpoint; just click a

new starting hole. A hover ring shows the point under your cursor and turns red the moment it's over an illegal target (already occupied, or the same point you started from) — the rubber band tints red too, so a bad landing is obvious before you click.

Press **Esc** to cancel a pending (anchored-but-not-yet-dropped) wire without leaving the tool, or **Esc** /click **Wire** again to disarm it entirely. **M** also disarms whichever of the wire, bus, or probe tools is currently armed — one key to "put the tools away." The wire tool (like the bus and placement tools) is unavailable while a simulation is running; editing is locked until you press **Stop**.

Wire colors

Chip Hippo ships eight wire colors: **red, black, blue, green, yellow, orange, white, and purple**. The active color is shown as a swatch dot on the toolbar's **Wire** button — click the dot itself to open the palette and pick a different one (picking a color sets it *without* arming the tool, so you can set up a color before you start wiring). Clicking the word **Wire** beside it still arms the tool as usual. While the tool is armed you can also press a digit **1 – 8** to jump straight to a color (1 = red, 2 = black, and so on through the list above). The color you pick **stays pinned** between wires — laying a chain of jumpers keeps whatever color you last chose, so pick red for a supply run and black for ground and they'll stay that way until you deliberately switch.

You can also recolor a wire after the fact: right-click it and choose **Properties...**, which offers the same eight swatches alongside the wire's name, description, layout method, and the drawing of the wire itself described below. (**Delete Component** on that menu is how a wire is removed.)

Wire length

The last thing in a wire's **Properties...** dialog isn't a field at all — it's a **picture of the wire**, drawn across the card in its own colour with the sleeve stripped back at both ends to bare tinned lead, and a dimensioned line underneath giving its length in centimetres.

It answers the one question the desk can't. On the desk a wire is a curve between two holes at whatever zoom you're at, so its length is unreadable — and its length is exactly what you need before cutting one off a spool.

Two things about that figure:

- It's the **cut** length, not the run. A real jumper doesn't stop at the surface of the board — it has to reach *into* both holes — so a wire crossing a single 0.1 in pitch is about 13 mm of wire, not 3. The sleeve in the drawing spans the run; the bare tips are the two stripped ends; the dimension spans the lot.

- The length follows the **drawn shape**, not the straight-line distance: the sagging curve's own arc for a direct wire, the polyline through the waypoints for a routed one, and for a bus member its lead plus the whole ribbon plus its far lead — a conductor inside a ribbon cable is as long as the cable, however short the ends sticking out of it are.

It's the same measurement, and the same wording, as the **Wires** cutting list in the [build guide](#) — so a wire can't read one length here and another there. Recolor the wire in this same dialog and the drawing repaints to match.

Cross-board wires

A wire's endpoints don't care which board or brick they land on — you can run a wire from one breadboard to another, from a board to a PSU terminal, or between two separate strips, exactly as freely as within one board. When you drag a board that a wire is attached to, the wire's other end stays put and the whole wire stretches and re-sags live, riding the move; nothing needs to be re-laid. Deleting a board cascades: any wire with an end seated on that board is removed along with it.

Moving a **part** is different, because a part's pins can land somewhere its wires can't follow: an ordinary part drag leaves every wire in the hole you laid it in, which on a wired part quietly rewires the circuit. Hold **Option** as you press and the wire ends plugged into it come along instead — for one part, or for a whole selection of them. See [Chips & Components](#) for the rules, and for what the Option-held preview rings show you before you commit.

Re-routing & deleting a wire

Grab a wire two different ways:

- **Its end cap** — drag just one endpoint to a new hole or terminal, re-routing that end while the other stays put. The end follows the cursor and snaps onto a free hole once it's within about a hole's reach of one. A hover ring and red/legal tint on the dragged end work exactly like placing a fresh wire; release with no free hole in reach and the end snaps back to where it started.
- **Its body** — drag anywhere along the wire itself to translate the whole wire rigidly, keeping its length and orientation and just sliding both ends together onto a new pair of holes. If either landing point isn't free, the wire snaps back on release.

To delete a wire, click it to select it, then press **Delete** or **Backspace** (the same shortcut removes whatever's currently selected — a part, a board, or a wire).

Wire layout: direct or routed

Every wire is drawn one of two ways, its **Layout Method** — right-click a wire and choose **Properties...** to switch it:

- **Direct** (the default) — the wire sags gently from one hole to the other, the way a real jumper lead does. There is nothing to position: the shape follows the two ends.
- **Routed** — the wire runs as straight lines, and *you* decide where it goes. This is the one to reach for when a run would otherwise cross the middle of a chip, or when a tidy right-angled layout reads better than a fan of curves.

A routed wire starts as a single straight line. To bend it:

- **Press anywhere along the wire and drag.** A new point is added where you grabbed it, and the wire now runs start → point → end. Repeat to take it around whatever is in the way — up to twenty points per wire.
- **Hover an existing point** and its knob appears; drag it to move that bend somewhere else.
- **Drop a point onto a neighbouring point, or onto either end of the wire,** and it merges away — that's how you take a bend back out. The point you are dragging turns red as it lands, so you can see the merge before you release.
- **Drop an end cap onto one of the wire's own points** and that point is absorbed: the wire now reaches where the bend used to be, so the bend has nothing left to do.

Because a routed wire's body drag is spoken for, it has no rigid whole-wire translate — move it by its end caps instead. Switching a wire back to **Direct** deletes every point it carried and redraws it as the plain sagging curve; switching to **Routed** straightens it out, ready for points. Either way it's one undo step (`Cmd/Ctrl+Z`), as is every individual bend.

Settings ▶ Appearance ▶ **Wire layout** picks which of the two a *newly laid* wire gets. It's read when the wire is laid, so changing it never disturbs wiring already on the desk. Circuits built by the [AI builder](#) always arrive direct, whatever that setting says — the builder places holes, not hand-drawn routes.

Addresses

Wires never store pixel positions — they store **addresses**, the one cross-module currency for anything wireable. An address is `<ownerId>.<point>` :

- `bb1.a12` — a grid hole (board id `bb1` , hole `a12`).
- `bb2.+7` — hole 7 on a rail strip's `+` rail.
- `psu1.+` — the `+` terminal of a PSU brick.

A wire's `from / to` are just two of these addresses, plus a `color`. One hole or terminal holds at most one lead — one pin or one wire end — so the occupancy check that keeps the wire tool's hover ring legal/illegal is the same one that governs seating a chip or a discrete. Because everything resolves through addresses rather than coordinates, a wire keeps working correctly no matter how the boards around it get rearranged.

Buses

For a wide, tidy bundle — a data or address bus — press `B` (or click **Bus** in the toolbar) to arm the bus tool instead of laying eight wires by hand. It's click-click like the wire tool: anchor a start hole, then click a second point to lay the whole run in one go, rendered as a single fat band rather than a fan of individual wires.

Before you click, set the **width**. The circled number on the toolbar's **Bus** button shows the active one, and clicking it opens the presets in a row: 2, 3, 4, 5, 6, 7, 8 and 16 bits, the current one ringed. Picking one sets the width without arming the tool, so you can set up before you start. From the keyboard — while the bus tool is armed — `2 - 8` lay a bus that many bits wide (`4` → `D[3:0]`) and `1` lays the 16-bit bus (`D[15:0]`); no single digit can spell 16, and the widest bus is worth the first key. The width names the bus too (`D[3:0]`); rename a placed one from its context menu.

Landing the second click on a **chip pin group** (a catalog-defined bus of pins, like a chip's data lines) fans the bus onto those pins in bit order, instead of marching it down a second run.

Both ends have to fit. A bus claims one hole per bit at each end, so as you move the cursor every hole it would take is ringed — in the hover color when they're all free and on the strip, in red when they're not, and a red set can't be clicked. Aim an 8-bit bus five columns from the end of a board and you'll see five red rings instead of eight, which is the whole explanation: the run doesn't fit there. That check runs on the **first** click as well, so a start that could never work is refused where you make it rather than looking like the far end is at fault.

A bus is metadata layered over the individual wires it lays — right-click a placed bus to **Rename**, **Recolor** (which recolors every member wire too), **Un-bundle** (keep the wires, drop the bus grouping), or **Delete bus**

- **wires**. Drag a bus by its band to move every member wire together.

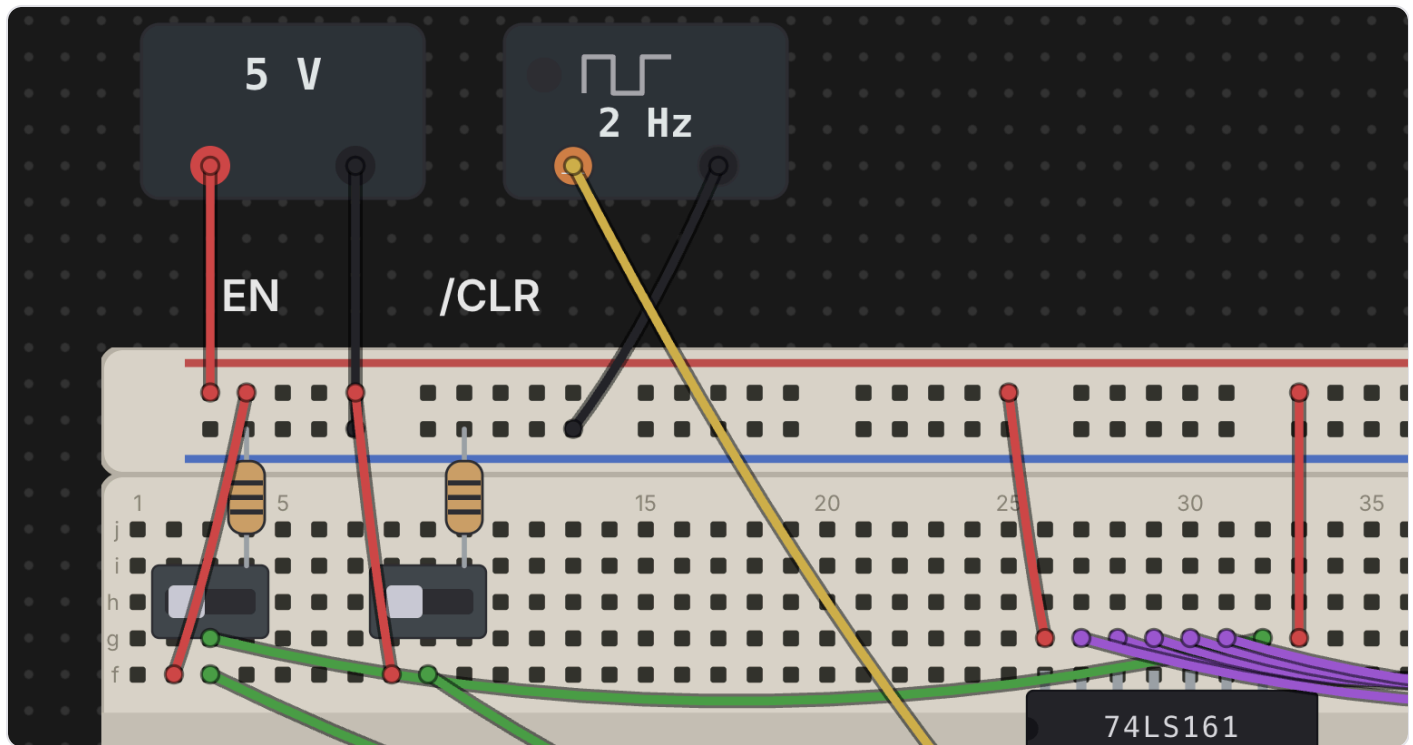
See also

- [Chips & Components](#) — placing the parts a wire connects.
- [Power & Clock Sources](#) — PSU and clock brick terminals, which wires reach exactly like board holes.
- [Probing & Net Names](#) — inspecting the nets your wires and buses form, and naming them.

Power & Clock Sources

Every circuit needs somewhere to draw power from and, for sequential parts, a signal to step them along. Chip Hippo gives you two kinds of desk-level **bricks** for this — a **power supply (PSU)** and a **clock source** — neither of which seats on a breadboard. They sit loose on the desk with their own addressable terminals, and you wire those terminals into a board's rails (or straight to a chip's pins) just like any other wire run.

There is a third kind of source, and it does not live on the desk at all: **external signals**, the bench stimulus buttons pinned down the right-hand edge of the viewport. They are covered at the end of this page.



PSU bricks

Add a **Power supply** from the parts palette (**Power** group) and drop it anywhere on the desk — it isn't tied to a board. It draws as a small body with a voltage badge and two terminal pads: a red **+** and a black **-**. Those pads are addressable wire endpoints, `psu1.+` and `psu1.-` for the first PSU you place, exactly like a breadboard hole — click-click a wire from each terminal into a power rail (or directly to a chip's VCC/GND pins) to energize a circuit.

A PSU has no on/off switch of its own — it's always "live" the moment the simulation is running; what matters is which voltage it's set to and what it's wired into.

Choosing a voltage — and the 12 V damage rule

Right-click a PSU brick and choose **Properties...** to pick its voltage: **3 V**, **5 V**, or **12 V**. It's a live setting — the dropdown stays available and applies immediately even while the simulation is running, so you can change voltage on the fly and watch the effect.

- **5 V** — normal operation. A chip whose VCC net carries a 5 V supply and whose GND net is properly grounded runs exactly to its datasheet behavior.
- **3 V** — underpowered. The chip is inert — every output floats/reads as if disconnected — but nothing is harmed. Useful for demonstrating what an underpowered chip looks like without any risk.
- **12 V** — **damage**. This is Chip Hippo's "magic smoke" rule: any chip (or oscillator can, which is powered the same way) whose VCC net sees 12 V is immediately marked **damaged** and goes inert for the rest of that run, independent of anything else on the net. It stays dead while the run lasts — a failed chip doesn't recover while the power is still wrong — and **Stop** makes it whole again.

A chip can also come up **reversed** — a PSU  on its VCC pin's net at the same time as a PSU  on its GND pin's net — which is reported separately from plain unpowered/underpowered, since it specifically means the supply leads are swapped.

Live chip health (powered / underpowered / reversed / damaged) shows as a badge on each chip while running — see [Running a Simulation](#) for how those badges and the rest of the settle model work. This page only covers what puts a chip into each state.

Recovering from a 12 V mistake

Press **Stop**. Every damaged chip on the desk is restored, and you can rewire and run again. You don't lose the part, and you don't have to delete and re-place anything.

12 V is meant to be *noticed*, not to cost you the chip: the red badge and the warning are the point, and they've done their job by the time you stop. Wiring a supply to the wrong rail is a mistake about the circuit — one you can see and fix — so it stays fixable. Find where 12 V is reaching a VCC net (the [probe tool](#) is the quick way), correct it, and press **Run**.

Clock sources

Add a **Clock source** from the palette (**Power** group) for a free-running or manually stepped square wave to drive a sequential chip's clock pin. Like a PSU, a clock brick is desk-level — it doesn't seat on a board — and exposes two addressable terminals: **out** and **gnd** (`clk1.out` / `clk1.gnd`). Wire **out** to a chip's clock input and **gnd** to your circuit's ground.

Right-click a clock brick and choose **Properties...** to set its rate:

- **1 / 2 / 5 / 10 / 20 / 50 / 100 Hz** — free-running. Once the simulation is running, the brick toggles its `out` level on its own at the chosen rate; a small lamp on the body lights while the output is HIGH. The slow end is for watching a single edge land; the fast end is for letting a counter or a CPU actually get somewhere while you watch. Above about 20 Hz the lamp and any LEDs on the circuit blur into a steady glow — that's the point at which the **logic analyzer** becomes the way to see what happened.

While the simulation runs, a free-running clock also shows a small **■** button in its top-right corner. Click it to hold **that one clock**: it keeps whatever level it was at — its lamp and everything it drives stay as they were — while the rest of the circuit, and every other clock, carries on. The button turns into a **▶**; click it again to set the clock going. A clock's own pause belongs to the run, like its phase: **Run** always starts every clock going, **Step** passes a paused clock by, and resuming the transport after **Pause** leaves it held.

- **Manual** — click-to-toggle. No timer runs it; instead, while the simulation is running, clicking the brick's body flips `out` from LOW to HIGH (or back) once per click — handy for single-stepping a counter or flip-flop by hand and watching each edge land.

The rate dropdown, like the PSU's voltage one, is a live setting — it applies immediately and stays available while running, so you can retune a clock's speed mid-simulation.

An **oscillator can** (a discrete part that seats directly on a board rather than as a desk brick) behaves the same electrically — it's a free-running square-wave source powered like a chip, with its own rate field in its Properties dialog — but it only ever free-runs; a real crystal has no click-to-toggle pin, so it has no manual mode.

The transport drives the edges

Free-running clocks (and oscillator cans) don't tick on their own outside a simulation — their edges are driven by the **Run/Pause/Step/speed transport** in the header, described fully in **Running a Simulation**. Briefly: **Run** (`Space`) starts every free-running clock at its configured rate; **Pause** freezes them in place without stopping the simulation; **Step** advances every free-running clock by exactly one half-period and re-settles the circuit, useful for watching a sequential chain edge by edge; and the **speed** control scales every free-running clock's rate together (it has no effect on a manual clock, which only ever moves on a click). A manual clock only responds to clicks while the simulation is actually running — stopped, its body is inert like everything else on the desk.

External signals

A **signal** is bench stimulus: a labelled button pinned to the right-hand edge of the desk, and a pointed **flag** whose tip plugs into any hole on any breadboard. Pressing the button injects a level at that hole. Nothing about a signal takes up board space, and — unlike a push button, which you can only hold one of at a time — **you can assert several signals at once**.

Add one from the parts palette's **SIGNALS** section: pick *Signal*, then click anywhere on the desk. The click position does not matter; the button always goes to the next free slot on the rail, with its flag waiting beside it.

Each button's coloured dot shows the **key** that presses it — **1** for the top button, down to **9**, then **0** for the tenth — and the flag prints the same key inside its body. That key is how you tell a flag's button at a glance. Selecting a flag on the desk also outlines its button on the rail.

Drag the flag onto a board and its point snaps into the hole under the cursor. Press **R** while it is selected to turn it a quarter-turn about that point, so it can lie in whichever direction has room. A flag claims its hole exactly like a wire end or a chip pin does, so nothing else can use it while the flag is there.

To take a flag off the board, drag it onto bare desk, or right-click it and choose **Remove Signal**. Either way the flag goes back beside its button on the rail, and its position and rotation are cleared, so the next drag starts fresh. The signal itself stays. **Delete Signal** on the same menu (or the **Delete** key while the flag is selected) removes the signal altogether, button included.

Momentary, toggle, and the resting level

Right-click a flag (or a button) for **Properties...**:

Field	What it does
Name	The label shown on the button
Description	The button's tooltip
Color	The colour shared by the button's dot and its flag — any of the seven signal colours, even one another signal already uses
Type	Momentary asserts while held; Toggle flips on each press and stays
Default value	The level the signal rests at — Low or High

A placed signal is **always** driving its net. A momentary signal rests at its default and asserts the *opposite* while you hold it, which is what makes "Default: High" the natural setting for an active-low `/RESET` line:

it sits high, and pressing the button pulses it low. A toggle's current level is reset to the default every time you Run, so a run always starts from a known state.

A signal drives at the same strength as a chip output, so two signals arguing over one net — or a signal fighting a chip's output pin — is reported as a conflict, exactly as two chip outputs on one net would be.

Pressing them

While the circuit is running, **1–9 and then 0 press the signal buttons**, top to bottom — the key each button (and its flag) shows. No modifier, so you can hold several down together — **1** and **3** at once is two signals asserted at once. (Those digits pick the wire colour and the bus width while the Wire or Bus tool is armed, but Run puts both tools away, so the two meanings never collide.) Clicking a button with the mouse does the same thing for one signal at a time.

There are **ten signals** per desktop — one per digit key. The palette's *Signal* row goes grey once all ten are in use.

A new signal takes the next colour in the sequence red, blue, green, yellow, orange, white, purple, starting with the first one no signal is using. Once all seven are in use the sequence starts again from red, so the eighth, ninth and tenth signals share a colour with the first three — their keys tell them apart. Deleting a signal frees its colour for the next one you add. The signal colours are the eight jumper colours **minus black**, which is the bench's ground colour: a black flag would read as a ground tie, and its button's dot would all but vanish against the dark rail.

A project saved before black was withdrawn still opens; any signal that was black is given a colour on load, and keeps its name, type, resting level and wherever its flag was planted. Nothing else on the rail is recoloured.

See [Wiring, Nets & Buses](#) for how to route power-rail and clock wiring generally, and [Running a Simulation](#) for how power state and clock edges feed into the settle model and live views.

The Chip Library

The parts palette's **CHIPS** folder holds a broad shelf of 74xx-family DIP logic — everything from a single quad NAND gate up to octal shift registers and 4-bit counters — every one with a datasheet-accurate pinout and real behavior you can wire up and run. Past them, an **Interface** group carries the 65xx peripherals (a PIA and a VIA) and a **PROCESSOR** group carries the CPUs, while a separate **Memory** group sits below for the address-indexed ROM/RAM parts, which get their own dedicated page. This page is a tour of what's on the shelf and how to read a chip's pin-assignments window once you've placed one.

Combinational gates

The basic gate families — the classic 7400-series building blocks:

Part	Description
74LS00	Quad 2-input NAND
74LS01	Quad 2-input NAND, open-collector — outputs on 1/4/10/13, <i>not</i> the classic quad-NAND layout
74LS02	Quad 2-input NOR
74LS03	Quad 2-input NAND, open-collector — the variant that <i>does</i> keep the classic layout (outputs on 3/6/8/11)
74LS08	Quad 2-input AND
74LS10	Triple 3-input NAND
74LS11	Triple 3-input AND
74LS20	Dual 4-input NAND
74LS27	Triple 3-input NOR
74LS30	8-input NAND
74LS32	Quad 2-input OR
74LS86	Quad 2-input XOR

Open-collector parts (`74LS01` , `74LS03` , `74LS05`) pull their outputs low only and assume an external pull-up on a real bench. Chip Hippo models them as plain gates, so they behave correctly without one — but wire the pull-up anyway if you're prototyping something you intend to build.

Alongside them, the inverter and buffer/bus-driver parts:

Part	Description
74LS04	Hex inverter
74LS05	Hex inverter, open-collector
74LS14	Hex Schmitt-trigger inverter
74LS125	Quad tri-state buffer, active-low enable per gate
74LS240	Octal inverting tri-state buffer/line driver
74LS244	Octal (non-inverting) tri-state buffer/line driver
74LS245	Octal bidirectional bus transceiver — the one part in the catalog with true bidirectional pins

Sequential & MSI parts

Everything with internal state, plus the mid-scale-integration decoders and multiplexers that build address/data logic around them.

Flip-flops & latches

Part	Description
74LS73	Dual JK flip-flop, clear
74LS74	Dual D flip-flop, preset & clear
74LS76	Dual JK flip-flop, preset & clear
74LS107	Dual JK flip-flop, clear
74LS112	Dual JK flip-flop, preset & clear
74LS174	Hex D flip-flop
74LS175	Quad D flip-flop
74LS173	4-bit D register, tri-state
74LS273	Octal D flip-flop, clear
74LS75	4-bit bistable (transparent) latch
74LS279	Quad SR latch
74LS259	8-bit addressable latch
74LS533 / 74LS573	Octal transparent latch, tri-state (inverting / non-inverting)

The 74LS73 , 74LS75 , and 74LS76 reproduce their datasheet's **non-standard power-pin placement** — real parts don't always put VCC and GND on the package corners, and neither do these.

Counters & shift registers

Part	Description
74LS90	Decade ($\div 10$) ripple counter
74LS161	Synchronous 4-bit binary counter
74LS169	Synchronous 4-bit up/down counter
74LS193	Synchronous up/down 4-bit counter
74LS164	8-bit serial-in, parallel-out shift register
74LS165	8-bit parallel-in, serial-out shift register
74LS595	8-bit shift register with output storage latch

Decoders & multiplexers

Part	Description
74LS138	3-to-8 line decoder
74LS139	Dual 2-to-4 line decoder
74LS151	8-to-1 line multiplexer
74LS153	Dual 4-to-1 multiplexer
74LS157	Quad 2-to-1 selector
74LS257	Quad 2-to-1 selector, tri-state

Arithmetic, comparison & encoding

Part	Description
74LS47	BCD-to-7-segment decoder/driver
74LS85	4-bit magnitude comparator
74LS148	8-to-3 priority encoder
74LS283	4-bit binary full adder
74LS83	The same adder on its original pinout — VCC on pin 5, GND on pin 12, not the later JEDEC corners
74LS181	4-bit arithmetic logic unit (DIP-24) — 16 logic or 16 arithmetic operations selected by <code>S0</code> – <code>S3</code> and <code>M</code> , with carry generate/propagate outputs for cascading

Interface chips (65xx)

Past the 74xx groups, the **Interface** group carries two Western Design Center 65xx peripherals — both DIP-40, both clocked off `PHI2`, both wired the same way you'd wire them on a real single-board computer:

Part	Description
W65C21	W65C21 PIA (CMOS 6521/6821) — two 8-bit bidirectional ports with per-line data-direction registers, plus four handshake/interrupt lines
W65C22	W65C22 VIA (CMOS 6522) — the same two ports plus two 16-bit interval timers, an 8-bit shift register, and four handshake lines

They're **logic-level**, not cycle-accurate: nothing here models wall-clock timing, so the VIA's timers count `PHI2` cycles rather than seconds.

A few practical notes for building with them:

- **Address one of the peripherals** by holding its chip selects (`CS0` · `CS1` high and `CS2B` low on the PIA; `CS1` high and `CS2B` low on the VIA), picking a register with `RS0` – `RS1` (PIA) or `RS0` – `RS3` (VIA), setting `RWB`, and pulsing `PHI2` — writes latch on the falling edge.
- **IRQB is open-drain** on both peripherals, so give it a pull-up.
- You can drive the bus **by hand** — set the address, selects and `RWB`, then pulse `PHI2` — or wire a CPU to it from the group below.

Processors

The **PROCESSOR** group carries the two 8-bit CPUs, both DIP-40. Each is a full instruction-set simulation, so a program in a ROM or RAM really does fetch and execute one instruction at a time:

Part	Description
W65C02	W65C02S 8-bit CPU — 16-bit address bus (A0 – A15), 8-bit data bus (D0 – D7), <code>RWB</code> , <code>RESB</code> , <code>IRQB</code> / <code>NMIB</code> , and <code>SYNC</code> pulsing high on each opcode fetch
Z80A	Zilog Z80A 8-bit CPU — the same 16-bit address and 8-bit data buses, plus <code>/MREQ</code> , <code>/IORQ</code> , <code>/RD</code> , <code>/WR</code> , <code>/M1</code> , <code>/RFSH</code> , <code>/HALT</code> , <code>/WAIT</code> and <code>/BUSRQ</code> // <code>/BUSACK</code>

They disagree about the clock, and it shows in how you wire them. The W65C02 makes exactly one bus access per `PHI2` cycle, so its address bus advances once per clock. The Z80 does not: an opcode fetch is four T-states with a memory-refresh cycle glued to its back half, a plain read is three, and an I/O cycle is four — so a Z80 instruction takes several clock cycles, `/M1` marks which cycle is the opcode fetch, and `/RFSH` pulses behind it.

A few practical notes:

- **Both power up in reset.** Wire the reset pin to a push button to hold it. The W65C02 then boots from the reset vector at `$FFFC` / `$FFFD` ; the Z80 has **no reset vector at all** and simply starts fetching at `$0000` .
- **Every Z80 control line is active LOW**, which is what the leading slash in its pin names records — the app has no way to draw an overbar.
- **The Z80 gives you real chip-select and strobe lines.** Wire `/MREQ` to a memory's `/CE` , `/RD` to its `/OE` and `/WR` to its `/WE` . That combination matters: during the refresh half of a fetch `/MREQ` pulses low again while `/RD` stays high, so a correctly wired memory does not drive the bus against the CPU's own refresh address.
- **`/IORQ` selects a separate 256-port I/O space**, reached with `IN` / `OUT` and entirely distinct from memory. The 65xx bus has no equivalent — there, peripherals are memory-mapped.
- **Z80 addressing is a little scrambled on the package.** `A0` – `A10` sit on pins 30–40 and `A11` – `A15` wrap round to pins 1–5; the data bus is not in pin order either. The pin-assignments window is worth keeping open.
- These parts have **no example circuit** (see below) — you can't demonstrate a CPU by flipping switches at it. What you want instead are the worked 65xx machines shipped as ordinary project files; see [Files, Saving & Undo](#).

Memory chips

The **Memory** group carries the address-indexed parts: a couple of generic teaching ROM/SRAM chips plus real-shaped EEPROM/EPROM/SRAM parts on wider DIP packages. Reads and writes are wired up the same way as every other chip in the catalog, but a non-volatile chip's contents live in a real file on disk and are programmed through a dedicated in-app tool rather than by the circuit itself. See [Memory Chips & the Inspector](#) for the full story — file-backing, the external programmer, and the hex/ASCII inspector.

The pin-assignments window

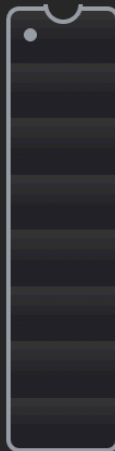
Right-click **any** chip — or a package-footprint discrete like the `bar8iso` LED bar, which seats and rotates exactly like a DIP chip even though it isn't one — and choose **Pin Assignment**, at the top of its context menu, to open its **pin-assignments window**: a small, floating window separate from the main desk, showing the physical DIP layout with the notch at the top, pin 1 at the top-left, and pin numbers wrapping down the left side and back up the right to the highest pin at the top-right, exactly as printed on the part.

74LS161 · Sync 4-bit binary counter



DIP-16 · pin assignments

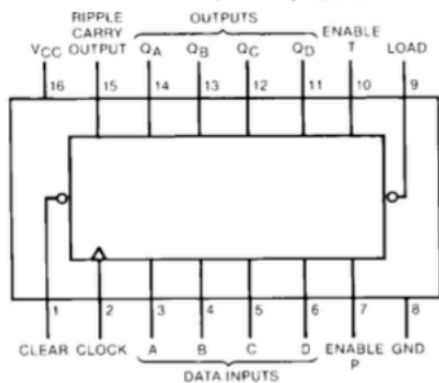
- 1 CLR IN
- 2 CLK IN
- 3 A IN
- 4 B IN
- 5 C IN
- 6 D IN
- 7 ENP IN
- 8 GND GND



- VCC PWR 16
- RCO OUT 15
- QA OUT 14
- QB OUT 13
- QC OUT 12
- QD OUT 11
- ENT IN 10
- LOAD IN 9

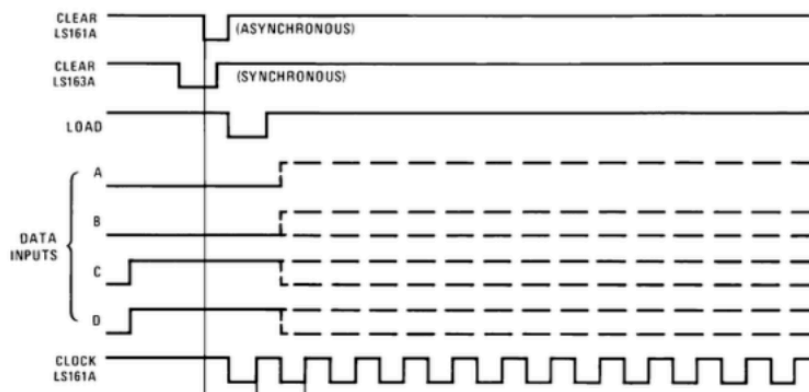
Datasheet – internal diagram & function table

Connection Diagram



Timing Diagram

LS161A, LS163A Synchronous Binary Counters
Typical Clear, Preset, Count and Inhibit Sequences



For a real chip this layout is always the **canonical, fixed arrangement** — it matches the physical part regardless of how you've flipped it on the desk, because a real chip's pin-1 dot is a physical feature of the package, not something rotation changes. `bar8iso` is the one exception: it has no real notch to key off, so its pin-assignments window reflects its **current R flip** on the desk — rotate it and the corners in the dialog swap to match.

Below the pin map, chips that have one show the manufacturer's **datasheet crop** — a cropped image of the connection diagram or function table pulled straight from the source datasheet PDF. If a chip has no crop on file, that part of the window simply isn't there — the pin map alone is still complete and accurate.

When **Settings → Data Sheets** points at a local folder containing that chip's full datasheet PDF, the window also grows a small document button in its top-right corner; clicking it opens the PDF itself in your system's PDF viewer. This is independent of the built-in datasheet crop — you may have one, both, or neither for any given chip.

Example circuits

A pin map tells you where the pins are; it doesn't show you the part working. So every 74xx chip in the catalog ships with a **worked example** — a small bench built around that one part — and the pin-assignments window is where you reach it. Look for the **circuit button** in the window's top-right corner, beside the datasheet one.

Click it and the example arrives as a **new desktop** in the open project, called `74LS138 example` (or whichever part it is), already framed on screen. Every one is the same bench, so once you can read one you can read them all:

- a **5 V brick** feeding both power rails, and a **clock** for the clocked parts;
- **switched inputs** on the left, each throwing between +5 V and a pull-down so an input is never left floating — a part with more inputs than will fit gets a DIP switch bank over a resistor network instead;
- the **chip under test** in the middle, straddling the trench;
- **LED read-outs** on the right, one per output, through its own resistor. An active-LOW output has its LED wired the other way up, so a lit lamp always means *this output is asserted*;
- a **caption** above the bench saying what the demo shows.

Press **Run** (Space) and flip the switches. Each example opens in a state chosen to show the part doing something.

A few practical notes:

- It's an ordinary desktop and an ordinary unsaved change — it doesn't reach your project's file until you save, and you can rename it, edit it, or delete it like any other.

- Asking for the same example twice doesn't make a second copy; you land back on the desktop you already have.
- Adding it stops a running simulation, exactly as switching desktops does.
- Parts with no bench have no button: the memory and interface chips (a RAM or a CPU can't be demonstrated by flipping switches at it — those are the computer demos, which need a program), and every discrete, brick and wire.

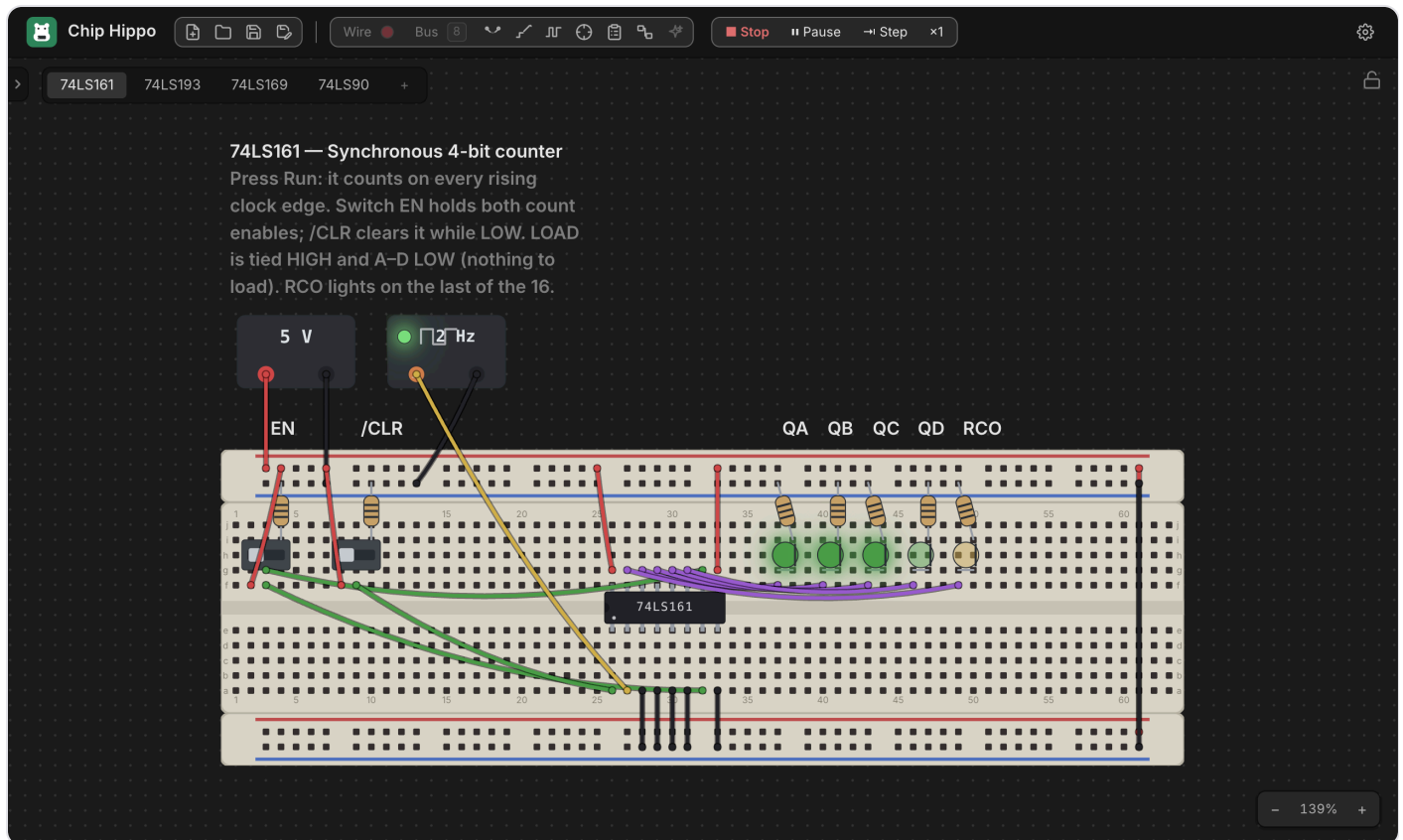
Datasheets

The datasheet crops shown in the pin-assignments window are committed image assets built once from the manufacturer PDFs (`make datasheets`), not fetched or rendered at runtime — they work offline and load instantly. Four parts in the sequential/MSI wave have no matching `74LS*` datasheet on file (`74LS164` , `74LS193` , `74LS27` , `74LS76`) and simply show their pin map with no crop below it. Pointing **Settings → Data Sheets** at your own folder of manufacturer PDFs is a separate, optional feature — it doesn't add or replace the built-in crops, it just adds the "open datasheet PDF" button for any part whose PDF you have on hand.

See also [Chips & Components](#) for how chips seat into a breadboard, and [Memory Chips & the Inspector](#) for the memory group's file-backing and inspector.

Running a Simulation

Once a circuit is wired up, **Run** hands it to the simulation engine, which traces power from every supply, resolves the electrical state of every net, and settles the circuit — LEDs light, chips report their health, and the probe and logic analyzer come alive. This page covers the transport (Run, Pause, Step, speed), what the signal levels mean, what triggers a re-settle, and what you can still interact with while the circuit runs.



Run & Stop

Press **Run** in the header toolbar — or the shortcut `Space` / `Cmd+R` (`Ctrl+R` on Windows/Linux) — to start the simulation. The button becomes **Stop**, editing locks (you can't move parts, place new ones, or lay wires while running), and the engine cold-starts: it settles the circuit once from scratch and begins driving live views from the result.

Press **Stop** (same shortcut) to freeze the simulation and return to editing. Stopping clears all run-volatile state — net levels, chip health badges, any sequential state (counter values, shift-register contents, clock phase), and **12 V chip damage**, so a chip you burnt out is whole again the moment you stop. Nothing that happens during a run outlives it.

`Space` is disabled while a placement, wire, or bus tool is armed (it might want the key for its own gesture) or while a text field has focus — use `Cmd/Ctrl+R` in that case, or just click **Run**.

Signal levels: H, L, Z, X

Every point in the circuit — a hole, a pin, a wire — settles to one of four levels. The probe tool and the logic analyzer both speak this same vocabulary (see [Probing & Net Names](#)):

- **H** — logic high, driven by a supply or a chip output.
- **L** — logic low, driven the same way.
- **Z** — floating: nothing is actively driving this point. Chip Hippo models real 74xx TTL behavior here — **a floating input reads as HIGH**, so an unconnected chip input doesn't just do nothing, it behaves as if tied high.
- **X** — a conflict: two drivers disagree on a net (two chip outputs fighting, or opposing supplies wired together), or the net never settled to a stable value at all (an oscillation — see below).

You don't need to know how the engine computes these to use the app, but recognizing **X** on a probe reading is the fastest way to spot a wiring mistake.

What re-settles the circuit

The circuit isn't a one-shot calculation — anything that could change what a net carries triggers a fresh settle while you're running:

- Clicking a **switch** or **button**.
- Turning a **PSU** on/off or changing its voltage.
- Changing a **clock**'s rate or manual/free-running mode.
- A manual clock click, or (for sequential circuits) a **Step** advance.

Each re-settle starts from the *previous* stable state rather than from scratch — this warm start is what lets a cross-coupled latch or a counter hold its value across small changes instead of resetting every time you touch something.

The transport — Pause, Step & speed

For circuits with a **clock source**, the toolbar grows a small transport cluster next to **Run** the moment you start:

- **Pause / Resume** — freezes the free-running clock's edges without stopping the simulation; everything stays lit exactly as it was. Click again (the button relabels to **Resume**) to continue. To hold

just ONE clock while the rest of the circuit keeps running, use the **■** button on that clock brick instead (see [Power & Clock Sources](#)).

- **Step** — advances the circuit by exactly one clock half-period: every free-running clock flips once, and the circuit settles around it. Stepping implies Pause, so you can single-step a counter or shift register edge by edge and watch each bit change. A manually-toggled clock steps only on its own click, not on **Step**.
- **Speed** ($\times\frac{1}{4}$ / $\times 1$ / $\times 4$) — click to cycle the free-running clock rate up or down. It scales every clock brick on the desk together, so a design with more than one clock keeps their relative timing. A clock never runs faster than 100 Hz, the top of its own rate list, so $\times 4$ applied to a rate that is already a quarter of the way there has nothing further to give.

See [Power & Clock Sources](#) for placing and configuring a clock brick, and [Logic Analyzer & Timing](#) for capturing what these edges actually do to your signals over time.

What lights up while running

Live views render entirely from the settled simulation state — nothing is computed by the views themselves:

- **LEDs** light when their anode net is **H** and their cathode net is **L** — provided something is limiting the current. An LED driven directly between a strong supply and strong ground, with no series resistor anywhere in the loop, shows a distinct **burnt** cue instead of lighting: that connection would fry a real LED, so it fries this one. A net reached through a resistor is only weakly pulled rather than strongly driven, which is what makes the resistor count. This is a physical rule, not a logical one — the levels alone would happily light it.
- **Chips** show a small health badge the moment they're powered: normal chips show nothing extra, an **underpowered** chip (running at 3 V) gets an amber corner dot, and a chip killed by 12 V shows **damaged** — a red X with a smoke cue. Hover the badge for the exact reason (also unpowered or reversed VCC/GND). A damaged chip stays dead for the rest of that run, and **Stop** restores it — see [Power & Clock Sources](#).
- **Probe highlights** tint whatever net you hover by its live level. Full detail on reading and naming nets lives in [Probing & Net Names](#) — this page won't duplicate it.
- **Clock bricks** carry a small pulse lamp that tracks their own current output level in real time.

Interacting live

Switches, push buttons, and clock bricks stay fully interactive while running — click a **slide switch** to flip it, hold a **push button** to press it, click one position of a **DIP switch bank** to open or close it, or click a **manual clock** to pulse it by hand. Each of these is exactly the kind of input event described above: it

doesn't just update its own view, it triggers a fresh settle of the whole circuit, so downstream LEDs and chips react immediately.

Everything that edits the circuit's *topology* — placing, moving, or deleting a part, board, or wire — is locked out until you Stop. Part state (a switch position, a button press, a clock's phase) is not topology, so it stays live the whole time you're running.

Oscillations & conflicts

"The circuit settles" just means: the engine keeps re-evaluating every net and chip, feeding each result back in, until nothing changes anymore. Most circuits reach that fixed point in a handful of passes, faster than you can perceive.

Two things can go wrong:

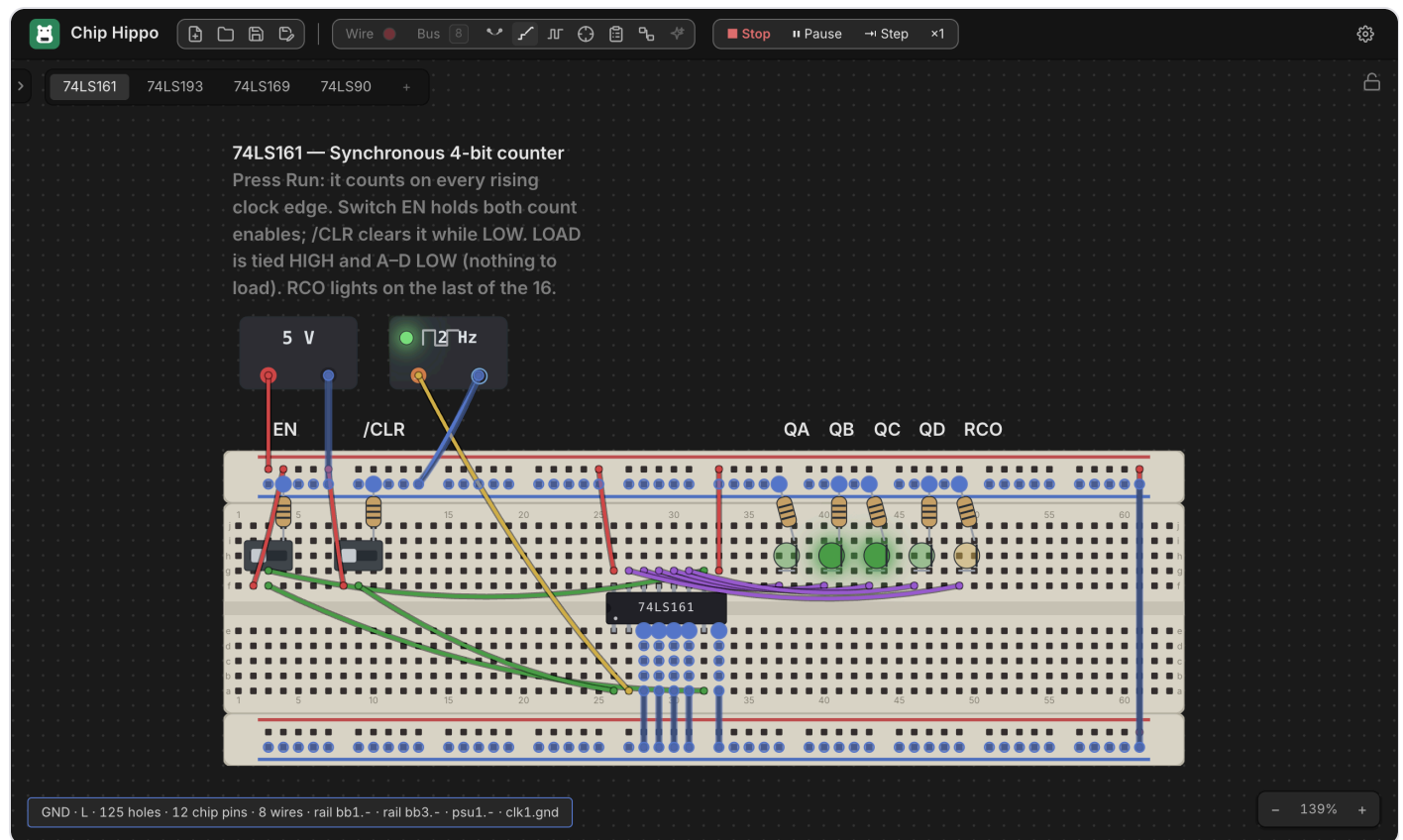
- **A conflict or short** — two chip outputs disagreeing on the same net, or opposing supplies tied together — settles immediately, but to `X`, and the app raises a warning naming the net.
- **An oscillation** — a circuit that never stops changing (an unbuffered ring of inverters, for instance) can't reach a fixed point at all. Chip Hippo detects this — after a bounded number of attempts it gives up, marks the still-changing nets `X`, and flags the oscillation rather than hanging.

Either way, the affected nets read `X` on the probe and in the logic analyzer, which is your cue to go find the wiring mistake causing it.

Next: [Power & Clock Sources](#) for supply voltages, the 12 V damage rule, and clock bricks in depth, or [Probing & Net Names](#) to read exactly what's happening on any net.

Probing & Net Names

Once a circuit is wired up, the probe is how you ask "what's this actually connected to?" — click any hole, pin, or terminal and Chip Hippo highlights every other point that shares the same electrical net, live, without needing to trace wires with your eyes. This page covers arming the probe, reading its net summary, naming a net so it reads clearly everywhere in the app, dropping free-text annotations on the desk, and sending a net to the logic analyzer.



The probe tool

Press **I** or **P** (either key toggles it — no modifier), or click **Probe** in the toolbar. With the probe armed:

- **Hover** a hole, rail point, PSU/clock terminal, chip pin, or a wire and its whole net lights up — a transient highlight that follows your cursor.
- **Click** a point to **pin** the highlight so it stays put while you move the cursor elsewhere (handy for comparing it against a second net). Click the pinned point again to unpin it.
- **Esc** unpins first, then a second **Esc** disarms the tool; **M** disarms whichever of the wire, bus, or probe tool is currently armed.

Unlike the wire, bus, and placement tools, **the probe stays available while a simulation is running** — editing locks on **Run**, but you can keep probing the live circuit the whole time. See [Running a Simulation](#) for what Run/Stop locks and what stays live.

Reading the net summary

A highlighted net draws as a glow: dots on member holes and terminals, rings around member chip pins, and a glow stroke along every member wire — brighter when pinned. Below it, a status readout summarizes the same net in one line, for example:

```
VCC · H · 23 holes · 3 chip pins · 2 wires · rail bb1.t+ · psu1.+
```

The pieces, in order, appear only when relevant:

- The net's **name**, if you've given it one (see below).
- Its current **level** (`H / L / Z / X`) while a simulation is running — gone when stopped, since an unpowered net has no level to report.
- A **connectivity summary** — hole/pin/wire counts, which rails it touches, and any PSU/clock terminals it reaches.

Under the hood, every hole, rail node, wire, active switch/button bridge, and component pin gets partitioned into these nets by a union-find pass over the whole document — the same netlist the simulator resolves each net's level from.

Naming a net

A net's default identity is just its smallest member address (`bb1.a12` , say) — useful internally, meaningless at a glance. Right-click a point while probing to give the net a real name:

- **Name this net...** (or **Rename net...** if it already has one) opens a prompt with quick picks for **VCC**, **GND**, and **CLK**, or type anything you like.
- **Clear name** removes it.

The name binds to the specific point you right-clicked, but resolves to whatever net that point sits on at any given moment — so it survives rewiring around it. If an edit later merges two separately-named nets into one, one name wins deterministically and the other is dropped; nothing crashes, but it's worth a glance if a net you named seems to have picked up its neighbor's label instead.

Naming isn't just cosmetic: a named net is what makes the [Schematic View](#) and the [Build Guide & BOM](#) readable — `VCC / GND / CLK` and your own signal names carry through instead of raw hole addresses.

Annotations

For notes that aren't about a net at all — reminders, section headers, "TODO: add a pull-up here" — drop a free-text **annotation** on the desk from the **Annotations** section at the bottom of the parts palette:

- **Label** — a short one-line caption.
- **Note** — a multi-line block.

Picking one arms a placement ghost like a part; click anywhere on the desk to drop it, which opens an inline editor immediately so you can type right away. Drop it **on top of a part** and it **anchors** to that part — the ghost shows the anchored state before you click — so the annotation rides along whenever that part moves. Drop it on open desk and it just sits at that world position.

Double-click an annotation any time to re-edit its text (`Enter` commits a label; a note keeps `Enter` for newlines, so use `⌘Enter` / `Ctrl+Enter` to commit). Drag its body to reposition it, and right-click for **Edit** / **Remove**. Annotations are pure decoration — they carry no electrical meaning and are invisible to the netlist, occupancy, and the simulator.

Sending a net to the analyzer

Probing a net you want to watch over time? Right-click it and choose **Add to analyzer** to pin it as a channel without leaving the probe tool. See [Logic Analyzer & Timing](#) for capturing and reading the resulting waveform.

See also

- [Wiring, Nets & Buses](#) — laying the wires and buses whose nets you're probing.
- [Running a Simulation](#) — what a net's level means and when the probe keeps working while editing locks.
- [Logic Analyzer & Timing](#) — capturing a probed net's waveform over time.
- [Schematic View](#) — where net names show up in the derived logical diagram.

Memory Chips & the Inspector

Chip Hippo's memory catalog covers both sides of the RAM/ROM divide: address an 8K/32K/128K SRAM for scratch storage that behaves exactly like real silicon (it forgets everything at power-off), or a ROM/EPROM/EEPROM that holds a real byte image between runs — the same image, opened in a floating **inspector**, that you can browse, hand-edit, and program from a file. This page covers what each kind actually stores, how to get data into a ROM, and how the inspector works.

Volatile vs. non-volatile

Every memory chip in the catalog is one of two kinds, and the difference matters more than the pinout:

- **Volatile (SRAM)** — `ram-8k`, `HM62256` (32K×8), `AS6C1024` (128K×8). Contents live only for the current run: they start as random noise every time you press **Run**, whatever the circuit writes stays readable until you **Stop**, and then it's gone. Nothing is ever saved to disk for these parts — there's no image to lose, because there was never a file.
- **Non-volatile (ROM/EPROM/EEPROM)** — `rom-8k`, `28C16` (2K×8 EEPROM), `AM27C1024` (64K×16 EPROM). These chips hold a real byte image that persists across runs, and **travels inside the project file**: save the project and every programmed ROM's contents are written into the `.chhippo` alongside the design, so copying that one file to another machine takes the ROMs with it. An unprogrammed chip reads back random noise too — just like an unprogrammed EPROM off the shelf — until you program it. (Between saves the bytes are kept in a working file in the app's own data folder; which file backs which chip is internal bookkeeping you never need to touch by hand.)

The chip's **blurb** in the parts palette and pin-assignments window always says which kind it is, and its behavior — read timing, chip-enable/output- enable pins, address/data bus width — is otherwise ordinary combinational logic. See [The Chip Library](#) for the full catalog and the pin-assignments window.

Programming a ROM

The circuit itself can never write a non-volatile chip. An EEPROM or EPROM's real write cycle isn't something the simulator drives — Chip Hippo treats every non-volatile part as read-only from the circuit's point of view, the same way you couldn't reprogram a 27C-series EPROM just by wiggling its pins on a breadboard. Any write a chip reports during simulation is silently dropped if the chip is non-volatile.

The only way to put data into a ROM is the **external programmer**:

1. Right-click the chip on the desk and choose **Properties....**
2. Click **Load image... (program)**.

3. Pick a `.bin` (raw bytes) or `.hex` (Intel HEX) file.

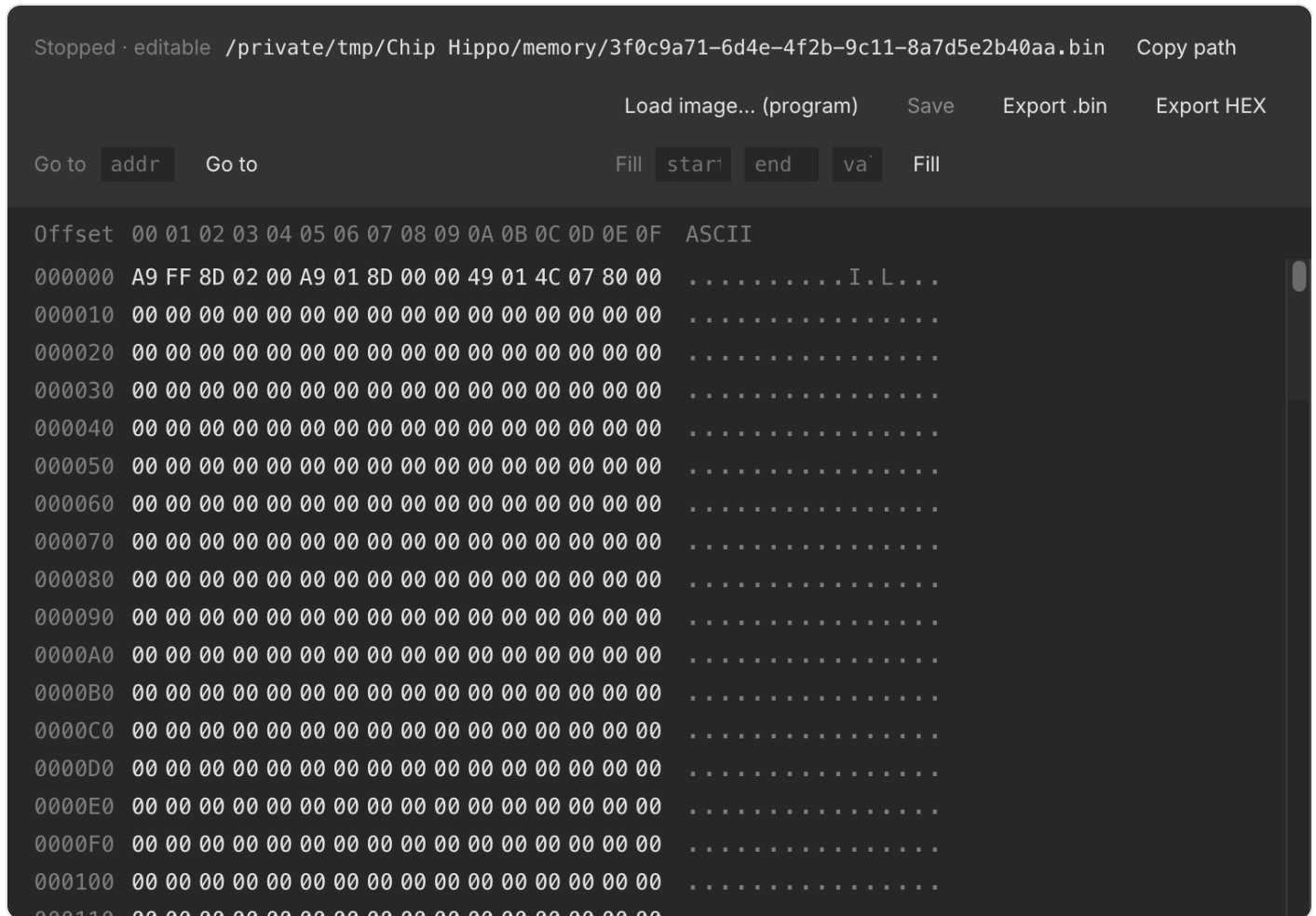
The image is copied to the start of the chip's memory. If the file is smaller than the chip, only the start is overwritten and the rest is left alone; if it's larger, it's truncated to fit — either way you get a warning telling you what happened. Once programmed, the chip is flagged **programmed**, which rides undo/redo like any other edit, and the chip keeps its new contents the next time you open the document or press Run.

The chip also **remembers which file you loaded**. The Properties dialog shows it as **Image file**, so you can tell at a glance whether a ROM holds the monitor image or last week's experiment. If you go on to edit the bytes by hand in the inspector, the file is still shown — marked (*edited*), since it is where the data came from but no longer what the chip holds.

Programming is only available while editing is unlocked (i.e. the simulation is stopped) — see [Running a Simulation](#).

The memory inspector

Every memory chip — SRAM or ROM alike — has a floating **inspector** window: right-click it on the desk, choose **Properties...**, then **Inspect memory...** It's a hex/ASCII grid, one row per 16 bytes, with a toolbar for **Go to** (jump to an address), **Fill** (write a byte value across a start/end range), **Save**, and Import/Export.



What you can do depends on whether the simulation is running and what kind of chip it is:

- **Stopped, ROM/EPROM/EEPROM** — the grid is editable. Click a hex or ASCII cell to type a new byte value directly, or select a range and **Fill** it. **Save** writes your edits back to the chip's file and flags it **programmed**, exactly like using the external programmer.
- **Stopped, SRAM** — shows the chip's last contents from before it was stopped (or noise, if it's never been run) as a read-only snapshot; there's nothing to save, since SRAM has no file.
- **Running, any chip** — the grid is a **live, read-only** mirror of the simulator's own byte image. Bytes the circuit writes are tinted as they change, so you can watch a counter or a shift register fill memory in real time. Editing resumes once you stop.

For a non-volatile part the inspector also names the file its image came from, with **Copy path** beside it; hover that line to see where the app is keeping the raw bytes on disk. It stays open and live-updating as long as the desk document does.

Import & export (Intel HEX)

Beyond loading a raw `.bin`, the inspector's toolbar can **Export** the current contents as either a raw `.bin` or an **Intel HEX** `.hex` file — useful for round-tripping through an assembler toolchain or another tool that expects the hex format. The external programmer (above) accepts either extension coming in: a `.hex` file is decoded before it's written to the chip.

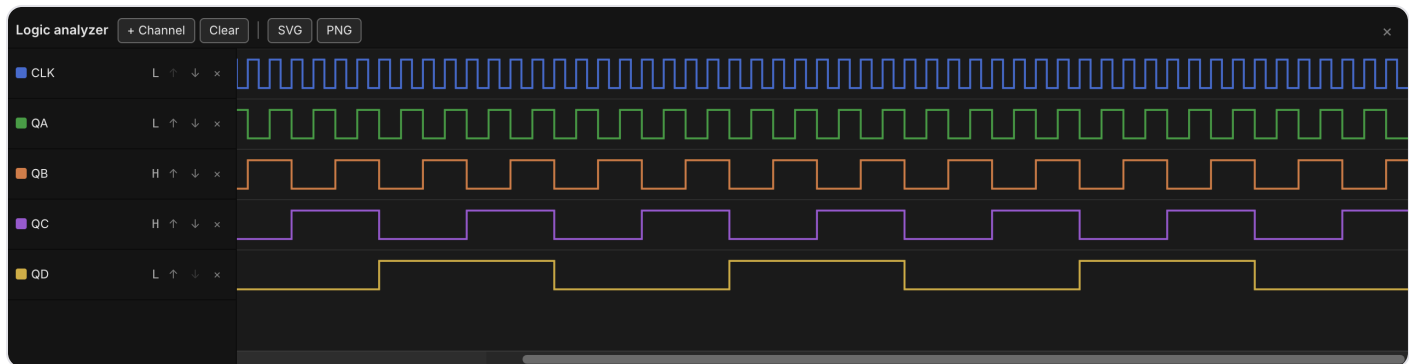
A missing backing file

Because a ROM's contents live in a file separate from the desk document, it's possible for that file to go missing without the document changing — the classic case is deleting the chip and then **undoing** the delete. If Chip Hippo finds a chip flagged **programmed** but its backing file is gone, it recreates the file as fresh noise and warns you that the chip's data was lost, so you know to reload the image rather than trusting stale contents.

See also [The Chip Library](#) for the memory chips in context with the rest of the catalog, and [Files, Saving & Undo](#) for how the project itself is saved.

Logic Analyzer & Timing

The **Analyzer** is a bottom-docked panel that records the live simulation and draws it as a scrolling timing diagram — one lane per signal you've chosen to watch, a step waveform for a single net or a hex value-lane for a bus, with two draggable cursors for measuring the time between two points. It's a passive recorder: it reads the same broadcast the LEDs and chip badges already render from, so opening it, closing it, or adding channels never changes how the circuit behaves.



Opening the analyzer

Click **Analyzer** in the toolbar, or press **A** — a bare key, like the desk's other tool letters, since **Cmd/Ctrl+A** is Select All — to dock the panel along the bottom of the window. Click the button again, press the shortcut again, or click the panel's **x** to close it — the panel remembers whether it was open and how tall you left it (drag its top edge to resize, from a compact strip up to half the window).

With no channels added yet, the panel shows a prompt instead of an empty grid: **add a net or bus**, or right-click a probed net and choose **Add to analyzer**.

Adding a channel

There are two ways to add a channel:

- **From the panel** — click **+ Channel** in the header. The menu lists every *named* net (see [Probing & Net Names](#)) and every bus in the document that isn't already on a channel; picking one adds it.
- **From the probe tool** — arm the probe (**I**), right-click any net on the desk, and choose **Add to analyzer**. This works on ANY net the probe can reach, named or not — it's the way to watch a signal you haven't bothered to name. Using it also opens the panel automatically if it was closed.

Each channel gets a row in the left-hand gutter: a color dot (cycled from a fixed palette unless the channel has its own color), the net or bus name, its current value, and three small controls — **↑ / ↓** to

reorder the channel, and **x** to remove it. Channels are saved with the document (`doc.scopeChannels`), so a desktop reopens with its analyzer setup intact, and adding/removing/reordering a channel rides the normal undo/redo stack. A channel that loses its target (the net was deleted, the bus was removed) simply reads as undriven rather than disappearing — it comes back to life if the target returns.

Reading a waveform

A single-bit channel (a **net**) draws as a step waveform inside its lane:

- **High** traces along the top of the lane, **Low** along the bottom.
- **Floating (Z)** draws a dashed line through the lane's midline.
- **Unknown/conflict (X)** — including anywhere the net is undriven — fills the region with a diagonal amber hatch instead of a line.

The gutter's value column always shows the channel's current level (or its value at cursor A, once one is placed — see below).

Multi-bit (bus) lanes

A channel bound to a **bus** (see [Wiring, Nets & Buses](#)) doesn't draw a two-level waveform — decoding eight or sixteen individual traces would be unreadable. Instead it draws a **value-lane**: a band that necks down to a point at every change and displays the bus's current value as hex (`0x1A` , uppercase) centered in the band, wide enough for the label to fit. The bus is decoded from its member wires' bit positions (as parsed from a name like `D[7:0]`); if even one member is floating or unknown, the whole word is undecodable for that span and the lane hatches just like an unknown net.

Cursors & Δ

Click anywhere in the waveform area to drop **cursor A** at that tick; **Shift**-click to drop **cursor B**. Drag either cursor to slide it to a different tick — the gutter's value column updates to show each channel's value at cursor A while it's placed. With only cursor A down, the header reads `t=<tick>` ; with both down, it reads the delta between them — **Δ N ticks**, plus a millisecond figure when a clock source on the desk gives the analyzer a time base to convert against.

While the simulation is running, the view auto-scrolls to keep the newest tick in sight; scrolling the lanes away from the right edge turns this "follow" behavior off until you scroll back. **Clear** in the header wipes the recorded trace and both cursors without touching the channel list; starting a fresh **Run** does the same automatically.

Export

Two buttons in the header export the diagram exactly as currently rendered — every channel, the full retained trace, and a label column identifying each lane:

- **SVG** — downloads a self-contained `timing.svg` (colors resolved to concrete values, not the app's live CSS variables, so it renders correctly outside Chip Hippo).
- **PNG** — rasterizes that same SVG at 2× scale and downloads it as `timing.png`.

Both are plain browser downloads (no file dialog, no IPC round-trip) and do nothing if there are no channels or nothing has been recorded yet.

See also

- [Probing & Net Names](#) — arming the probe, naming a net, and the "Add to analyzer" context-menu entry.
- [Wiring, Nets & Buses](#) — laying the buses a bus channel decodes.

Build Guide & BOM

Once a circuit is wired up on the desk, Chip Hippo can turn it into the paperwork you'd actually want on the bench: a bill of materials (what to buy, and how many — down to a numbered cutting list of every jumper), and an ordered assembly checklist that names each of those jumpers as you come to run it. None of this is stored on the document — it's derived fresh from the live desk every time you open or change it, so it never drifts out of sync with what you've actually built.

The screenshot shows the Chip Hippo software interface. The main workspace displays a breadboard simulation with a 74LS161 synchronous 4-bit counter IC. The circuit includes a 5V power supply, a 2Hz clock source, and various jumpers connecting the IC pins to the power rails and LEDs. The output pins QA, QB, QC, and QD are connected to green LEDs, and the RCO pin is connected to a yellow LED. The right-docked panel, titled 'Build guide', contains a 'BOM' (Bill of Materials) tab. It lists the components required for the build, categorized into Breadboards, Chips, Discrete Parts, Power, and Wires. The 'Wires' section lists 10 specific jumper wires with their colors and lengths.

74LS161 — Synchronous 4-bit counter
Press Run: it counts on every rising clock edge. Switch EN holds both count enables; /CLR clears it while LOW. LOAD is tied HIGH and A-D LOW (nothing to load). RCO lights on the last of the 16.

Build guide

BOM	Steps
BREADBOARDS	
Full pin-board	×1
Full power rail	×2
CHIPS	
Sync 4-bit binary counter	×1
DISCRETE PARTS	
LED (green)	×4
LED (yellow)	×1
Resistor	×7
Slide switch (SPDT)	×2
POWER	
Clock source (2 Hz)	×1
Power supply (5 V)	×1
WIRES	
[1] Jumper wire (red, 2.6 cm)	×1
[2] Jumper wire (red, 2.7 cm)	×2
[3] Jumper wire (red, 3.0 cm)	×2
[4] Jumper wire (red, 5.5 cm)	×1
[5] Jumper wire (black, 2.0 cm)	×5
[6] Jumper wire (black, 2.8 cm)	×1
[7] Jumper wire (black, 3.1 cm)	×1
[8] Jumper wire (black, 5.5 cm)	×1
[9] Jumper wire (green, 5.7 cm)	×1
[10] Jumper wire (green, 8.4 cm)	×1

Opening the build guide

Click the **Bill Of Materials** clipboard button at the right-hand end of the desk-tools pill, or press **Ctrl+B** / **⌘B**, to open the build guide as a right-docked panel. Clicking it again, or the panel's own **×** close button, hides it — and the button stays lit for as long as the panel is open, however you opened or closed it. The panel's open/closed state is remembered between launches, same as the parts palette. It has two tabs — **BOM** and **Steps** — and is read-only, so it stays available even while a simulation is running.

The guide re-derives itself automatically whenever the document changes while it's open (adding/removing a part or wire, renaming a net, flipping a switch) — there's nothing to refresh by hand.

Warnings

Above whichever tab you're on, the guide surfaces anything that looks un-buildable or likely forgotten, with a count badge on the panel's title when there's something to see:

- **Floating leads** — a chip or discrete pin whose lead sits over no hole.
- **Unpowered chips** — a chip whose VCC or GND pin has no connection, so it will never power up.
- **Single-member nets** — a wired net that only reaches one point, which usually means a connection you meant to make and didn't.

The bill of materials

The **BOM** tab lists everything on the desk as counted line items, grouped into five sections (only the non-empty ones show): **Breadboards**, **Chips**, **Discrete parts**, **Power**, and **Wires**. Boards are counted by strip type (a Full 830 kit counts as its constituent rail/pin strips, not as one line), and components are counted by catalog identity — with a few splits that matter for actually buying the right part: LEDs split by color, PSU bricks by voltage, and clock sources by rate. Each line reads as `title xcount`.

The cutting list

Wires comes last, because you wire after you seat, and it's a different kind of list: a numbered **cutting list**, one line per colour *and cut length*, tallied.

```
[3] Jumper wire (red, 6.1 cm)  x3
```

That's three leads to cut the same, which is how you'd actually work through a drawer or a spool — so the length belongs in the line rather than beside it. The length is the **cut** length: the run from hole to hole plus the stripped end that goes down into each hole, so a jumper crossing a single 0.1 in pitch is about 13 mm of wire, not 3. It's the same measurement, and the same wording, as the drawing at the bottom of a wire's own **Properties...** dialog (see [Wiring, Nets & Buses](#)) — a wire can't read one length there and another here.

Lines are sorted by the app's colour order, then shortest first, and **numbered in that order**. The number is a cross-reference: every assembly step that runs a wire calls it out as `[3]`, so you cut the pile once, number it, and never have to re-measure anything mid-build. It only moves if the desk itself changes — never because a colour was renamed or the language switched.

Assembly steps

The **Steps** tab is an ordered checklist for building the circuit from scratch, grouped in the order you'd actually work: **Place the boards, Power, Seat the chips, Add discrete parts, Run the signal wires**. Each step has a checkbox — ticking it is a session-only visual aid (nothing is persisted), useful for tracking progress while you build.

A few notable details in how steps are phrased:

- A grouped breadboard kit (rails + pin strip snapped together) is one step, not one per strip; a loose strip gets its own step.
- Power steps cover PSU/clock bricks (set to their configured voltage/rate) and then any wire that distributes power — a brick terminal or rail hole at either end.
- A chip's step spells out its straddle, e.g. *"straddling e5–f11, pin 1 at bb1.e5"* (plus a flipped note if it's rotated 180°); a linear discrete lists its resolved lead addresses instead.
- Signal wires are grouped last: whole buses first (one step per bus, one detail line per bit), then the remaining signal nets, each with its wires listed as `[n] from → to`. The leading `[n]` is that wire's item number from the cutting list above, so the callouts form a column you can read down while you cut.

How an endpoint is named

A wire's two ends are resolved to the friendliest label available rather than printed as raw addresses, in this priority order:

1. A component pin at the hole — `"74LS00 pin 3 (1Y)"` (part + pin number + datasheet pin name, when the pin has one).
2. A pin sharing the hole's 5-hole node — a bus tap lands *beside* a pin rather than on it, and still resolves to that pin.
3. A PSU or clock terminal — `"Power supply +"`.
4. A power rail — `"+ rail (bb1)"`.
5. The bare hole address, as a last resort.

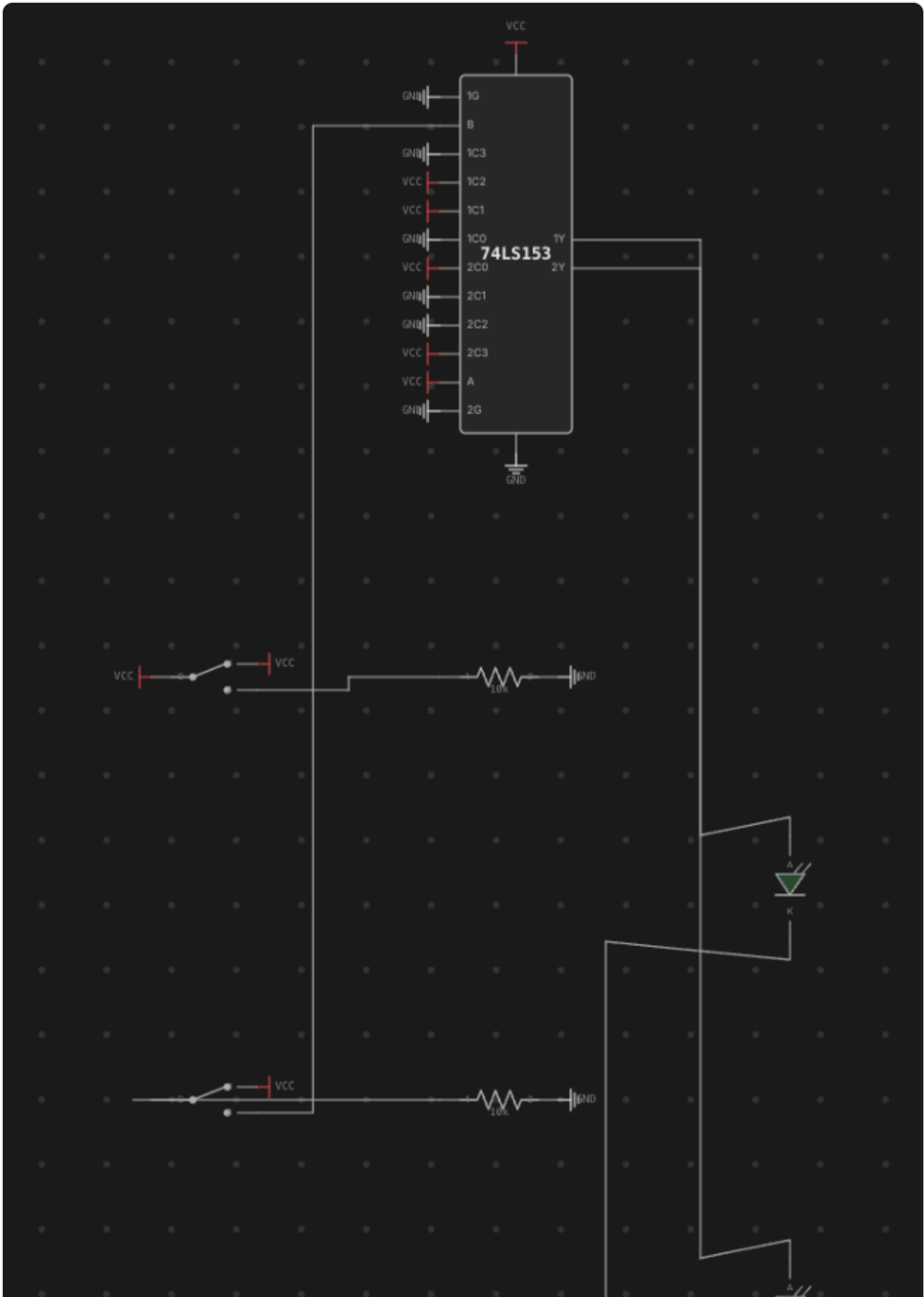
A net you've named (see [Probing & Net Names](#)) leads its step with that name instead of an anonymous net id, so naming your nets up front, before opening the guide, makes the whole tab read far more like a real assembly sheet and far less like an address dump.

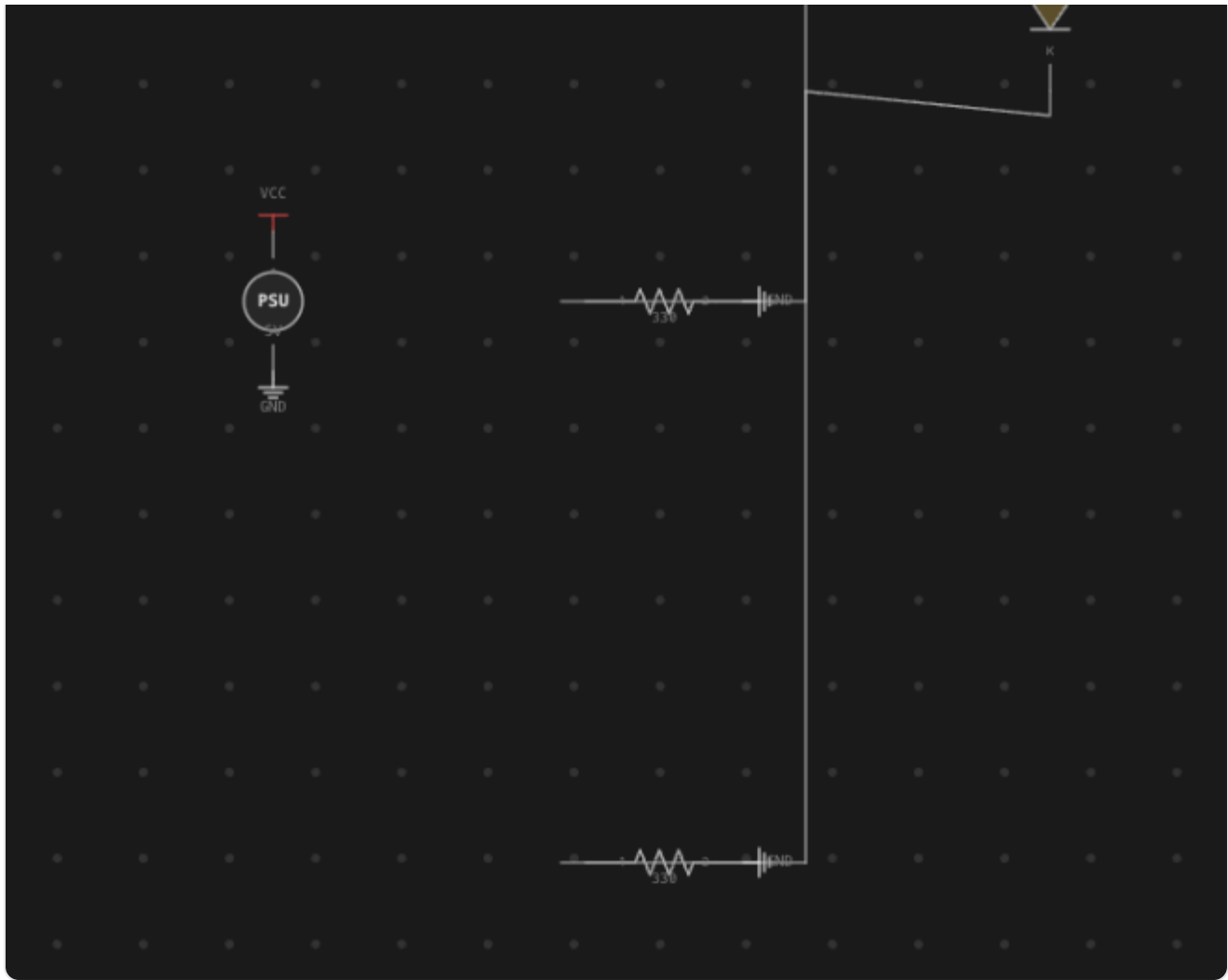
Downloading the BOM

The download icon in the panel header exports the **whole current plan** — the BOM and the assembly steps together, headings, item numbers and all — as a Rich Text Format (`.rtf`) document, named `<project name>-bom.rtf` . It mirrors the panel's tabs, so what you get on paper is what you were looking at. It's a plain browser download (no save dialog, no main-process IPC involved) generated entirely in the renderer, so it reflects exactly what the panel is showing at the moment you click it. `.rtf` opens in any word processor, which makes it easy to print or hand off as a build sheet alongside the physical parts.

Schematic View

The desk shows your circuit exactly as you built it — a physical breadboard with chips seated in holes and wires running point to point. The **schematic view** shows the same circuit the way an engineer would draw it: chip symbols, named nets, and bus lines, laid out automatically. It's a **projection**, not a second document — the breadboard is still the single source of truth, and nothing you do in the schematic changes a hole, a pin, or a wire on the desk.





Switching views

Press **Tab** to flip between the **Breadboard** and **Schematic** views (not while a text field has focus — typing a **Tab** there behaves normally, e.g. to move between fields), or click the **schematic** button in the toolbar, between **Bill Of Materials** and the **AI builder**. Its icon shows the view it will take you to, and fills in while the diagram is up. The schematic reuses the desk's own camera, so pan and zoom feel identical in both views and your position is preserved when you switch back. The first time you open the schematic in a session it automatically fits the whole diagram in the viewport; after that, panning and zooming are exactly what you left them.

Cmd/Ctrl+F re-fits whichever view you are looking at — with the schematic showing it frames and centers the diagram again, the same way it frames the desk. (Fitting the schematic only moves the camera: symbol positions are derived, so there is nothing on the diagram to relocate.)

If the desk has no chips on it yet, the schematic shows a short hint instead of an empty canvas — add a chip on the breadboard and switch back to see it appear.

Chip symbols & routed nets

Every chip is drawn as a labelled box (or, for a plain gate chip, a small distinctive shape — AND/OR/NAND/NOR/XOR/NOT/BUFFER — badged at the top of the box) with named pin stubs instead of physical DIP pins. Discretes get their own recognizable symbols too: an LED's diode triangle, a resistor body, a switch, a push button, a PSU or clock source block. VCC and GND pins don't route across the page like a signal would — each one drops a small power-rail symbol right at the pin, the way a real schematic keeps power off the signal routing.

Everything else is connected by **routed nets**: orthogonal lines that run chip-to-chip along the net they belong to, labeled with the net's name where there's room. A group of pins wired as a bus (see [Wiring, Nets & Buses](#)) collapses into a single **fat bus line** instead of drawing every bit as its own wire — the same visual shorthand a hand-drawn schematic uses for a data or address bus.

Auto-layout & nudging

The schematic lays itself out automatically and deterministically: the same circuit always produces the same diagram, with chips arranged left-to-right roughly along the direction signals flow and edges routed to keep crossings down. You never have to arrange it by hand — but if a particular symbol lands somewhere inconsistent, drag it to a better spot. That drag is stored as a **per-symbol position hint**, not a real coordinate: it only ever affects where that symbol sits on THIS derived diagram, and it has no effect on the component's actual place on the breadboard. Nets simply re-route to follow wherever you drop it.

An **Auto-layout** button in the schematic clears every position hint at once and lets the algorithm re-place everything from scratch — useful after nudging things into a mess, or after a big edit has made your old arrangement stale.

Live simulation tint

Press **Run** and the schematic lights up exactly like the breadboard does: the routed lines tint by their net's live level, an LED symbol lights when its anode reads high and its cathode reads low, and a chip symbol picks up the same health badge (unpowered / underpowered / reversed / damaged) you'd see on

its physical package. Both views are driven from the same simulation state, so whichever one you're looking at when you hit Run shows the identical picture — there's nothing to keep in sync by hand. See [Running a Simulation](#) for what each level color and health badge means.

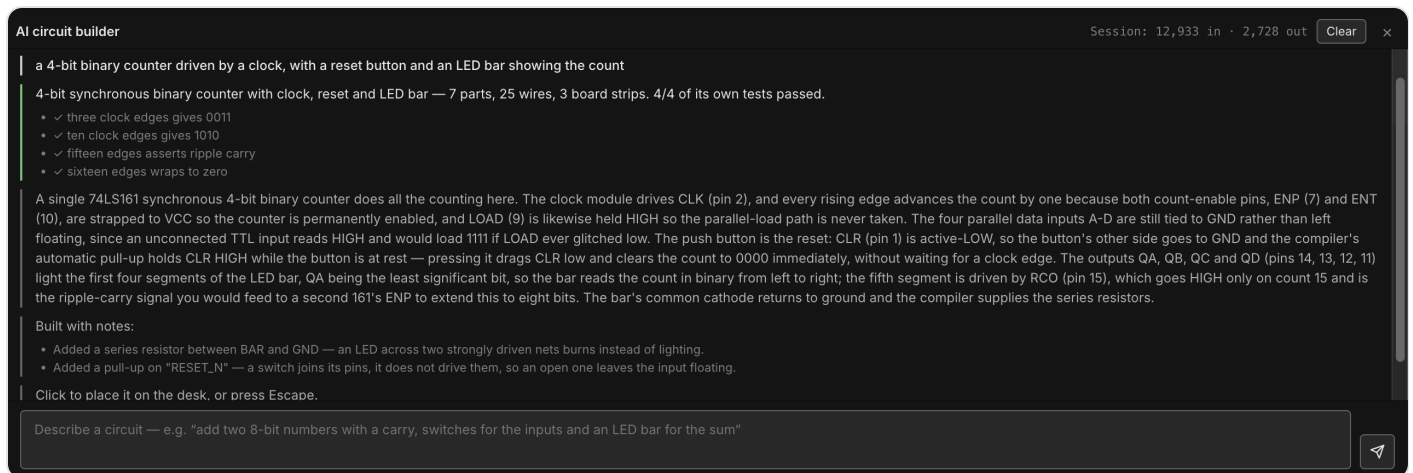
The probe is shared

The connectivity **probe** (see [Probing & Net Names](#)) is armed and driven from the breadboard, but its highlight is not confined to it: probing a net on the desk highlights that same net in the schematic too — by its stable net id, not by screen position — so you can hover a wire on the physical board and immediately see which chip symbols and bus lines it corresponds to in the logical diagram. The highlight is applied to both views at once, whichever one happens to be visible, so switching to the schematic after probing shows the net already picked out. It's the same underlying net, just drawn two different ways.

See also: [The Desk & Breadboards](#) for the physical view this diagram is derived from, and [Wiring, Nets & Buses](#) for how wires and buses become the nets the schematic routes.

AI Circuit Builder

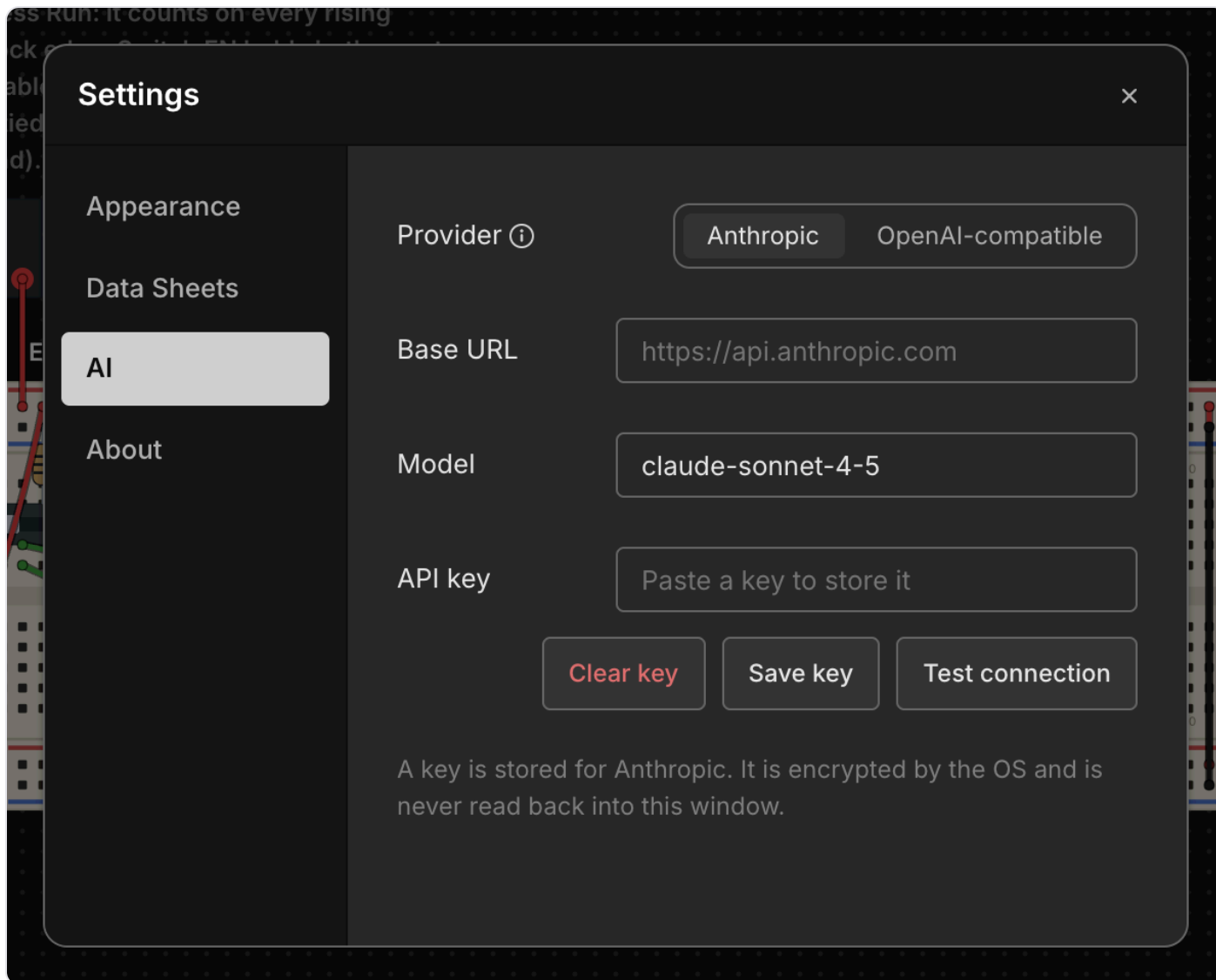
The **AI builder** is a bottom-docked panel where you describe a circuit in plain English and get a real, wired, **simulation-proven** design to place on the desk. It uses *your own* AI connection — your key, your provider, your account — and Chip Hippo contacts no AI provider until you've set one up and asked it for something.



The important thing to understand about it is what the model is and isn't allowed to decide. It never places a chip, never picks a hole, and never draws a wire. It answers one question — *which parts, and which of their pins are joined?* — and Chip Hippo's own compiler does all the geometry, then proves the result by running it through the same simulation engine the **Run** button uses. Nothing that fails is ever offered to you.

Setting up a connection

Open **Settings** (`Cmd/Ctrl+,`) and pick the **AI** tab. There are four things there:



- **Provider** — **Anthropic**, or **OpenAI-compatible**. The second is deliberately not "OpenAI": the same request format is what Ollama, LM Studio, OpenRouter and vLLM all speak, so one setting plus a base URL covers every local and hosted endpoint worth using.
- **Base URL** — leave blank to use the provider's own default (shown as the field's placeholder). Point it at `http://localhost:11434` for a local Ollama, or at any compatible proxy.
- **Model** — also blank for the default. Any model id your endpoint accepts.
- **API key** — paste it and click **Save key**.

Your key is never stored in Chip Hippo's settings file. It goes straight to your operating system's own secure credential store (Keychain on macOS, the Credential Manager on Windows, the desktop keyring on Linux), encrypted, and it is never read back into the app's window afterwards — the panel only ever learns *whether* a key is configured, not what it is. If your system has no secure store available, Chip Hippo **refuses to save the key** and tells you so, rather than quietly writing it to disk in the clear.

Click **Test connection** to check the base URL, key and model in one small request before you rely on them. **Clear key** forgets the stored key.

A local server that needs no key at all works fine — leave the key blank and just set the base URL.

Asking for a circuit

Click the **AI builder** button in the toolbar to dock the panel along the bottom of the window (drag its top edge to resize it; it remembers whether it was open and how tall you left it). Type a description and press **Enter** — **Shift+Enter** gives you a newline for a longer brief.

The toolbar button is **disabled until there's a connection to ask** — no key stored, or a base URL that doesn't parse, and it greys out with the reason as its tooltip. It's the one tool in Chip Hippo that can't work on your own machine alone, so the button says so rather than failing after you press it. (Whether the server will actually *accept* your key is only the server's to say — that's what **Test connection** above is for.)

Descriptions that work well are the ones an engineer would accept:

add two 8-bit numbers together with a carry, switches for the inputs and an LED bar for the sum

a 4-bit counter driven by a clock, showing its count on an LED bar

Be specific about the **inputs and the outputs** — what you want to set, and what you want to see. The parts themselves you can leave to it: the model is told, on every request, exactly which chips are in Chip Hippo's catalog and exactly what every pin on each of them is called, so it has no reason to invent a part that doesn't exist.

You don't need to mention power, and you shouldn't. Every chip declares its own VCC and GND pins, so the compiler wires power itself, plants a 5 V supply, and bridges the breadboard's two power rails. You also don't need to ask for a series resistor on an LED — see below.

While it's working the panel shows progress; the same button becomes a **cancel**, so a long or expensive request is always interruptible.

What happens to the answer

Before you are shown anything at all, the answer is put through a ladder of checks. Each one exists because of a specific way a machine-generated circuit goes wrong *while still looking perfectly fine*:

1. **It compiles.** Parts get real seats on real breadboard columns, with an allocator that makes it impossible for two parts to accidentally share a column-half — which on a real breadboard means

they're shorted together. Before that, the netlist is held to the electrical rules a simulation would happily let through: no output wired straight to a power rail (the supply silently wins), and no two outputs on one net unless every one of them can be switched off — a tri-state bus.

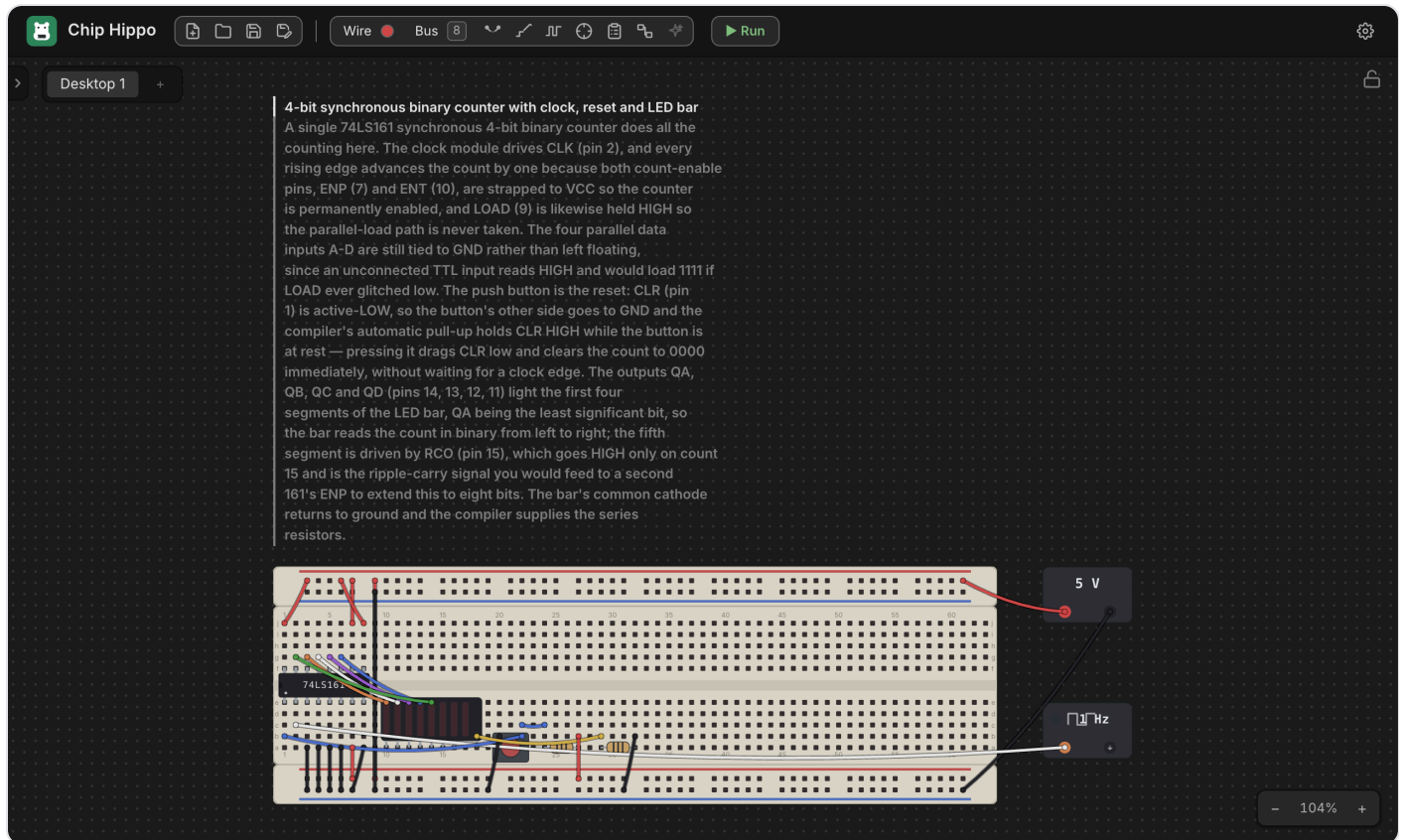
2. **Everything seated.** No part hangs off the end of a board with pins over nothing. A part like that loads without complaint and is electrically dead.
3. **The nets match.** The connections the design *asked for* are compared against the connections the finished breadboard *actually has*. This is the check that catches an accidental short, which is otherwise invisible — the circuit loads clean and simulates happily while computing something else.
4. **It powers up and settles.** Every chip reports OK, with no conflicts, no shorts, no oscillation, and no LED burning out.
5. **Nothing is left floating.** A signal net nothing drives is reported rather than shipped — and so is an input the design uses but never connects, such as a decoder's enable, which would otherwise quietly read HIGH. (The spare gates of a chip it only half uses may float.)
6. **Its own tests pass.** This is the interesting one. The design is asked to state its own acceptance tests — "with these switches set this way, after this many clock edges, these LEDs should read this" — and Chip Hippo **runs them**. It's the only check that tests what the circuit *means* rather than whether it's internally consistent, and it's what catches a perfectly-built adder with its bit order reversed. A design must bring at least two, each with different inputs; every test starts from the circuit as built and must spell out every switch position and every segment it reads; and each test's state is checked for shorts and fights too, not just for the lamps it names.

The panel reports which tests ran and whether they passed as part of the summary.

Placing what you get

A design that passes is **armed as a placement ghost** — the whole circuit follows your cursor, snaps against boards already on the desk, and turns red where it won't fit. Click to place it, or press `Escape` to discard it. A circuit you positioned yourself is a great deal easier to trust than one that simply appears.

However large it is, placing it is **one undo step**: `Cmd/Ctrl+Z` removes the entire thing — boards, chips, wires and all — in one go.



Every generated circuit **explains itself**. The description above is stamped on the desk as a caption over the boards, in the design's own words: which chip does the work, why each tie-off is where it is, and what the compiler added on your behalf. You didn't build it and can't ask it why it's wired the way it is — so it says so up front. The caption is an ordinary annotation anchored to the design, so it travels with it, and you can edit or delete it like any other note.

When it doesn't work

Two very different things can go wrong, and the panel distinguishes them.

The design was wrong. A pin named that doesn't exist, an input left floating, an output shared with another output, a test that fails. These go back to the model as a precise list of what failed — not as prose — and it gets up to **two** attempts to fix them. If it still can't, the panel lists what's still wrong and stops rather than looping at your expense.

Chip Hippo was wrong. If the design was described correctly but the compiler couldn't build it, the panel says so plainly and doesn't send it back — re-asking would only spend your tokens on a fault the model has no way to influence.

A few things are outside what a netlist can express, and it will tell you rather than guess:

- **Memory contents.** A ROM, EPROM or EEPROM in a generated design arrives **unprogrammed** — a netlist has nowhere to carry its bytes. The panel says so when it offers the design; load the image through the chip's **Properties** before you run it.
 - **Multi-corner cans** — the oscillator packages. The model isn't offered them.
-

Review: asking what's wrong with a circuit

The panel has two modes, picked with the **Build / Review** switch in its header. Everything above is **Build**. **Review** turns the panel around: it reads the desktop you already have, runs Chip Hippo's own checks over it, and explains what it found.

Type a question and press `Enter` — or just press `Enter` with the box empty, which asks *"what is wrong with this circuit?"*.

Chip Hippo finds the faults; the model explains them. This is the same division of labour that keeps Build honest. The app derives every finding itself, from the real netlist and a real settle of the exact circuit on your desk — the model is never asked to read a wiring list and decide whether something is shorted, because a wrong answer to that would be indistinguishable from a right one. What it *is* good at is the sentence afterwards: what a fault means for your design, which one to fix first, and what to do about it.

The checks it runs are the ones that leave a circuit looking perfectly wired and quietly not working:

- **Nothing is powered** — no supply on the desk, a chip whose power pins go nowhere, or one whose VCC net never reaches a rail.
- **An input nothing drives.** The most convincing lie a real circuit tells: a floating TTL input reads HIGH, so the chip *works*, just not as designed. Only inputs that are actually in use are reported — an unused gate on a quad package is idle, not broken.
- **A part switched off.** An active-low output enable left unwired reads HIGH, so the chip drives nothing at all while its wiring looks flawless.
- **Two outputs on one net**, whatever they happen to be driving today.
- **Shorts, driver conflicts, and a circuit that won't settle**, straight from the engine.
- **A chip at 3 V or 12 V**, and **an LED with nothing limiting its current**.

The findings appear in the transcript immediately, before the request even goes out — so if the connection fails you still have the app's own answer.

Review never changes anything. No part is placed, no wire is moved, and there is nothing to undo afterwards. That's also why it's the one thing here that works **while the simulation is running**: a review can't be refused by a frozen desk, and a running circuit is often exactly when you want to ask what it's doing.

Switching between Build and Review starts a fresh conversation — the two ask the model quite different things — but your transcript stays where it is.

LEDs and resistors

You never need to ask for a current-limiting resistor, and you shouldn't put one in your description. Chip Hippo models the *physical* requirement, not just the logical one: an LED conducting between two strongly-driven nets burns out instead of lighting, exactly as it would on a real bench. So whenever a lamp leg goes straight to a power rail — a display's common leg, a single LED's cathode to GND or its anode to VCC, a segment of an isolated bar — the compiler interposes a series resistor for you (a resistor network, when there are several). See [Chips & Components](#) for the same rule when you're wiring by hand.

Cost, privacy and the simulation

Requests go to whichever endpoint you configured, billed to your own account — Chip Hippo neither proxies them nor sees them. The parts catalog that makes up most of each request is sent in a form your provider can cache, so a repair attempt costs far less than the first ask.

The panel won't build while the simulation is **running** — press **Stop** first. A generated circuit is a document edit like any other, and the desk is frozen while the circuit runs. **Review** is exempt: it changes nothing, so there is nothing for a frozen desk to refuse.

A review sends a compact description of your circuit — the parts, which pins are joined, and the findings — but never the layout, and never anything about your project beyond the desktop you asked about.

See also

- [Settings](#) — the AI tab in full, alongside every other preference.
- [Running a Simulation](#) — the engine every generated circuit is proved against before you see it.
- [Files, Saving & Undo](#) — how a placed circuit is saved, and how one undo step removes it.

Arduino Integration

Chip Hippo can talk to a real **Arduino** over USB while the circuit runs. An **Output** hands a value from the breadboard to a sketch — a number to show on a real display, a bit to switch a real relay — and an **Input** hands a value from the sketch back onto the breadboard: a sensor reading, a key press, the answer to a question the circuit just asked.

The sketch side is written for you. Chip Hippo **generates a header file** for each board, declaring one function per Output and one small object per Input; you include it, call two functions from `setup()` and `loop()`, and fill in what each Output should do. The link underneath — framing, checksums, acknowledgements, retries — is the header's job and never yours.

No Arduino to hand? Every Chip Hippo has a built-in **Mock** connection that plays the board's part itself (see [The Mock connection](#)), so you can build and try an integration before a wire is connected.

What you need

- An Arduino (any board with a USB serial port — an Uno, a Nano, a Mega, a Leonardo...) and the Arduino IDE to upload sketches to it — or just the built-in Mock.
- A **connection** for it in Settings ▶ Serial I/O (below), naming it and saying which port it is plugged into.
- One or more **Outputs** or **Inputs** on the desk, each assigned to that connection.

Nothing here touches the network: a connection is a USB cable, and Chip Hippo opens the port only while the circuit is running.

Connections

A **connection** is a named Arduino: *Display board*, *Sensor Nano*. Open Settings (`Cmd/Ctrl+,`) and choose the **Serial I/O** tab. **+** beside the connection list creates one; give it a name and pick its **Port** from the list of what is plugged in right now (**Re-scan** after plugging a board in). **Baud rate** defaults to 115200, which suits every current board. **Advanced** holds the rest of the serial framing — data bits, parity, stop bits and flow control. Leave those alone unless you have a reason.

Apply saves the connection. If the port isn't plugged in at that moment you're told so, and the button becomes **Apply anyway**, which saves it regardless — useful when you're setting up before the board arrives. **Open window...** opens the connection's [window](#), and the bin beside the list (click it twice) deletes the connection. An Output or Input still using it keeps its assignment — its Properties show the connection as *(not on this computer)* — and the circuit won't run until you choose another one there.

A connection whose port is missing, or which still **Needs configuration**, says so in the list, and its **Port** is marked in red.

The list always starts with the **Mock**, marked *Built-in*. It has nothing to configure and can't be removed, and its name is reserved: a connection of your own can't be called *Mock*.

Connections travel with the project

The connection's **name** and serial settings are saved in the project file, so a design opened on another computer still knows which board each Output talks to. The **port** is not — `/dev/cu.usbmodem14101` on your Mac means nothing on someone else's PC — so a connection that arrives in a project is added to that computer's Settings marked **Needs configuration**, and the circuit won't run until a port has been chosen for it.

The Mock connection

The **Mock** is a pretend Arduino inside Chip Hippo. Choose it as an Output's or Input's **Connection** like any other, and press **Run**. There is no port to pick, no sketch to upload and no header to generate. It speaks the same protocol a real board does, byte for byte, so what works with the Mock works with a board.

When a run starts, the Mock's **connection window** opens beside the desk (without taking focus from it), with a send panel under its stream. You can also open it from Settings ▶ Serial I/O: pick **Mock** and press **Open window....** It shows:

- **What arrives.** Each value an Output sends appears as a line, named and split into its fields: `→ OUTPUT Display value=0x3F dp=1`. The Mock acknowledges it at once, so the circuit carries straight on.
- **What you send.** There is one row per Input on the Mock. A bit field is a toggle; a byte or a word is a row of bit toggles plus a hex field, and editing either updates the other. **Send** sends that Input's whole value, exactly as a sketch's `send()` would, and it lands on the board like one. **Send all** sends every row in turn. The rows appear once the circuit has run with the Mock, since that is when the window learns your design.
- **Log text.** Type a line and press **Log** (or Enter). It arrives in the log exactly as a sketch's `ChipHippo.println()` would.

Faults, folded away at the bottom, make the Mock misbehave once, on purpose, so you can see how the link copes:

Fault	What happens
Drop next ACK	The Mock swallows its acknowledgement of the next Output. Chip Hippo waits, sends it again, and the circuit carries on — the value still arrives only once.
Corrupt next frame	The next acknowledgement or Input the Mock sends is damaged in transit. Chip Hippo notices and asks for it again.
Ignore handshake	The Mock doesn't answer on the next Run, which stops with <i>No answer</i> .
Wrong signature	The Mock answers the next Run as a sketch built for a different design, which refuses to start.

A fault stays armed (its button highlighted) until it has happened. Click it again to disarm it. You can arm one while the circuit is stopped, ready for the next Run. What each fault causes shows up in the stream as a warning line (turn on **Protocol** to see the traffic around it too).

Outputs and Inputs

Add either from the parts palette's **SIGNALS** section: pick **Output** or **Input**, click anywhere on the desk, and choose how many **pins** it has — 1, 2, 4, 8 or 16. You can change that afterwards in its Properties.

Each one appears as a **card** on the right-hand edge of the desk, under the signal buttons: its direction arrow (← Output, pointing out; → Input, pointing in), its colour, its name and how many pins it has. Under the name sits a small numbered chip for every pin that isn't on the board yet, plus one more — the **trigger**, marked with an arrow — unless the element's trigger is **Auto**, which has no trigger tag at all. **Drag a chip onto a breadboard hole** to plant that pin there as a **tag**; drag a planted tag to move it, and press **R** while dragging (or with the tag selected) to turn it. **Delete** unplugs a selected tag back onto its card. The card's right-click menu offers **Properties...**, **Remove All Tags** and **Delete Output / Delete Input**; a tag's own menu adds **Remove Tag** and **Add to analyzer**.

A tag is plugged into its hole the way a wire end is — one hole, one lead — and deleting the breadboard under a tag sends it back to the card; the Output or Input itself is never lost with it.

Pins and fields

An element's pins are grouped into **fields**, and each field becomes one parameter of the Arduino function (for an Output) or one setter (for an Input). A field is a **Bit** (1 pin, a `bool`), a **Byte** (8 pins, a `uint8_t`) or a **Word** (16 pins, a `uint16_t`), up to 16 pins in all. The Properties dialog lists them in pin order with a name each; **+ Bit**, **+ Byte** and **+ Word** add one, and the **x** removes one.

Pin 1 is the **least significant bit** of the element's value, pin 2 the next, and so on. A pin whose tag isn't planted, or whose net is floating or unknown, **reads as 0**.

Properties

Right-click a card (or one of its tags) and choose **Properties...** to edit:

Setting	What it does
Name, Description	The name is also the function (Output) or object (Input) name in the header, turned into a valid C++ identifier
Color	The card's dot and its tags — any of the seven signal colours
Connection	Which Arduino it talks to. The gear beside it opens Settings ▶ Serial I/O
Trigger	Auto, Rising, Falling or Either . Auto watches no line: an Output sends whenever its value changes, an Input puts each value on the board as it arrives. The others name which transition of the trigger line counts. A new Output starts on Rising , a new Input on Auto
Trigger starts	Low or High — what the trigger line is taken to have been before the run began, so a line that starts high can still fire (or not) on the first settle. Greyed out on Auto, which has no line
Pins	The fields, as above

How values move

Chip Hippo only ever looks at the board **between settles** — when every chip's output has stopped changing. A glitch inside one settle is never sent anywhere, and nothing from the Arduino ever lands on the board halfway through a ripple.

An Output fires on its trigger, or on a change

When an Output's trigger line makes the chosen transition, Chip Hippo samples the Output's pins and **sends the value**. On **Auto** there is no trigger line: after every settle Chip Hippo samples the pins and sends the value if it differs from the last one it sent — and always at the very first settle of a run, so the sketch learns where the circuit starts. **The circuit then waits** — no clock edges, no further settles — until the Arduino's function has *returned* and the board has acknowledged it. So whatever the sketch does in response has already happened before the circuit moves on, and an Input the sketch sends from inside that function lands in the same step: the request/response shape (the circuit asks, the Arduino answers, the circuit carries on with the answer) works with no timing of your own.

Auto sends **every** value the pins settle to, including the in-between ones: flip two switches one after the other and the Arduino hears both steps. When the pins only hold a meaningful value at one moment — a data bus, a latch strobe — use an edge on that strobe instead.

An Output or Input waiting for an edge **won't run without its trigger tag on the board**: pressing Run says which one is missing it, and offers its Properties, where you can switch it to Auto instead.

An Input is live or triggered

An Input drives its pins at the strength of a chip output — two things disagreeing over one net is reported as a conflict, exactly as two chip outputs would be — but **drives nothing at all** until the Arduino has sent it a first value.

- On **Auto**, an Input is **live**: each value the Arduino sends is put on the board at the next opportunity between settles.
- On an edge, the latest value the Arduino sent **waits** and is put on the board only when the trigger line makes its transition — the way a latch loads on its clock. Earlier values the circuit never took are simply replaced.

Switching an element to Auto takes its trigger tag off the board, but Chip Hippo remembers where it was: switch back to an edge and the tag returns to the same hole — unless something else has been plugged in there meanwhile, in which case it waits on the card.

Generating the board's code

The **Generate** button in the desk tools pill opens a card listing every connection this desktop's Outputs and Inputs use. Each has one button: **Generate** while its code has never been generated or is out of date, which generates it and opens the files, and **View files...** once it is up to date, which just opens them. In the file viewer, **Save As...** saves the file on show: save `ChipHippo.h` into that board's sketch folder and include it:

```

#include "ChipHippo.h"

void setup() {
  ChipHippo.onConnect(sendInputs); // optional – see below
  ChipHippo.begin();
}

void loop() { ChipHippo.poll(); }

// The Output "Display" arrives here: its name + In, its fields the parameters.
void DisplayIn(uint8_t value, bool dp) {
  ChipHippo.print("showing ");
  ChipHippo.println(value);
}

// The Input "Keypad" is sent from here: set its fields, then send it whole.
void keyPressed(uint8_t code) {
  ChipHippo.KeypadOut.setCode(code);
  ChipHippo.KeypadOut.setReady(true);
  ChipHippo.KeypadOut.send();
}

// Runs at the start of every run: give each Input its starting value.
void sendInputs() {
  ChipHippo.KeypadOut.setCode(0);
  ChipHippo.KeypadOut.setReady(false);
  ChipHippo.KeypadOut.send();
}

```

In the sketch, names read from the **Arduino's** side. An Output leaves the circuit and *arrives* at the Arduino, so the Output called *Display* is the function `DisplayIn()` ; an Input is *sent* from the Arduino into the circuit, so the Input called *Keypad* is `ChipHippo.KeypadOut` . The two different endings also mean an Output and an Input can share a name. A name that already ends that way isn't doubled (an Output called *DataIn* stays `DataIn`). A name that still can't be used as it is (two Outputs with the same name, a field called `delay`) is changed, and the Generate card lists each change under its connection.

The file viewer shows the header beside `ChipHippoExample.ino` , an example sketch that uses it, each in its own tab, with line numbers. The example shows the three things a sketch does, each labelled where it happens: **receiving** (a function per Output that logs what arrived), **sending** (every Input's starting value at the start of a run, then the first Input counting up once a second from `loop()` — a stand-in for

reading a switch or a sensor) and **logging** (a line when a run starts, one per Output that arrives, one per value sent). The Python programs do the same, timing the loop with the module's own `ticks_ms()` and `ticks_diff()`, which read the same on every board. Select part of a file and **Copy** (or ⌘C / Ctrl+C) takes just that; with nothing selected it takes the whole file, and ⌘A / Ctrl+A selects all of the file on show. **Save As...** opens where you last saved that file for this board and desktop, so a new version goes over the old one without hunting for the folder. Closing the viewer returns to the Generate card. The header's own opening comment points at the example too.

An Input's `send()` returns `true` once Chip Hippo has the value. It returns `false` when nobody is listening: at once before a run has greeted the sketch, and after one has stopped (the first time, after a second and a half of resends). `ChipHippo.connected()` says which state the link is in.

An Input puts nothing on its pins until its first value arrives, so a sketch should say where its Inputs stand as soon as a run begins. `ChipHippo.onConnect(fn)` is for exactly that: `poll()` runs `fn` at the start of **every** run. That matters on boards that don't restart when Chip Hippo opens the port — a Leonardo, a Micro, most boards with USB built into the chip — where the sketch simply carries on from one run into the next and `connected()` alone can't tell that a new one has begun.

The Mock needs no header. A desktop whose Outputs and Inputs are all on the Mock is told so, and the Mock never marks the Generate button out of date.

The header never contains your code, so saving over it after a change to the design is always safe. Three rules keep the link healthy:

- **Call `ChipHippo.poll()` often**, and keep Output functions short. The circuit is stood still while an Output is being handled; a `delay()` there is time it waits. Chip Hippo resends a value it hasn't had acknowledged after half a second, and stops the run after three tries.
- **Log with `ChipHippo.print()` / `println()`, never `Serial`**. Chip Hippo owns the port while the circuit runs; anything printed straight to `Serial` is noise on the link. What you print through `ChipHippo` appears in the [connection window](#).
- **One header per connection**. Two boards on one desktop get two headers, each declaring only its own Outputs and Inputs.

Python boards (MicroPython and CircuitPython)

A board that runs Python — a Raspberry Pi Pico, an ESP32, an Arduino Nano ESP32 or Nano RP2040 Connect, most Adafruit boards — can take part too. Set its connection's **Language** to **Python** in [Settings > Serial I/O](#), and Generate writes `chhippo.py` instead of a header: one module that runs on MicroPython and CircuitPython alike. (An Uno, Nano, Mega or Leonardo is too small for Python and stays C++.)

```

from chiphippo import link

@link.display_in # the Output "Display" arrives here
def display_in(value, dp):
    link.print("showing", value)

@link.on_connect # runs at the start of every run
def send_inputs():
    link.keypad_out.set_code(0)
    link.keypad_out.set_ready(False)
    link.keypad_out.send()

link.begin()
while True:
    link.poll()

```

It is the header said in Python: the same names in Python's own style (`display_in` , `keypad_out.set_code`), an Output handled by decorating a function with `@link.<its name>_in` , and `link.print()` for the log. The file viewer shows the module beside `main.py` (MicroPython) and `code.py` with `boot.py` (CircuitPython). Save `chiphippo.py` and the program for your board onto it.

Where the link runs is the one real difference between the two:

- **MicroPython** uses the board's own USB serial port — the one its REPL uses — so point the connection at that port. Inside the link's data a `0x03` byte is just data, not Ctrl-C; between messages, Ctrl-C still stops the program, so Thonny and `mpremote` can always get in to replace its files. A plain `print()` would land in the middle of the link, so log with `link.print()` . On a board whose USB is a separate serial chip (a classic ESP32 DevKit), MicroPython always talks at **115200 baud, 8N1**: leave the connection there — the Generate card warns if it isn't. A board with USB built in (a Pico, a Nano ESP32) ignores the rate.
- **CircuitPython** gives the link a second USB serial port of its own, which `boot.py` turns on (`usb_cdc.enable(console=True, data=True)`). After the board resets, a new port appears beside the REPL's. For a Python connection the **Port** list names the two — *port 1 of 2: REPL* and *port 2 of 2: CircuitPython data* — so choose the data port. (Any board with several ports shows which of them each one is.)

A function that raises an exception is reported in the connection window and the circuit carries on — on a board there is nowhere else it could be seen. Most Python boards don't restart when Chip Hippo opens the port, so give your Inputs their starting values from an `@link.on_connect` function, as above. `link.begin()` can also be handed a port you have opened yourself — a `machine.UART` or `busio.UART` at the connection's baud rate.

Keeping the header current

Every header carries a **design hash** of what it was generated from — the connection's settings and the names, fields and order of its Outputs and Inputs. Change any of those and the header is **out of date**: the Generate button shows a dot, and on the Generate card that connection's button reads **Generate** again. Positions of tags on the board, colours and descriptions don't change the hash. A header generated by an older Chip Hippo whose generated code has since changed shows as out of date too, so that generating it again picks the change up.

Pressing **Generate** is what marks a header up to date — the hash is kept in the project file — so save the new file over the old one (**Save As...** offers the same place) before you rebuild the sketch.

A run checks something narrower: the **layout** the sketch was built for — which Outputs and Inputs there are, in what order, and how each one's pins are split into fields. Names aren't part of it, so a design you've only renamed things in still runs while you get round to regenerating. But a sketch built for a different layout would hand values to the wrong functions or parameters, so that **stops the run** before it starts, asking you to generate the header again and re-upload the sketch. A sketch built by a version of Chip Hippo that speaks a different protocol is refused the same way.

Running with an Arduino

Press **Run** as usual. Before anything simulates, Chip Hippo checks that every Output and Input in use has a connection, that each connection is configured, and that its port is plugged in — and if not, says which and offers to open the right Properties or Settings. It then **opens each port and greets the sketch**, asking again every quarter of a second for up to five seconds; many boards reset when the port opens, so this is also the time a sketch has to boot. A board that never answers stops the run with the port named, and so does a sketch built for a different layout (above).

While it runs:

- **TX**, **RX** and **LG** lamps beside the zoom controls flash as values go to an Arduino, come back from one, and as log text arrives. Click any of them to open a **connection window** (a menu asks which, if there are several).
- A board that **restarts** mid-run (a reset button, a brown-out), one that is **unplugged**, one that fails to acknowledge a value after three attempts, and one whose sketch sends a value that Chip Hippo

never acknowledges all **stop the run** with a message naming the connection. A sketch that has left the run has forgotten it, so press **Run** again to reconnect.

Stop closes every port, so the Arduino IDE can have it back to upload a new sketch — there's no need to quit Chip Hippo between uploads.

The connection window

Each connection has a window showing everything that happens on it, in one stream: what its sketch printed through `ChipHippo`, and the values and protocol frames going back and forth. Because it is one timeline, cause and effect read in order — a value arrives, then the line the sketch printed in response.

Open one by clicking the lamps, with **Open Connection Window** on an Output's or Input's right-click menu (it works while the circuit runs), or by picking the connection in Settings ▶ Serial I/O and pressing **Open window...**

Every line starts with a mark saying what it is:

Mark	Line	Example
(none)	Text the sketch printed	got 41
→	A value sent to the board (an Output)	→ OUTPUT Display value=0x3F dp=1
←	A value from the board (an Input)	← INBOUND Keypad code=0x07 ready=1
·	Protocol traffic: the handshake, acknowledgements, the port opening and closing	· ← ACK seq 12
!	Something went wrong, and what the link did about it	! No ACK for seq 12 – resending (attempt 2/3)

Arrows are always from Chip Hippo's side: → leaves Chip Hippo, ← arrives. Hover over a value's line to see the exact bytes it travelled as. A value that had to be sent again appears once, with the resend as a warning line beneath it.

The footer holds the controls:

- **Log**, **Data** and **Protocol** choose which lines to show — the first two on by default, Protocol off. Hidden lines are still recorded and come back when you turn them on again. **Warning lines are always shown.**
- **Timestamps** adds each line's time since the run started, to the millisecond (+ 12.345) — the way to line a frame up with what the sketch printed.
- **Clear** empties the window. **Save...** writes the whole stream to a text file — every line, with its time, whatever the filters are showing.

A new Run starts the window afresh. After **Stop** everything stays, so a failure can be read at leisure — even after closing and reopening the window. It keeps the last 2,000 lines, follows new lines only while you're scrolled to the bottom, and its text can be selected and copied. Each connection's window remembers where it was and how it was set up.

Under the hood

For the curious, or for anyone writing their own firmware: the link is a small framed protocol over the serial port, **version 1**. Every frame is `0x7E · TYPE · SEQ · LEN · PAYLOAD · CRC`, with `0x7E` and `0x7D` escaped everywhere after the start byte and a CRC-16/CCITT-FALSE (polynomial `0x1021`, initial value `0xFFFF`) over the unescaped `TYPE` to `PAYLOAD`. A value's payload is the element's index, its width in bits, and the value as two bytes, little-endian.

At the start of every run Chip Hippo sends a HELLO carrying the protocol version, a fresh 16-bit session number and the layout signature (a CRC-32 of the layout described above), and the sketch answers with its own version and signature. Both sides take part in the run only if the two match. Outputs and Inputs are then acknowledged and resent on a timeout or a NAK. A resend is recognised by its sequence number, so a function never runs twice for one value, and an Output is acknowledged only once its function has returned. Log text travels in unacknowledged chunks that never split a character.

The whole protocol — every frame, rule and constant, with worked examples — is on the [Serial Protocol](#) page. The book icon at the top right of Settings ▸ Serial I/O and of the Generate card opens it too.

Serial Protocol

Chip Hippo serial protocol, version 1. This page defines the bytes that travel between Chip Hippo and a board at the other end of a [serial connection](#). You never need it to use the Arduino integration — the header and module Chip Hippo generates speak it for you — but it is the whole contract. Anything that follows this page (a sketch in another language, firmware in Rust, a test harness on a pseudo-terminal) can take the board's place.

1. Conventions

- **Host:** Chip Hippo. **Device:** whatever is at the other end of the connection, usually an Arduino.
Circuit: the simulated breadboard.
 - **Output** and **Input** are named from the circuit's side: an Output's value leaves the circuit for the device; an Input's value comes from the device into the circuit.
 - A **data frame** is an `OUTPUT` or an `INBOUND` — the only frames delivered reliably (§6). A data frame is **outstanding** from its first transmission until it is acknowledged or its delivery fails.
 - A **session** is one run's conversation on one connection, from the handshake (§5.4) until the port closes.
 - **MUST**, **MUST NOT**, **SHOULD** and **MAY** state requirements, as in RFC 2119. A paragraph marked *Note* explains a rule and adds none.
 - Byte values are hexadecimal (`0x7E`). Multi-byte integers are **little-endian**. A frame is written as its bytes on the wire, in hex.
-

2. Transport

- A byte stream with no framing of its own: a UART, USB CDC, a pseudo-terminal. It **MUST** deliver bytes in order. It may lose or corrupt them; §3.5 and §6 recover.
- The serial parameters — baud rate, data bits, parity, stop bits — come from the connection (Settings ▶ Serial I/O). The default, and the recommendation, is **115200 baud, 8-N-1, no flow control**.
- **No software flow control.** XON (`0x11`) and XOFF (`0x13`) are ordinary bytes here — `0x11` is the `INBOUND` type — and only `0x7E` and `0x7D` are escaped, so a link that swallows them loses frames.
- One host and one device per connection. The device sees only the Outputs and Inputs assigned to its connection. A run may use several connections at once; each is an independent link with its own session.
- The host writes **nothing but whole frames**: no preamble, no line endings, no break. A device **MAY** rely on this (the MicroPython library does, §13).

- The host opens the port when the circuit starts running and closes it when it stops. Opening the port resets many boards (a classic Arduino's auto-reset), and a bootloader may swallow bytes for a second or two. Many others — native-USB boards such as a Leonardo, and most Python boards — do **not** reset, and their program carries on from one run into the next. So a device learns that a run has begun from the handshake (§5.4), never from its own start-up.

3. Frames

3.1 Frame format

START	TYPE	SEQ	LEN	PAYLOAD	CRC_LO	CRC_HI
0x7E	1	1	1	LEN bytes	1	1

Field	Size	Meaning
START	1	Always 0x7E , never escaped. Begins a frame.
TYPE	1	The frame type (§4).
SEQ	1	In OUTPUT and INBOUND : the frame's sequence number, 1–255 (§6.1); one with SEQ 0 is ignored. In ACK : the sequence number acknowledged. In every other frame: 0, which the receiver MUST ignore.
LEN	1	The number of payload bytes, counted before escaping: 0–255.
PAYLOAD	LEN	Defined per type (§4).
CRC	2	CRC-16/CCITT-FALSE (§3.3) of the unescaped bytes TYPE SEQ LEN PAYLOAD , little-endian.

A frame's **body** is everything after START , from TYPE to CRC_HI .

3.2 Size limits

Quantity	Bytes
Payload	0–255
Body, unescaped: TYPE + SEQ + LEN + payload + CRC = 1 + 1 + 1 + 255 + 2	at most 260
Frame on the wire: START + every body byte escaped = 1 + 2 × 260	at most 521

- A sender MUST NOT send a payload longer than 255 bytes: LEN could not say so.

- Every receiver has a **payload limit**, the longest payload it accepts. The host's is 255. A device's MAY be smaller, but MUST be at least 7 — the longest payload the host sends (`HELLO`).
- A receiver that reads a `LEN` greater than its payload limit MUST treat the frame as damaged at once (§3.5), without waiting for the rest of it.

Note: so a receiver never holds more than 3 + its limit + 2 bytes of a frame — 260 for the host, 12 for a device with the smallest limit. Escaping needs no buffer of its own: each escape pair is undone as it arrives.

3.3 CRC

CRC-16/CCITT-FALSE: polynomial `0x1021`, initial value `0xFFFF`, input and output not reflected, no final XOR. The check value, for the ASCII bytes `123456789`, is `0x29B1`.

```
uint16_t crc16(const uint8_t *p, size_t n) {
    uint16_t crc = 0xFFFF;
    while (n--) {
        crc ^= (uint16_t)(*p++) << 8;
        for (uint8_t i = 0; i < 8; i++)
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
    }
    return crc;
}
```

3.4 Escaping

Every body byte that is `0x7E` or `0x7D` is sent as two bytes: `0x7D`, then the byte XOR `0x20`. No other byte is escaped.

Unescaped	On the wire
<code>0x7E</code>	<code>0x7D 0x5E</code>
<code>0x7D</code>	<code>0x7D 0x5D</code>

Escaping applies to every body byte — `TYPE`, `SEQ`, `LEN`, the payload and both CRC bytes — while `LEN` and the CRC are computed over the unescaped bytes. So `0x7E` on the wire is **always** the start of a frame, which is how a receiver finds the next frame after noise, a torn frame or a reset.

3.5 Receiving a frame

A receiver is either **hunting** (between frames) or **in a frame**.

1. While hunting, it discards every byte but `0x7E` , which begins a frame.
2. In a frame, a `0x7E` abandons the partial frame and begins a new one. A frame cut short this way is **born**, not damaged: it is dropped without a word, and the sender's timer (§6.4) covers it. Any other byte is unescaped (§3.4) and read, in order, as `TYPE` , `SEQ` , `LEN` , `LEN` payload bytes and the two CRC bytes.
3. The frame is **damaged**, and the receiver goes back to hunting, when:
 - a `0x7D` is followed by anything but `0x5E` or `0x5D` — or `0x7E` , which tears the frame instead (step 2);
 - `LEN` is greater than the receiver's payload limit (§3.2) — known as soon as `LEN` is read; or
 - the CRC does not match — known when the second CRC byte is read.
4. A damaged frame is discarded. If the receiver is in a session (ACTIVE, §5.2 and §5.3), it MUST then send a `NAK` (§4.4) at once — **unless** the damaged frame's `TYPE` , as received, is `HELLO` , `HELLO_ACK` , `ACK` , `NAK` or `LOG` : frames never resent in answer to a NAK. A frame damaged before its `TYPE` arrived is NAKed. A receiver not in a session MUST NOT send a `NAK` : before a handshake, damage is most likely a bootloader's noise.
5. An intact frame is dispatched by `TYPE` . A frame of a type the receiver does not know, or never receives (a device never receives `INBOUND`), MUST be ignored.

Note: a `LEN` damaged into a larger value is damage at once if it exceeds the receiver's payload limit (step 3). Otherwise the receiver waits for bytes that belong to the next frame, whose `0x7E` then tears it (step 2), and the sender's timer (§6.4) covers the loss. A `LEN` damaged into a smaller value fails the CRC.

4. Frame types

Value	Name	Sent by	SEQ	Payload	Delivery
0x01	HELLO	host	0	7 bytes (§4.1)	resent until answered (§5.4)
0x02	HELLO_ACK	device	0	7 bytes (§4.1)	—
0x06	ACK	either	the SEQ acknowledged	none (§4.3)	—
0x15	NAK	either	0	none (§4.4)	—
0x10	OUTPUT	host	1–255	4 bytes (§4.2)	reliable (§6)
0x11	INBOUND	device	1–255	4 bytes (§4.2)	reliable (§6)
0x20	LOG	device	0	1–255 bytes (§4.5)	not acknowledged

The values 0x00, 0x7D, 0x7E and 0xFF are never frame types. Every other unassigned value is reserved for later versions (§11).

4.1 HELLO and HELLO_ACK

The payload of both (7 bytes):

Offset	Size	Field
0	1	Protocol version: 1.
1	2	Session (§5.1), little-endian.
3	4	Layout signature (§7.3), little-endian.

For protocol version 1, the first three payload bytes are the version and the session. They are all a device needs to answer a HELLO, and all the host needs to judge a HELLO_ACK's version. A later version MAY change the HELLO and HELLO_ACK payloads after establishing that it is speaking to a different version (§11).

- The host sends HELLO with its version, the session it has chosen and the signature of the layout it expects.
- A device MUST answer every intact HELLO of at least 3 bytes, for any session but 0, with a HELLO_ACK carrying the HELLO's session and **its own** version and signature — in any state, and even

when they differ from the `HELLO` 's. Only the host has to explain a mismatch, so only the host needs to see one. (A `HELLO` longer than the device's payload limit is not intact: it is damaged, §3.5.)

- A device MUST ignore a `HELLO` shorter than 3 bytes, or for session 0, without answering it.
- A `HELLO_ACK` for session 0 is the device's **announcement** that it is in no session (§5.3) — sent when its program starts, and when it leaves a session after a failed delivery. It answers nothing.
- A host MUST ignore a `HELLO_ACK` shorter than 3 bytes. What it does with any other is in §5.2 and §5.4.

4.2 OUTPUT and INBOUND

`OUTPUT` carries an Output's value from the circuit to the device; `INBOUND` carries an Input's value from the device to the circuit. The payload of both (4 bytes):

Offset	Size	Field
0	1	Element index (§7.1), within this connection and direction.
1	1	Width, 1–16 : the element's pin count.
2	2	Value, little-endian and right-aligned: bit 0 is the element's pin 1.

- A sender MUST send every value bit at or above the width as 0. A receiver MUST ignore them.
- An element is an ordered list of **fields** — bit (1 pin), byte (8) or word (16), 16 pins in all at most — packed into the value in order from bit 0. A byte followed by a bit puts the byte in bits 0–7 and the bit in bit 8. The fields need no wire support of their own: they only say which bits the generated code hands to which parameter, which is why the layout signature covers them (§7).
- An `INBOUND` always carries the **whole** element — there are no partial updates. The device keeps its own copy of the state and sends all of it.
- A data frame is **unusable** when it is intact but its `LEN` is not 4, its width is not 1–16, its index names no element, its width is not that element's, or (at a device) the program has no handler for its Output. An unusable frame MUST be acknowledged like any other (§6.2) and then discarded: a resend would carry the same bytes.

4.3 ACK

`SEQ` is the sequence number of the data frame acknowledged. The payload is empty, and a receiver MUST NOT examine it. When an `ACK` is sent, and what one means, is §6.2.

4.4 NAK

`SEQ` is 0 and the payload empty; a receiver MUST NOT examine either.

A `NAK` means: *a frame arrived damaged — if you have a data frame outstanding, send it again now*. It names no frame, because the damaged frame's `TYPE` and `SEQ` cannot be trusted; and it needs to name none, because each side has at most one data frame outstanding (§6).

- A side receiving a `NAK` with a data frame outstanding **MUST** retransmit that frame at once — the same bytes, the same `SEQ` — and **MUST NOT** advance its sequence counter. The retransmission is one of the frame's sends (§6.4).
- A side receiving a `NAK` with nothing outstanding **MUST** ignore it.

A `NAK` is never acknowledged, never resent and never answered with a `NAK`, so it can cause no more retransmissions than the send limit allows.

4.5 LOG

The payload is 1–255 bytes of UTF-8 text, with no terminator. A newline (`0x0A`) ends a line; text without a trailing newline continues on the same line when the next `LOG` arrives.

- A sender **MUST NOT** send an empty `LOG`, and **SHOULD** split longer text between characters, never inside a multi-byte UTF-8 sequence. A receiver **MUST** ignore an empty `LOG`, and **SHOULD** decode `LOG` text as one stream, so that a character split across frames still arrives whole.
- `LOG` is **not acknowledged**: its `SEQ` is 0, and it is never NAKed, resent or de-duplicated. Log text changes nothing on the circuit, so losing a line to a damaged frame is cheap, and blocking the device on every `print()` is not.
- A device **MAY** send `LOG` in any state, in a session or not. Text printed during start-up reaches a host that is listening, and costs nothing when none is.
- `LOG` never touches the circuit. The host shows the text in the connection's window.

5. Sessions

5.1 Session numbers

A session number is 16 bits, **1–65535**. **0** is reserved: it is never a run's session, and a `HELLO_ACK` for session 0 is a device's announcement that it is in no session.

- The host **MUST NOT** use the session of its previous handshake on the same connection again, and **SHOULD** make repeating any recent session unlikely. Chip Hippo counts upward, across all its connections, wrapping $65535 \rightarrow 1$, from a random starting point.
- *Note*: the session is what tells a device that did not reset between runs that a new run has begun, and what tells a late copy of this run's `HELLO` from the next run's. A host picking, by chance, the very session a device still remembers from before would be taken for a late copy: the device would keep

the old run's sequence numbers and not see a new run begin. Sixteen bits make that a 1-in-65535 event on each restart of Chip Hippo.

5.2 Host states

State	Accepts	Ignores	Moves on
CLOSED	—	—	Run opens the port → HANDSHAKE.
HANDSHAKE	HELLO_ACK for its session; LOG	Every other frame — including the announcement and a HELLO_ACK for any other session. Damage is not NAKed.	Its session is established → ACTIVE. A mismatch, no answer in 5 s, or the port lost → FAILED.
ACTIVE	INBOUND , ACK , NAK , LOG ; the announcement	Every other HELLO_ACK (a late copy of its own, another session's); types it never receives	The announcement, a failed delivery (§6.4), or the port lost → FAILED. Stop → CLOSED.
FAILED	LOG	Everything else. Damage is not NAKed.	The run stops and the port closes → CLOSED.

- The host sends data frames only while ACTIVE: never during the handshake, and nothing at all once FAILED.
- **The announcement while ACTIVE** means the device is in no session: it has restarted, or it has left this session after one of its own deliveries failed (§5.3). Either way it will ignore everything until greeted again, which only a new run does, so the host fails what it has outstanding and stops the run. Without the announcement, a device that had left on an Inputs-only design would simply fall silent.
- **The port lost** — a cable pulled, a board gone — stops the run the same way.
- **On FAILED** the host fails every Output waiting to be sent, stops the run and tells the user which connection failed and why: no answer, another protocol version, another layout, the device leaving the session, a failed delivery, or a dropped connection. It still shows log text until the port closes — often the device's own account of what happened.
- There is no goodbye frame. Stop closes the port, and a device learns of it only when a send of its own fails (§5.3).

5.3 Device states

A device remembers the **last session** a HELLO named — 0 when its program starts.

State	Accepts	Ignores	Moves on
WAITING	HELLO	Every other frame. Damage is not NAKed, and no data frame is sent.	A HELLO for a new session that matches the device → ACTIVE.
ACTIVE	HELLO , OUTPUT , ACK , NAK	Types it never receives	A HELLO for a new session → ACTIVE if it matches, else WAITING. A failed delivery → WAITING, announced.

- **Start.** A program starts WAITING, its last session 0, and MUST send the announcement once: a HELLO_ACK for session 0 carrying its version and signature.
- **A HELLO for the last session** — a late copy; the host resends until it hears — is answered and changes nothing else, in either state. In particular it MUST NOT reset the device's sequence numbers: rewind, its next INBOUND would look like a duplicate to the host and be dropped without a word.
- **A HELLO for a new session** is answered, and starts that session. The device MUST discard everything it held for the previous one: its sequence numbers (§6.1), an OUTPUT held for later (dropped), an OUTPUT whose handler is running (never acknowledged — the run that sent it has gone) and an INBOUND waiting for its ACK (given up at once). It records the new session as its last, and becomes ACTIVE only if the HELLO **matches**: exactly 7 bytes, version 1, and the device's own layout signature. Otherwise it is WAITING.
- **A handler belongs to its OUTPUT's session.** If that session ends while the handler runs — a new HELLO , or a failed delivery — every send the handler makes afterwards MUST fail at once: its Inputs belong to a run that has gone, and must not reach the next one.
- **Becoming ACTIVE** is the moment for the device to send every Input its starting value: an Input drives nothing until its first INBOUND (§8). The generated libraries run the program's onConnect function (on_connect in Python) here — the only sign a board that did not reset gets that a new run has begun.
- **A failed delivery (§6.4)** means nobody is listening: the device leaves the session. It MUST discard a held OUTPUT , MUST NOT acknowledge an OUTPUT whose handler is running, and MUST then send the announcement, so that a host still listening stops rather than wait on a device that no longer answers. It returns to WAITING — **offline** — and keeps its last session, so only the next run's HELLO brings it back. While offline, a send fails at once instead of waiting three times 500 ms, which keeps a program left running after Stop from crawling: there is no goodbye frame, and a board that resets on open does not reset on close. The generated libraries report the state as connected() .

Note: the byte stream keeps two sessions apart without a session number in every frame. Whatever a device sent in an old session precedes its HELLO_ACK for the new one, and the host accepts nothing but that HELLO_ACK until it arrives; after it, the device sends nothing that belongs to the old session. The same holds the other way round.

5.4 The handshake

1. **Open.** The host opens the port. On many boards this resets the device (§2).
2. **Greet.** The host chooses a session (§5.1) and sends `HELLO` at once, then again every **250 ms** — the same session each time — until a `HELLO_ACK` for that session arrives, for up to **5 s**.
3. **Judge.** Of a `HELLO_ACK` for its session, the host:
 - refuses the run — **version mismatch** — if its version is not 1, whatever its length;
 - ignores it and keeps waiting if it is not exactly 7 bytes;
 - refuses the run — **layout mismatch** — if its signature is not the host's;
 - otherwise establishes the session and becomes ACTIVE.
4. **No answer** in 5 s refuses the run.

A run is all-or-nothing: if any of its connections fails to open, or fails its handshake, every port the run opened is closed again.

Note: the device judges the `HELLO` as the host judges the `HELLO_ACK` (§5.3), so after a mismatch neither side is in the session. A device built for another design never runs `onConnect`, or sends an Input, into a run that has refused it.

6. Reliable delivery

`OUTPUT` and `INBOUND` are delivered **stop-and-wait**. A side has at most one data frame outstanding, and sends the next only when that one has been acknowledged or its delivery has failed.

6.1 Frame identity and sequence numbers

- A data frame is identified by its **session, direction and SEQ**. The two directions number their frames independently: the host's `OUTPUT` 7 and the device's `INBOUND` 7 are different frames, and an `ACK` 7 acknowledges whichever of them its receiver sent.
- Each side has a send counter for the session, starting at 0. A new data frame takes the counter's next value, 1–255, wrapping 255 → 1: a session's first data frame in each direction is `SEQ` 1. `SEQ` 0 is never a data frame's: a receiver MUST ignore an `OUTPUT` or `INBOUND` carrying it — neither acknowledge nor process it.
- A retransmission, after a timeout or a `NAK`, is the same frame: the same bytes and the same `SEQ`. The counter advances only for a new frame.

6.2 Acknowledgement

In a session, a receiver acknowledges every intact data frame — at the time the table below gives — with an `ACK` carrying its `SEQ`. A frame is **delivered** when an intact `ACK` with its `SEQ` arrives while it is

outstanding. Any other `ACK` — another `SEQ` , or nothing outstanding — is **stale**, and MUST be ignored.

Receiver	Frame	The ACK is sent...
device	<code>OUTPUT</code>	after the Output's handler returns.
host	<code>INBOUND</code>	as soon as the frame arrives — even while the circuit is stalled. The value is applied at the next settle boundary (§8).

Acknowledging an `OUTPUT` after its handler means everything the handler sent — each `INBOUND` , delivered and acknowledged, and each `LOG` — reaches the host **before** the `ACK` . The host stalls the circuit until every `OUTPUT` is acknowledged, so it always sees the device's reaction before the circuit moves on. That is what makes request and response deterministic (§8).

- A handler that fails still counts as returned: the Python library catches the exception, logs it, and acknowledges.
- A device MUST NOT run a handler from inside another handler, or while it waits for an `INBOUND` 's `ACK` . An `OUTPUT` arriving then is held — the host sends one at a time, so one slot is enough — run afterwards, and acknowledged when it has run.
- Handlers must return promptly: within 500 ms if possible, and certainly within the host's three sends, 1.5 s in all (§6.4). Long work belongs in the program's main loop.

6.3 Duplicates

Each receiver remembers the `SEQ` of the last data frame it acknowledged in the session — none, when the session begins.

- A data frame carrying that `SEQ` is a **duplicate**: its `ACK` was lost and the sender has resent it. The receiver MUST acknowledge it again and MUST NOT process it again. Without this, a lost `ACK` would call an Output's handler twice.
- **Exception**: a duplicate of the `OUTPUT` whose handler is still running, or which the device is holding to run next, MUST be ignored — not acknowledged. Its `ACK` follows when the handler returns. This happens whenever a handler sends an `INBOUND` : the device reads the port while it waits for that `ACK` , and the host's timer can resend the `OUTPUT` in the meantime. Acknowledging it then would let the circuit run on before the handler's `INBOUND` had been applied.

Note: because a sender never has two frames outstanding, a receiver only ever sees the frame it last acknowledged again, or the next one. One remembered `SEQ` is enough to tell them apart.

6.4 Timeout, NAK and retry

- **The timer**. Each transmission of a data frame — the first, or any retransmission — starts an ACK timer of **500 ms** when the frame has been written to the port.

- **Retransmit** the outstanding frame when its timer expires, or at once when a `NAK` arrives (§4.4). The retransmission restarts the timer.
- **The limit.** A frame is sent at most **3 times** in all, the first transmission included, whatever caused the resends. If the timer of the third transmission expires, or a `NAK` arrives after it, the delivery has **failed**:
 - the **host** stops the run (FAILED, §5.2). Whatever was queued behind the frame fails with it.
 - the **device** drops the frame, reports the failure to its caller (the generated `send()` returns false), and leaves the session, saying so with the announcement (§5.3) — which stops a host still listening.
- `ACK` s, `NAK` s and retransmissions are the protocol's own traffic, not the circuit's: they never light Chip Hippo's activity lamps. The connection window's protocol view shows them, a resend as a warning.

6.5 Ordering

Each direction's data frames are processed in the order they were sent, since each waits for the one before. The two directions interleave freely. A side waiting for an `ACK` MUST keep receiving meanwhile: it acknowledges the other side's data frames, answers a `HELLO` and acts on a `NAK` as usual. `LOG` frames sit in the byte stream in the order they were written, among the data frames around them.

7. Element indices and the layout signature

Names never travel over the wire. Elements are identified by index, and the layout signature makes sure both ends number them alike.

7.1 Element indices

- Indices are assigned **per connection, per direction**, from 0: Outputs 0 to $n-1$, Inputs 0 to $m-1$.
- The generated code, the element index in every data frame, and the layout string (§7.2) all use this one numbering.
- *Note:* Chip Hippo orders each direction by the number in the element's internal id (`out2` before `out10`). That number only ever counts up, so an element's index changes only when one before it, on the same connection and in the same direction, is added or removed.

7.2 The layout string

A connection's layout is written as an ASCII string, with no byte-order mark, no NUL terminator, no newline and no spaces:

```

layout = [ element *( "," element ) ]
element = kind index ":" width *( "+" width )
kind     = "O" / "I"
index    = "0" / ( %x31-39 *DIGIT ) ; decimal, no leading zeros
width    = "1" / "8" / "16"          ; one field: bit, byte or word

```

- Every Output comes first, in index order, then every Input, in index order.
- An element lists its fields' widths in field order, joined by `+` ; they add up to the element's width, at most 16.
- A connection with no elements has the empty string as its layout.

For example, Outputs of fields `[byte, bit]` and `[bit]` , and an Input of one word, are `00:8+1,01:1,I0:16` .

7.3 The signature

The **layout signature** is the CRC-32 (IEEE 802.3, also known as CRC-32/ISO-HDLC) of the layout string's bytes: polynomial `0x04C11DB7` , reflected (`0xEDB88320`), initial value `0xFFFFFFFF` , final XOR `0xFFFFFFFF` . It travels little-endian. The check value, for the ASCII bytes `123456789` , is `0xCBF43926` ; the empty layout's signature is `0x00000000` .

Layout string	Signature
<code>00:8,01:1,I0:16</code>	<code>0xF8ACB506</code>
<code>00:8+1,01:1,I0:16</code>	<code>0x7EF66B41</code>
<code>00:1,I0:1+1+1+1</code>	<code>0x0EC32022</code>
<code>I0:16</code>	<code>0x43F1C7E9</code>

The signature covers what decides which bits reach which parameter: the elements, their order, and each one's field boundaries, not only its total width. A `[bit, byte]` element reordered to `[byte, bit]` keeps its width of 9 but would hand every value to the wrong parameters, so it must be refused.

Everything else is the circuit's business and changes nothing on the wire: names, colours, descriptions, where the tags are planted, and each element's trigger. Renaming an element or a field changes the generated code's **design hash** — a second fingerprint, written into the generated file as a comment and compared by the Generate button — so the code reads as out of date, but a run still works until it is regenerated. The design hash never travels on the wire.

8. What the values mean

The protocol moves values; this is what Chip Hippo does with them, which a device has to know to answer sensibly.

- **Pins and bits.** Pin k of an element is bit $k-1$ of its value. On the host, a pin whose tag isn't planted on the board, or whose net is floating or unknown, reads as **0**.
- **Chip Hippo looks only at settle boundaries** — the moments the circuit has stopped changing. A glitch inside one settle is never sent anywhere, and nothing from the device lands on the board halfway through a ripple.
- **When an OUTPUT is sent.** An Output with an **edge** trigger (rising, falling or either) is sent when its trigger line's settled level has made that transition since the previous boundary (its *Trigger starts* setting stands in for "previous" at the first). An Output on **Auto** has no trigger line: it is sent whenever the value its pins settle to differs from the last value it sent, **and always at the first boundary of a run**, so the device learns where the circuit starts.
- **The stall.** Once a boundary has sent its **OUTPUT**s, the circuit stands still — no clock edges, no settles — until every one is acknowledged, that is, until every handler has returned (§6.2).
- **When an INBOUND is applied.** A received value is held, and a newer one for the same Input replaces one not yet applied. An Input on **Auto** is *live*: its value goes on the board at the next boundary — at once, if the circuit is idle. An Input with an edge trigger releases its value only when its trigger line makes that transition, the way a latch loads on its clock. All the values eligible at a boundary are applied together, and the circuit settles again. A value that repeats what the Input already drives changes nothing.
- **Request and response.** An **INBOUND** that arrives while the circuit is stalled on an **OUTPUT** still counts for that boundary. So a handler that answers with an Input lands its answer before the circuit moves on.
- **Inputs start undriven.** An Input drives its pins at the strength of a chip output — two drivers disagreeing on one net are reported as a conflict — but drives **nothing at all** until its first **INBOUND** of the run. Send every Input its starting value when a session begins (§5.3).

9. Invariants

The rules above, reduced to what must always hold:

1. **SEQ** is non-zero only in data frames and **ACK**s. **HELLO**, **HELLO_ACK**, **NAK** and **LOG** carry 0; a session travels in the **HELLO** payload; a data frame with **SEQ** 0 is ignored.
2. Each side has at most one data frame outstanding.
3. A retransmission is the same frame — the same bytes, the same **SEQ** — and a data frame is sent at most 3 times.

4. A data frame is identified by session, direction and `SEQ` . The two directions' counters are independent.
5. An `ACK` carries the `SEQ` of the frame it acknowledges, and only an `ACK` with the outstanding frame's `SEQ` delivers it.
6. A receiver processes each data frame at most once: a duplicate is acknowledged again, never processed again.
7. A device acknowledges an `OUTPUT` only after its handler has returned, and never one received in a session it has since left.
8. `HELLO` , `HELLO_ACK` , `ACK` , `NAK` and `LOG` consume no sequence numbers and are never acknowledged.
9. A `NAK` names no frame, and causes no more retransmissions than the send limit allows.
10. `LOG` is never acknowledged, resent or de-duplicated, and may be sent outside a session.
11. Session 0 is never a run's: a `HELLO_ACK` for session 0 is the device's announcement that it is in no session.
12. Only a `HELLO` for a new session resets a device's session state.
13. Neither side enters a session unless the versions and the layout signatures match.
14. A failed delivery ends the session on both sides: the host stops the run; the device leaves it and announces that, which stops a host still listening, and stays offline until the next session.
15. A device sends nothing that belongs to a session it has left — not an `ACK` , and not an `INBOUND` from a handler of that session.

10. Constants

Constant	Value	Meaning
PROTOCOL_VERSION	1	This protocol's version.
START	0x7E	Begins a frame.
ESC	0x7D	Escapes the next byte.
ESC_XOR	0x20	XORed into an escaped byte.
HEADER_BYTES	3	TYPE , SEQ , LEN .
CRC_BYTES	2	The CRC.
MAX_PAYLOAD	255	The longest payload.
MAX_FRAME_BODY	260	The longest body, unescaped.
MAX_WIRE_FRAME	521	The longest frame on the wire.
DATA_PAYLOAD	4	An OUTPUT or INBOUND payload.
HELLO_PAYLOAD	7	A HELLO or HELLO_ACK payload.
HELLO_PREFIX	3	The shortest HELLO a device answers, and HELLO_ACK a host reads: version, session.
MAX_HOST_PAYLOAD	7	The longest payload the host sends: a device's smallest payload limit.
ANNOUNCE_SESSION	0	The session of the announcement: the device is in no session.
MAX_SESSION	65535	Sessions are 1 to this.
MAX_SEQ	255	Data SEQ s are 1 to this.
MAX_WIDTH	16	The widest element, in pins.
CRC16_POLY	0x1021	The frame CRC's polynomial.
CRC16_INIT	0xFFFF	The frame CRC's initial value.
HELLO_INTERVAL_MS	250	How often the host sends HELLO while it waits.
HELLO_WINDOW_MS	5000	How long the host waits for HELLO_ACK .
ACK_TIMEOUT_MS	500	The ACK timer.
MAX_SENDS	3	The most times a data frame is sent.

These are protocol values, not settings: no connection overrides them. The names are those of Chip Hippo's source (§13).

11. Versions

- This page is version **1**. A host and a device talk only when their versions are equal.
- In version 1, the version is the first byte of the `HELLO` and `HELLO_ACK` payloads, and the session the next two (§4.1). A version-1 device answers a `HELLO` of any version with its own version, and joins only a version-1 `HELLO` (§5.3). A version-1 host refuses a `HELLO_ACK` of any other version as a version mismatch, whatever its length (§5.4).
- Any change to a frame's layout, the frame types, the CRC or the meaning of a field is a new version. A later version MAY change the `HELLO` and `HELLO_ACK` payloads after establishing that it is speaking to a different version.
- *Note:* a version-1 side answers or refuses another version, rather than falling silent, only when it can read the frame: framed as in §3, the version first and the session next — and, sent to a version-1 device, no longer than 7 bytes (§3.2).
- Not part of version 1: a goodbye frame, a device-initiated handshake (the announcement tells the host something happened; it establishes nothing), windowed delivery, flow control and keep-alives.

12. Worked examples

These examples use the layout `00:8,01:1,I0:16` — signature `0xF8ACB506`, sent as `06 B5 AC F8` — and session `0x1234`, sent as `34 12`. Each frame is written as it is on the wire. They are test vectors: Chip Hippo's test suite checks every frame on this page against its encoder.

12.1 The handshake

The host's `HELLO`: version 1, session `0x1234`, its signature (CRC `0x9C63`).

```
7E 01 00 07 01 34 12 06 B5 AC F8 63 9C
```

The device's answer, `HELLO_ACK`: the same session, with its own version and signature (CRC `0x2DAC`).

```
7E 02 00 07 01 34 12 06 B5 AC F8 AC 2D
```

The same device announcing its start-up: a `HELLO_ACK` for session 0 (CRC `0x4458`).

```
7E 02 00 07 01 00 00 06 B5 AC F8 58 44
```

Escaping reaches the session as well. Session `0x7D7E` is sent as `7E 7D`, both of them markers, so a `HELLO` for it is (CRC `0x0C54`):

```
7E 01 00 07 01 7D 5E 7D 5D 06 B5 AC F8 54 0C
```

12.2 An OUTPUT and its ACK

The host sends `OUTPUT SEQ 1`: element 0, width 8, value `0x7E` — deliberately a value that needs escaping.

- Unescaped: `TYPE=10 SEQ=01 LEN=04 PAYLOAD=00 08 7E 00`.
- CRC-16 of `10 01 04 00 08 7E 00` is `0x88E4`, sent as `E4 88`.

```
7E 10 01 04 00 08 7D 5E 00 E4 88
```

The device calls the handler for Output 0 with `0x7E`. The handler returns, and the device sends `ACK 1` (CRC `0x4D0D`):

```
7E 06 01 00 0D 4D
```

Escaping reaches the CRC too. `ACK 16` has the CRC `0x7D4F`, and `ACK 17` `0x4E7E`:

```
7E 06 10 00 4F 7D 5D
```

```
7E 06 11 00 7D 5E 4E
```

12.3 An INBOUND, a LOG and a NAK

The device sends `INBOUND SEQ 1` — its own first data frame, since each direction counts separately (§6.1): element 0, width 16, value `0xBEEF` (CRC `0xB788`).

```
7E 11 01 04 00 10 EF BE 88 B7
```

The host acknowledges it with the same bytes as the `ACK` in §12.2: `7E 06 01 00 0D 4D`. It is the device's frame this `ACK` answers, because the device receives it.

The device logs `ok` and a newline, unacknowledged (CRC `0x6104`):

```
7E 20 00 03 6F 6B 0A 04 61
```

A `NAK`, from either side (CRC `0x640F`):

```
7E 15 00 00 0F 64
```

12.4 A retransmission

The host sends Output 1 (width 1) the value 1, as `SEQ 2`. The frame is damaged on the way, and the device's `ACK` is later lost:

Step	Direction	What happens	On the wire
1	host → device	<code>OUTPUT SEQ 2</code> , first send. It arrives damaged.	<code>7E 10 02 04 01 01 01 00 46 B6</code>
2	device → host	The damage is <code>NAKed</code> .	<code>7E 15 00 00 0F 64</code>
3	host → device	Second send, at once: the same bytes.	<code>7E 10 02 04 01 01 01 00 46 B6</code>
4	device → host	The handler runs; <code>ACK 2</code> . It is lost.	<code>7E 06 02 00 5E 18</code>
5	host → device	500 ms after step 3, the third and last send.	<code>7E 10 02 04 01 01 01 00 46 B6</code>
6	device → host	A duplicate of the last frame acknowledged: <code>ACK 2</code> again, the handler not run again.	<code>7E 06 02 00 5E 18</code>

Had step 6 been lost too, the host would have stopped the run 500 ms after step 5. At no point does the `SEQ` change, and the handler runs once.

13. Implementations

Four implementations speak this page, and each is tested against the others.

- **The host** — Chip Hippo's main process: the bytes in `src/app/serial/protocol.js`, the session and reliability rules in `src/app/serial/link.js`. Every number on this page lives in one module, `src/web/scripts/model/serial-wire.js` — the names in §10 are its exports — which the host reads

and the code generators write the devices' constants from, so the two ends cannot drift. The element order and the layout string are one function each (`elementsFor` and `layoutString` in `src/web/scripts/model/integration.js`), called by the generators and by the run-time handshake alike.

- **The C++ header** (`ChipHippo.h` , from `Generate`, for a connection whose Language is C++) — the device side for Arduino boards. Its payload limit is the smallest allowed, so it buffers 12 bytes of a frame; it computes the CRC a bit at a time (no table, so no flash spent on one); it decides whether a `HELLO` matches by comparing it, byte for byte, with the `HELLO_ACK` it answers with; and it sends log text in frames of up to 60 bytes — at every newline, at every `poll()` , and before every `INBOUND` and every `ACK` of an `OUTPUT` .
- **The Python module** (`chippippo.py` , for a Language of Python) — the header said again, line for line, for MicroPython and CircuitPython. It picks its port by platform. On **MicroPython** it uses the REPL's own USB port, with Ctrl-C switched off inside a frame (where `0x03` is data); since the host never sends a byte outside a frame (§2), a lone `0x03` between frames can only be a tool such as Thonny or `mpremote` , and it stops the program as Ctrl-C would. (The module reads a damaged frame to the end its `LEN` gives it, so a `0x03` in the rest of one is still that frame's.) On a board whose USB is a separate serial chip (a classic ESP32 DevKit) that port always runs at 115200 baud, 8N1, so the connection must too. On **CircuitPython** it uses the second USB serial port, which its `boot.py` turns on. Any stream can be handed to `begin()` instead.
- **The Mock** — Chip Hippo's built-in pretend board (`src/app/serial/mock-device.js`): the device side in JavaScript, behind the same port interface as a USB cable, so the host cannot tell the two apart. Its one-shot faults — drop the next `ACK` , corrupt the next frame, ignore a handshake, answer with the wrong signature — exercise the recovery rules (see [The Mock connection](#)).

Chip Hippo's test suite checks this page's frames, constants and signatures against the code, drives each device — a generated header compiled with the host's C++ compiler, the Python module under Python and MicroPython, and the Mock — with the real host link, and feeds all of them the same bytes to show they answer alike.

Files, Saving & Undo

Chip Hippo keeps your work in **one ordinary file** you can see, move, and share. This page covers what's in it, the four commands that manage it, the dirty marker, and undo/redo.

One file is the whole design

The desk in front of you is always a **desktop of an open project**, from the very first launch. A brand-new project has no name, no home of its own, and exactly one desktop called `Desktop 1` — which is simply a desk to build on. See [Projects & Desktops](#) for keeping several desktops side by side.

The project is the document. A single `.chiphippo` file holds all of it:

- every desktop — its boards, chips, wiring, parts, and net names;
- what those desktops are called, and which one you were on;
- whether you left the [desk padlock](#) shut;
- the contents of every ROM you programmed.

Nothing else is needed to open a design. Copy that one file to another machine and it opens whole, ROM contents included — which is what makes a Chip Hippo design something you can actually send to someone.

Before you choose where it goes

Chip Hippo never stops you mid-thought to ask for a file name. A project that hasn't been given a home lives in a **working slot** inside the app's own data directory, and `Cmd/Ctrl+S` writes it there with no dialog at all. Design something, save, quit — it's there when you come back.

Save As... is what gives the project a real file, when you want one. From then on Save just writes that file. When you launch, Chip Hippo opens the unsaved project from its working slot if there is one; otherwise it reopens the **project you used most recently**.

New, Open, Save, Save As

Four commands — on the **File** menu, in the header toolbar's File pill, and under the usual shortcuts — manage the project's file:

- **New Project** (`Cmd/Ctrl+N`) — a fresh unnamed project with one empty desktop. What's open now is asked about first.

- **Open...** (`Cmd/Ctrl+O`) — shows a native file picker. A `.chhippo` project opens as itself; a loose design or an exported desktop opens as a new project of one desktop.
- **Save** (`Cmd/Ctrl+S`) — writes the project to its file, or to the working slot when it hasn't got one yet. No dialog either way.
- **Save As...** (`Shift+Cmd/Ctrl+S`) — asks where to keep it and writes it there. An unnamed project takes its **name from the file you pick**, so there's no separate naming step.

If the file you pick already exists, your system's own save dialog asks whether to replace it, exactly as it does in any other app — Chip Hippo adds no second question of its own. Decline there and nothing is written.

File ▶ Project Properties... shows the project's Location alongside its Name and Description, without opening any dialog that writes.

Unsaved changes

The window title shows a • when the project has changes that aren't in its file yet: `• 6502 SBC — Clock module — Chip Hippo`. Save clears it. There is one marker because there is one document — it covers every desktop's design *and* the desktops themselves, so adding one, renaming one, or deleting one shows up there exactly as wiring a board does.

Nothing is written to your file until you save. Add a desktop, rename it, delete another — none of it touches your `.chhippo`, and closing the project without saving takes all of it back. Tabs carry no marker of their own for the same reason: a desktop can't be saved on its own, so a dot on one would have nothing to clear it.

And nothing is lost to a crash either. About every 30 seconds, Chip Hippo tucks a copy of whatever you're working on into its own private folder — not into your file. So both things are true at once: the • still means "not in my file yet", closing without saving still takes it all back, and a power cut or a crash still costs you half a minute at worst.

If the app does go down unexpectedly, the next launch puts that work straight back on the desk and tells you so. It arrives **unsaved** — the • is showing — so you decide the same way you always would: save it, or close without saving to go back to what your file holds. A project you'd never saved anywhere is kept too; it was already living in that folder, and it still opens on its own.

You can ignore all of it. There's no setting, no prompt to answer, and nothing appears while you work. ⌘S is still what commits your work to your file, and still worth pressing when you want it there *now*.

Nothing that would lose your changes happens silently. Starting or opening another project asks first, and so does quitting. Both offer to **save** — and choosing that carries the action through, so you are never made to ask twice.

A project still in the **working slot** is asked about every time, even when nothing is unsaved: the incoming project is about to claim that slot, so there's nowhere for this one to stay. **Save** there means Save As — a file of its own.

When the save is the app's idea rather than yours — quitting with a • in the title — nothing is asked about *where*: the project goes to the file it already has, or to the working slot, where the next launch will find it.

Two things that are deliberately **not** counted as changes: which desktop is on the desk, and where the camera is. Moving between tabs and panning around never mark a design dirty.

Undo & redo

`Cmd/Ctrl+Z` undoes the last edit; `Shift+Cmd/Ctrl+Z` redoes it. Undo/redo covers the full editing history of the circuit — placing and moving boards, chips, and discretes; wiring and unwiring; deleting anything; net names — as far back as your session's history allows, and both menu items disable themselves automatically when there's nothing left to undo or redo.

Undo/redo is **per desktop**: switch away and back, and `Cmd/Ctrl+Z` still undoes that desk's last edit, not the other one's. Adding, deleting, renaming and importing a desktop are not on any undo stack — they're changes to the project's own structure, and what takes them back is closing the project without saving.

Undo/redo does **not** cover simulation state. While the circuit is running, editing — and with it, recording new undo steps — is locked, so nothing that happens mid-run (sequential chip state, clock phase, a chip taking 12 V damage) ever becomes an undo step of its own. All of it is **run state**: it exists while the circuit is running and not a moment longer.

That includes **12 V damage**. Feed a chip 12 V and it lets its smoke out on the spot — the warning appears, its badge goes red, and it stays dead for the rest of that run, because a chip that's failed doesn't come back while the power is still wrong. Press **Stop** and every damaged chip is whole again. Wiring 12 V to the wrong rail is a mistake about the *circuit*, and a mistake you can see and fix; it was never meant to cost you the chip, so it doesn't. Fix the wiring, run again. What undo/redo restores is always the circuit you built, never a moment in its simulated behavior.

Example circuits

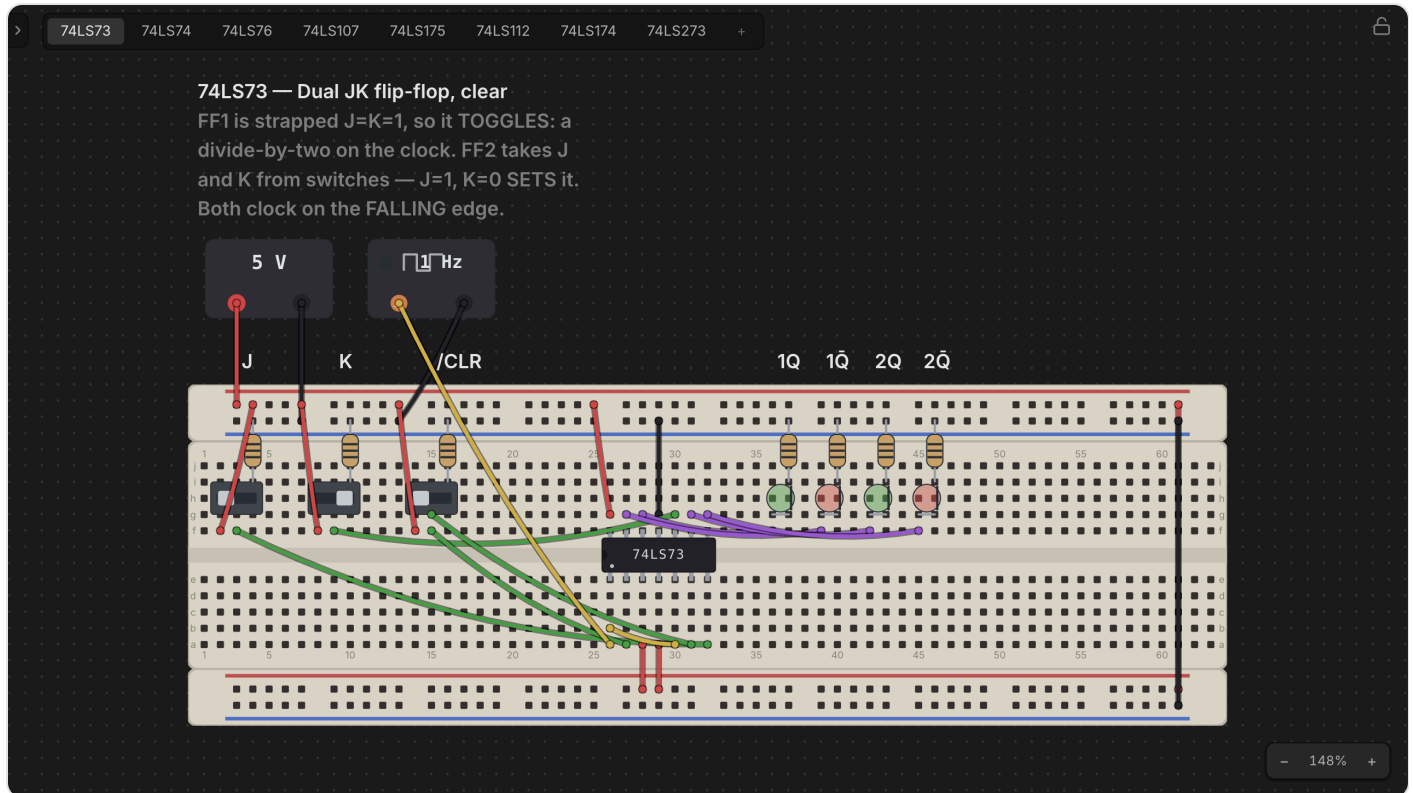
Chip Hippo's project repository ships a handful of ready-to-load example circuits as ordinary `.chiphippo` files — currently small W65C02-based breadboard computers built from 74xx glue logic, each paired with a `.hex` ROM image. Open one the same way as any saved design: **File ▶ Open...** loads it as a project of its own. Then load its matching `.hex` into the ROM chip via the memory inspector or the external programmer before pressing **Run**.

See [Getting Started](#) for building your first circuit from scratch, and [Settings](#) for what else persists between sessions.

Projects & Desktops

A big build eventually needs a bench beside it. You want to work out a clock module, an address decoder, or a memory map *without* doing it in the middle of everything else — and then drop the finished thing into the main design.

That's what a **project** is for. A project is a workspace holding several **desktops**, shown as tabs above the desk. Each desktop is a complete desk of its own — its own boards, chips, wiring, camera position, and undo history — and a design worked out on one can be copied straight onto another.



The project is the document. One `.chippipo` file holds all of it: every desktop, and the contents of every ROM you programmed — each image stored once however many chips across however many desktops are holding it. That's the whole design in a single file you can email, drop in a repo, or carry to another machine. There are no companion files to keep together and nothing to go missing.

Every desktop is the same. There is no special "main" desk and no lesser one: they are peers, all with the same menu, and which is the build and which is the bench is entirely up to you. Any of them can be renamed, duplicated, or deleted — the only rule is that a project always keeps at least one.

There is always a project open. From the first launch, the desk you are looking at is a desktop of one — a brand-new project with no name, no home of its own, and a single desktop called `Desktop 1`. Nothing has to be created before you can start building, and nothing has to be named before you can add a second desktop.

Adding desktops

Just press + at the end of the tab strip. It asks you nothing: a new empty desktop appears beside the others, named `Desktop 2`, `Desktop 3`, and so on, and you land on it.

Right-click the + for the other way one arrives:

- **New Desktop** — the same thing the plain click does.
- **Import Desktop...** — one read in from a file (see *Moving a desktop between projects* below).

Either way it's an addition, and nothing already there is touched.

There is a third way one arrives, and it doesn't belong on the **+** because it belongs to a *part*: a chip's **example circuit**. Right-click any 74xx chip on the desk, choose **Pin Assignment**, and click the circuit button in that window's top-right corner — a worked bench for that part lands as a desktop of its own, called `74LS138 example`. See [The Chip Library](#).

The **Desktop** menu has the same two plus the rest, acting on the desktop you're looking at:

- **New Desktop** — the same thing as pressing **+**.
- **Duplicate Desktop** — a copy of this desk, landing right beside it. The copy is brand-new hardware: a ROM on it gets its own backing file, so programming the copy never touches the original.
- **Import Desktop...** — read a `.desktop.chiphippo` (or a loose `.chiphippo` design) in as a new desktop. See *Moving a desktop between projects* below.
- **Export Desktop...** — write this desktop out as a self-contained file.
- **Desktop Properties...** — its **Name** and **Description**.
- **Delete Desktop** — remove it from the project.

Adding, renaming, duplicating and deleting a desktop are ordinary unsaved changes, exactly like moving a chip: nothing reaches the project's file until you save it, and closing without saving undoes all of it.

Saving

The **File** menu is the project's, because the project is the only document there is:

- **New Project** (`Cmd/Ctrl+N`) — a blank slate: a fresh unnamed project holding one new, empty desktop. However many desktops you had, a new project always starts as a single one, numbered from `Desktop 1` again.
- **Open...** (`Cmd/Ctrl+O`) — pick a `.chiphippo` project to open.
- **Open Recent** — the last ten projects you saved or opened, most recent first. The same list is one keystroke away as `Shift+Cmd/Ctrl+O`, or a right-click on the toolbar's **Open** button; each entry's **x** forgets it without opening it.

- **Save** (`Cmd/Ctrl+S`) — write the project to its file.
- **Save As...** (`Shift+Cmd/Ctrl+S`) — write it to a new one.
- **Project Properties...** — the project's **Name**, **Description**, and the read-only **Location** of its file (blank until it has been saved).

The toolbar's four file buttons are the same four actions (with the recent list behind a right-click on **Open**), and the window title's • means the project has changes you haven't saved.

You never have to choose a file. A project that has never been given a home lives in Chip Hippo's own working slot, and `Cmd/Ctrl+S` writes it there with no dialog at all — design something, save it, quit, and it's there when you come back. **Save As...** is what gives the project a real file when you want one; it takes the project's name from the file you pick, so there's no separate "name this project" step. From then on Save just writes that file and moves it to the top of the recent list.

Changing projects

New Project and **Open...** both replace what's open, so nothing is allowed to go with it unasked:

- A project that has **never been given a home** is always asked about, whether or not anything in it is unsaved. It lives in Chip Hippo's one working slot, the project taking its place is about to claim that slot, and there is nowhere else for it to go — so replacing it loses it either way, and even a `Cmd/Ctrl+S` into the slot doesn't change that. You're offered **save** (which gives it a file of its own), **discard**, or **cancel**.
- A **saved** project has a file nothing is claiming, so it's only asked about when it has changes: **save**, **discard**, or **cancel**.

Choose to save and it goes through, then the action you asked for carries on — you're never made to ask twice. Cancel, at that dialog or at a dialog behind it, and nothing happens at all.

Working across tabs

Click a tab to put that desktop on the desk. Everything follows the active tab:

- **Undo/redo** is per desktop. Switch away and back, and `Cmd/Ctrl+Z` still undoes that desk's last edit, not the other one's.
- Each desktop remembers **where you were** — its camera position comes back with it. Panning and zooming are never counted as changes to the design.
- The **simulation stops** when you switch. A running circuit belongs to the desk it's running on, so the transport resets rather than following you to another desktop. Any open pin-assignment or memory-inspector windows close for the same reason: they were pointing at chips on the desk you just left.

Tabs carry no marker of their own. There is one save and one dirty marker, because there is one document: the window title shows the project — `Untitled` until you name it — the active desktop, and a leading • if anything anywhere in the project is unsaved.

Managing tabs

Right-click a tab for its menu:

- **Properties...** opens the same dialog every part, board, and wire has, with the same two fields — a **Name**, which is what the tab reads, and a **Description**, a note on what this bench is for. Hover the tab to see the description.
- **Duplicate Desktop** copies it, landing the copy right beside it.
- **Export Desktop...** writes it out on its own (below).
- **Delete Desktop** removes it from the project. Any desktop can go — there's nothing special about the first one — except the **last** one left, because a project with no desktops would have nothing to open. Its design goes with it, but its file is untouched: close the project without saving and it comes back.

Desktop numbers only ever count up: deleting `Desktop 2` doesn't make the next one `Desktop 2` again, so a name you wrote in a note keeps meaning the same bench. Rename them to whatever the benches actually are — `CPU`, `Clock`, `Decoder` — through Properties...

Moving a desktop between projects

Since a desktop isn't a file, moving one somewhere else is a **snapshot**:

- **Desktop ► Export Desktop...** writes the desk to a `.desktop.chiphippo` file — its boards, parts and wiring, plus the contents of any ROM on it. The file is self-contained and has no link back: it is a copy, and nothing you do to the project afterwards changes it.
- **Import Desktop...** — on the **Desktop** menu, and by right-clicking the tab strip's **+** — reads one back as a **new** desktop, keeping the name it was exported under. Import is always an addition, so no file operation can ever replace the desk you're looking at.

An imported desktop is brand-new hardware in the same way a duplicate is: import the same file twice and the two copies' ROMs are independent. Opening a `.desktop.chiphippo` with **File ► Open...** works too — it becomes a new project of one desktop.

Copying a design from one desktop to another

For a *part* of a desk rather than the whole of it, copy and paste:

1. On the desktop holding it, **shift-drag a marquee** around the design — the boards, the chips on them, and the wiring. A marquee takes in any board it encloses *completely*, and the selected set is outlined as one block. `Cmd/Ctrl` -click adds or removes an individual item afterwards, so a box that caught one thing too many (or one too few) doesn't have to be redrawn.
2. Press `Cmd/Ctrl+C` . The whole sub-assembly is copied: the boards, everything seated on them (whether or not the box touched it), every wire with **both** ends inside the set, and the buses, net names, and anchored labels riding them. A wire with one end outside is left behind — it would have to be cut.
3. Click the tab you want it on and press `Cmd/Ctrl+V` .

The design appears **under the cursor as a translucent ghost**, following it with no button held — the same way a fresh breadboard does when you pick one from the palette. Move it where you want it:

- Boards outline **green** where the design will land and **red** where it overlaps something already on that desk.
- Bring it near a board that's already there and it **snaps flush**, dovetailing exactly as a board you dragged into place would.
- **Click** to drop it. **Esc** throws it away and leaves the desk untouched.

The drop is all-or-nothing: if any board would land on top of something, the click is refused rather than pasting half a design and silently cutting its wiring. The whole paste is a single undo step, and the pasted design is brand new hardware — fresh boards, fresh parts, and (for a ROM) its own fresh backing file, so editing the copy never touches the original.

The clipboard survives switching tabs, so you can paste the same design onto several desktops, or stamp it more than once on the same one.

Leaving a project

A project stays open between sessions — named or not. Relaunch Chip Hippo and it comes back on the desktop you were last on: the unsaved project from its working slot if there is one, and otherwise the project you used most recently. Starting or opening another one asks first about anything you'd lose (see *Changing projects* above).

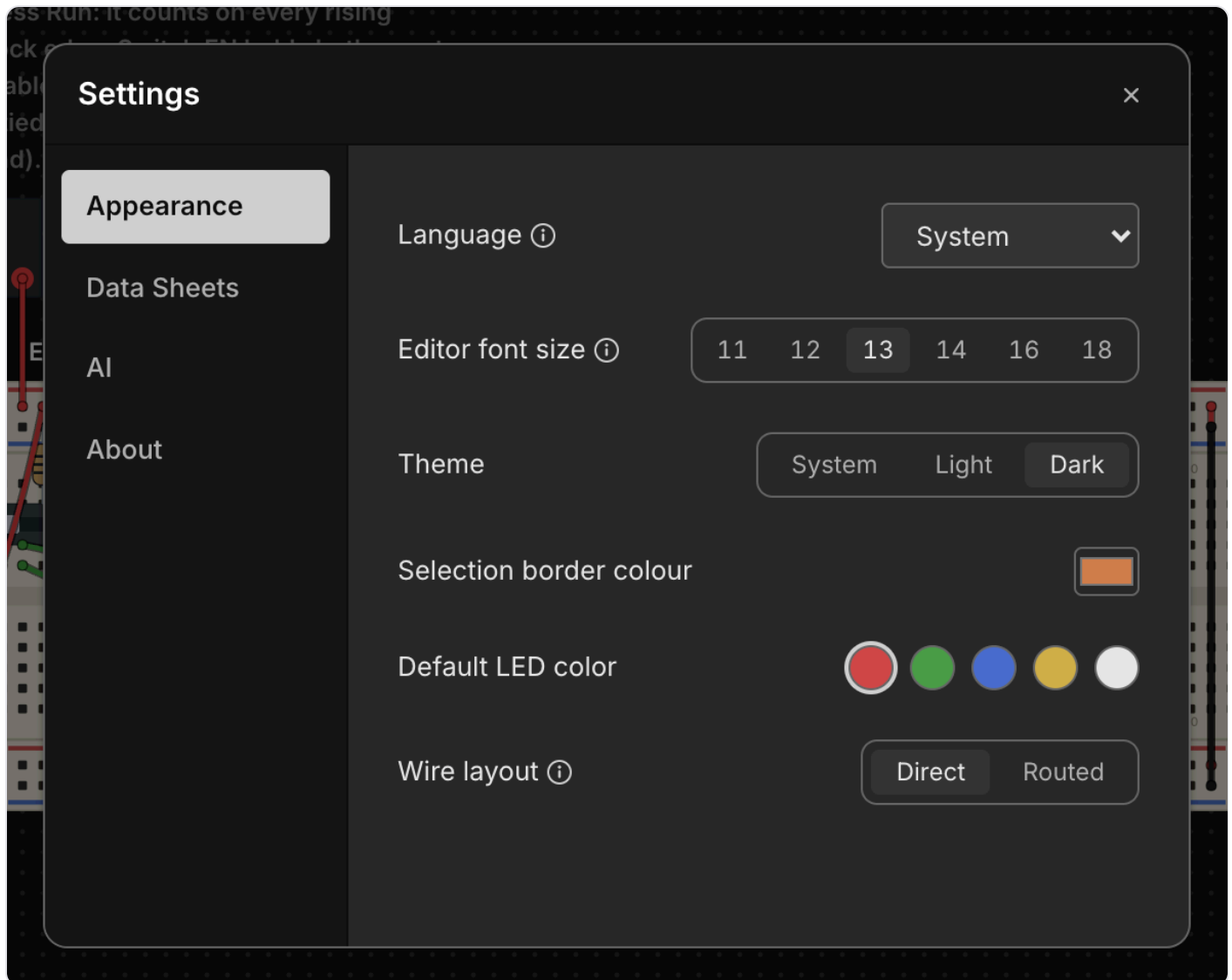
Closing the window or quitting asks the same question. Because you didn't ask for a save, it doesn't ask you where anything goes: the project is written to the file it already has, or — having none yet — to the working slot, where the next launch will find it. Discard instead and the session's changes are gone. Cancel and nothing closes at all.

Opening an older project

Projects saved by an earlier version of Chip Hippo kept each desktop in a `.desktop.chiphippo` file of its own and listed their paths. Opening one brings every one of those designs into the single file, and **nothing of yours is deleted** — the old desktop files are left exactly where they are, and it is the project that changes shape. If one of those files has since been moved or deleted, that desktop opens empty and Chip Hippo tells you which file it was looking for.

Settings

Chip Hippo keeps its handful of app-wide preferences in one small **Settings** dialog — a tabbed master-detail card with a left nav rail and a panel on the right. It's deliberately minimal: five tabs, a few controls each, applied live the moment you change them.



Opening Settings

Open the dialog with `Cmd/Ctrl+,`, or click the gear icon in the top-right corner of the header. Both routes lead to the same dialog seeded with your current settings, so it doesn't matter which you use.

Several settings carry a small ⓘ beside their label. Click it for a note explaining what the setting does and what it doesn't touch; `Esc`, a click anywhere outside the note, or the ⓘ again closes it. The first `Esc` closes the note and the next one closes Settings, so you never lose the dialog by dismissing a note.

Appearance

The **Appearance** tab is the one the dialog opens on:

- **Language** — the language Chip Hippo's own menus, panels, dialogs and tooltips are shown in: **System, English, Deutsch, Español, Français, Italiano,** or **System**. *System* follows your operating system's own language, falling back to English if Chip Hippo doesn't ship that one. Every language names itself in its own words rather than in the one you're currently reading, so the list stays usable if you land on one you can't. The change applies **immediately and everywhere** — the menu bar and every open window included, with no restart and nothing reloaded, so an unsaved design is never at risk. This user guide is only available in English.
- **Editor font size** — how big Chip Hippo's own text is drawn, in pixels: **11, 12, 13, 14, 16** or **18**, default **13**. It applies to every window at once, including the floating pinout, memory inspector and user-guide windows, and to the boxes around the text — buttons and panels grow with it rather than clipping it. `Cmd+=` and `Cmd+-` step through the sizes from the keyboard and `Cmd+0` restores the default. What it does *not* change is the desk itself: chip labels, board addressing and your own annotations are printed on the circuit and drawn to the breadboard's scale, so they grow with the desk zoom (`Option+Cmd+=`) instead.
- **Theme** — **System, Light** or **Dark**. *System* follows your operating system's own setting; the other two pin it. The choice applies to every Chip Hippo window at once, including the floating pinout, memory inspector and user-guide windows, and to the native menus and dialogs.
- **Selection border colour** — a `#rrggbb` colour picker for the outline drawn around whatever's selected on the desk. Leave it unset to use the theme's default accent colour instead of a custom one.
- **Default LED color** — which colour a newly placed LED, LED bar or seven-segment digit gets. Unlike the others this one isn't applied live; it's read when you place a part, so changing it doesn't recolour anything already on the desk. Any placed part's colour can still be changed from its **Properties...** dialog.
- **Wire layout** — **Direct** (the default) or **Routed**, the layout method a newly laid wire gets. Direct wires sag from hole to hole; routed wires run straight and can be bent around obstacles by dragging points into them (see [Wiring](#)). Like the LED colour it's read when you lay a wire, so wires already on the desk keep the layout they have — change one from its own **Properties...** dialog.
- **Auto-close tray folders** — **On** or **Off** (the default). When it's on, opening a folder in the parts tray closes every other one, so only the section you're working in stays open, along with the folder it sits inside. Clicking an icon on the shut tray's strip does the same. It acts on the next folder you open, so switching it on doesn't close anything by itself.

Everything but the LED colour and the wire layout takes effect immediately — there's no separate Apply or OK step. (Changing the language is the one setting that closes and reopens this dialog, so the card

you're looking at comes back in the language you just picked, on the tab you were reading.)

Serial I/O

The **Serial I/O** tab holds the **connections** — the named Arduino boards that **Outputs and Inputs** talk to. A dropdown at the top picks one, and the settings under it are that connection's. The list always starts with the built-in **Mock**, a pretend board with nothing to configure (see [The Mock connection](#)); its **Open window...** shows what it receives and lets you send values back. **+** beside the list adds a connection, and the bin removes the one picked — click it twice (the first click turns it into a **?**; clicking anywhere else, or **Esc**, puts it back). The ⓘ says what connections are for.

Each connection has a **Name**, the **Port** it is plugged into (**Re-scan** looks again after you plug a board in), a **Baud rate**, the **Language** Generate writes the board's code in — **C++** (the default, an Arduino header) or **Python** (a module for MicroPython and CircuitPython boards; see [Python boards](#)) — and under **Advanced** the rest of the serial framing (data bits, parity, stop bits, flow control). Unlike the other tabs, nothing here applies until you press **Apply**, which also checks the port is present — if it isn't, you're told so and the button becomes **Apply anyway**. You can edit several connections before applying: each keeps its changes while you look at another, and one with changes not yet applied is marked • in the list. **Open window...** opens the connection's **window**. The book icon beside the card's close button (shown on this tab only) opens the [Serial Protocol](#) reference in the user guide.

A connection the circuit can't run with yet says why in the list — **Port unavailable**, or **Needs configuration** when it arrived in a project from another computer (a project carries its connections' names and settings, but never a port) — and its **Port** is marked in red. See [Connections](#) for the details.

Data Sheets

The **Data Sheets** tab points Chip Hippo at an external folder of manufacturer datasheet PDFs on your machine. Click **Browse...** to pick a folder with the native file picker; the chosen path is shown next to it, with a **Clear** button that resets it to no folder selected.

Name the PDFs in that folder after each chip's catalog reference (for example **74LS00.pdf**). When a chip's file is found there, its **pin-assignments window** grows a button that opens the PDF in your system's default viewer. This is separate from the small datasheet crop the pinout window already shows for most chips — the external folder is for the full manufacturer document, not the built-in crop.

If you don't already have a collection, **Download...** fetches one for you. It pulls a datasheet for every part Chip Hippo has a published source on file for, names each file after the part it belongs to, and points the setting above at that folder when it's done — so the buttons appear with nothing else to set up. A

progress window counts the files in and names anything that couldn't be fetched; closing it stops the download and keeps whatever already arrived. The sources are:

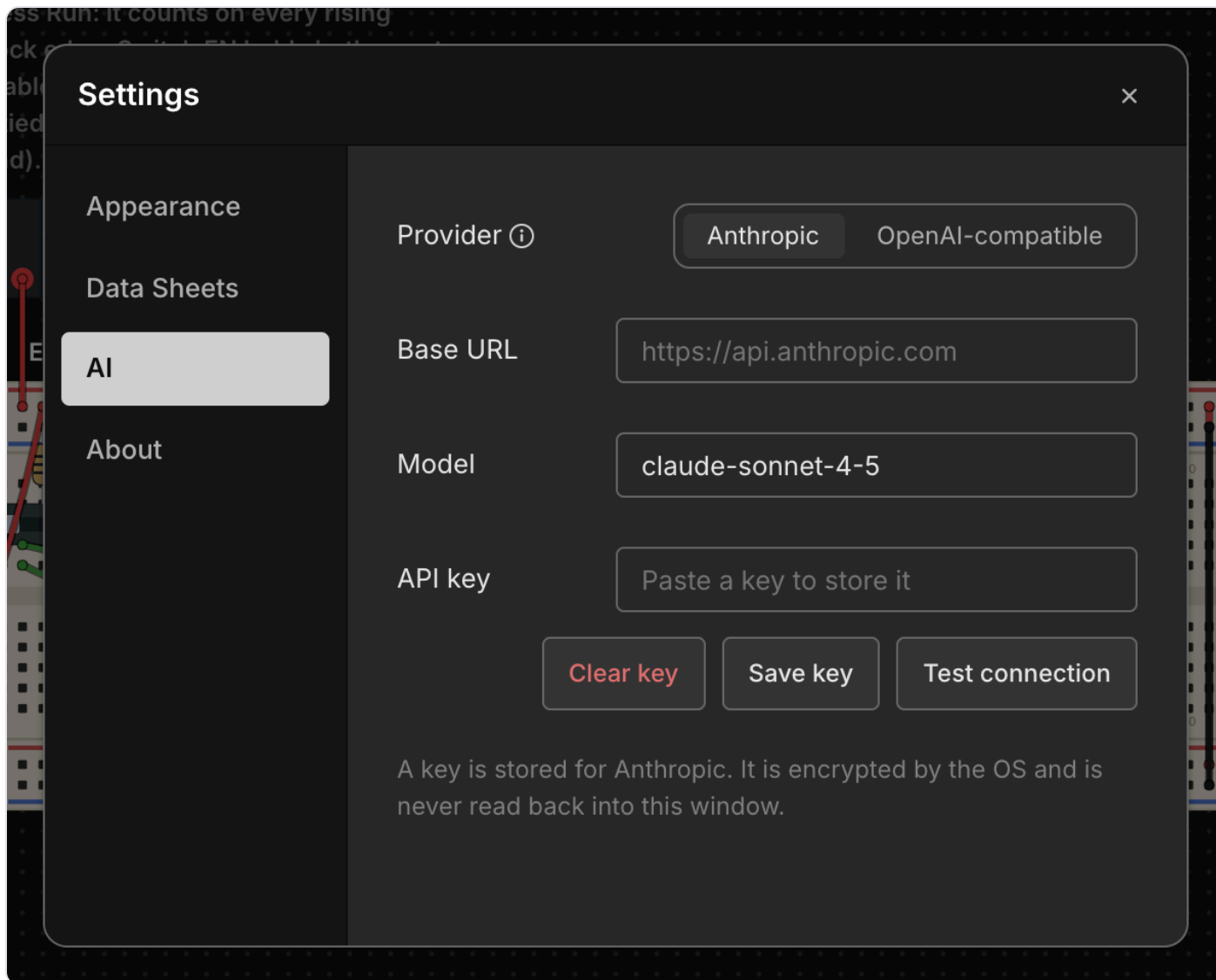
- [Texas Instruments](#) — the current manufacturer's own documents for much of the 74LS family;
- [Microchip](#) — the Atmel EEPROM parts;
- [Alliance Memory](#) — the large SRAM;
- the [Western Design Center](#) — likewise for the 65xx parts;
- [Zilog](#) — the Z80 family user manual;
- the [USC EE 459Lx reference library](#) — vendor scans of the older 74LS parts, collected for a university course, plus the HD44780, the memory parts and the DIP resistor/oscillator packages;
- a datasheet archive, for the one part whose maker no longer publishes it.

Expect a few tens of megabytes in total; the modern manufacturer PDFs are much larger than the old scans.

Between them they don't carry every part in the catalog, so some chips will still have no PDF button. Press **Download...** again whenever you like — it replaces what it fetched last time. It always writes to Chip Hippo's own folder and never to one you picked yourself, so a collection of your own is never overwritten: **Browse...** and **Download...** are simply two ways of setting the same folder, and whichever you did last is the one in effect.

AI

The **AI** tab is where you point Chip Hippo at your own AI connection, for the [AI circuit builder](#). Nothing here is required — the app works completely without it, and contacts no AI provider until a connection is configured and you ask the builder for something. (The only other times Chip Hippo reaches the network at all are the datasheet **Download...** button above, while you're pressing it, and the update check on the **About** tab below, which is switched off until you turn it on.)



- **Provider** — **Anthropic**, or **OpenAI-compatible**. The second covers Ollama, LM Studio, OpenRouter, vLLM and anything else speaking that request format; you supply the base URL.
- **Base URL** and **Model** — leave either blank to use the chosen provider's default, which is shown as the field's placeholder. Both commit when you leave the field or press **Enter**, not as you type.
- **API key** — paste it and click **Save key**. **Clear key** forgets it. A local server that needs no key works with this left blank.
- **Test connection** — one small request that checks the base URL, key and model together, so a typo is found here rather than on your first build.

The key is never written to Chip Hippo's settings file. It's handed to your operating system's own secure credential store — Keychain on macOS, the Credential Manager on Windows, the desktop keyring on Linux — encrypted, and it's never read back into the app's window afterwards; the dialog only learns *whether* a key is stored. If no secure store is available on your system, Chip Hippo **refuses to save the key** and says so rather than quietly writing it out in the clear.

About

The **About** tab shows which version of Chip Hippo you're running, and is where updates are handled.

- **Check automatically** — **On** or **Off**, and **Off** is the default. On, Chip Hippo asks GitHub once, shortly after launch, whether there's a newer release. Off, it never asks on its own — but the button below still works whenever you want to know.
- **Check for updates** — checks right now and says what it found on the line underneath: that you're up to date, that a version is downloading, or that the check couldn't be made.

When an update is found it downloads quietly in the background; you can carry on working, and there's no progress bar in your way. Once it's ready a **Restart to update** button appears here, and a notice appears in the corner of the desk with a **Restart** button of its own. **Nothing is installed until you say so.** If you'd rather not stop what you're doing, ignore it — the update installs by itself the next time you quit Chip Hippo, and restarting here runs the same "you have unsaved changes" prompt that quitting does, so a design in progress is never lost to an update.

The check itself is the only thing sent, and only to GitHub. Chip Hippo has no analytics and reports nothing about you or your circuits.

You'll see this tab differently depending on where your copy came from. A copy installed from the **Mac App Store** updates through the App Store like every other app there, so this tab shows only the version and says so — there are no controls, because there'd be nothing for them to do. A development build says that updates are only available in installed builds.

What else persists automatically

A few things aren't part of this dialog but are remembered between sessions without any action on your part: the app window's position and size, the desk camera — your current pan position and zoom level — and which panels you left open (the parts tray, the build guide, the analyzer, the AI builder). The ones you can resize remember that too: the parts tray's **width**, and the analyzer and AI builder's **height**. Close Chip Hippo and reopen it, and you're back exactly where you left off.

The desk padlock isn't one of these: it's saved in the project file, so it belongs to the project rather than to the app (see [The Desk & Breadboards](#)).

See [Files, Saving & Undo](#) for how your circuit itself is saved.

Keyboard Shortcuts

Chip Hippo follows Electron's `CmdOrCtrl` convention throughout, so every shortcut listed here as `Cmd` also works as `Ctrl` on Windows and Linux. Most shortcuts are disabled while a text field has focus, and some are further gated on the current tool or simulation state — those conditions are noted alongside the shortcut.

Desk & simulation

Shortcut	Action
<code>Space</code>	Run / Stop the simulation (not while typing, or while a placement/wire/bus tool is armed)
<code>Cmd+R</code>	Run / Stop the simulation
<code>1 – 9 , 0</code>	Press signal buttons 1–10, top to bottom (<code>0</code> is the tenth) — only while the circuit runs , and several at once (see External signals)
<code>Tab</code>	Switch between the Breadboard and Schematic views (not while typing)
<code>Escape</code>	Abandon a drag in flight, then unpin a probed net, then disarm the probe, then cancel a pending wire/bus, then cancel a placement in hand, then deselect — whichever applies first
<code>Delete</code> / <code>Backspace</code>	Remove the current selection (a part, wire, bus, annotation, board, or a whole multi-selection); a selected Output or Input tag is unplugged back onto its card instead

Selecting & moving

These are mouse chords rather than keystrokes, but they are the modifiers the desk answers to. [The Desk](#) covers building a selection; [Chips & Components](#) covers moving one.

Chord	Action
Shift -drag	Rubber-band a selection — it replaces whatever was selected
Cmd -click	Add one item to the selection, or take it back out (Ctrl -click on Windows/Linux; on a Mac Ctrl -click stays the context menu)
drag a part	Re-seat that part alone, leaving every wire where it was
Option -drag a part	Move it with everything plugged into it — the wire ends, and the legs of the resistors and LEDs, in the column-halves its pins occupy
drag any member of a selection	Move the whole selection as one rigid unit
Option -drag a member of a selection	The same, carrying everything riding any member of it
hold Option over a selection	Ring every hole an Option -drag would carry, before committing to the drag (a selected part, or a whole multi-selection)
Option -drag a board	Tear off the run of mated strips reachable forward (below/right) from the one you grabbed
Option + Shift -drag a board	The same, running backward (above/left)

Tools

Shortcut	Action
W	Arm/disarm the wire tool
B	Arm/disarm the bus tool
I or P	Arm/disarm the probe tool (works even while the simulation is running)
M	Disarm whichever of the wire/bus/probe tool is currently armed
H	Fade the wires down to a stub at each end, and back (works while running — it only changes what's drawn)
1 – 8	While the wire tool is armed, pick a wire color
1 – 8	While the bus tool is armed, pick the bus width — 2 – 8 for that many bits, 1 for 16-bit

Placing & rotating parts

Shortcut	Action
R	Rotate or flip the part being placed or selected — see Chips & Components for the exact behavior per part type
R (Output/Input tag)	Turn the tag being dragged, or the selected tag, a quarter-turn
F	Flip an LED's polarity while its placement ghost is armed
Cmd+C	Copy the selection — one part, a whole multi-selection of parts, or (if it includes boards) the whole sub-assembly
Cmd+V	Paste a copy as a new placement ghost

View

Shortcut	Action
Cmd+F	Fit the desk (recentring it) — or the schematic, when it is showing
Cmd+Shift+F	Zoom out to fit everything at once
Cmd+=	Increase the interface text size
Cmd+-	Decrease the interface text size
Cmd+0	Reset the interface text size (13 px)
Option+Cmd+=	Zoom the desk in
Option+Cmd+-	Zoom the desk out
Option+Cmd+0	Reset the desk zoom
Cmd+L	Lock or unlock the desk against the scroll wheel
A	Toggle the Logic Analyzer panel
Cmd+P	Toggle the parts palette panel

Both scale shortcuts want the **unshifted** key. On a layout where **+** needs **Shift**, use **Cmd+=** rather than **Cmd+Shift+=** — the shifted chord is deliberately ignored, so it can't collide with anything else.

The two families above do different jobs. **Cmd+=** and friends change **Chip Hippo's own text**: toolbars, panels, menus and dialogs, in every window at once — the same setting as Settings ▸ Appearance ▸

Editor font size. Hold **Option** as well and you move the **desk camera** instead, zooming the circuit, chips, boards, wires and all. Markings on the desk are printed on the circuit, so they follow the desk zoom rather than the text size.

On a Mac, `Option+Cmd+=` and `Option+Cmd+-` are also macOS's own Zoom shortcuts (System Settings ▶ Accessibility ▶ Zoom ▶ *Use keyboard shortcuts to zoom*). If you have that turned on, the system takes the keys first and Chip Hippo never sees them — use the zoom cluster in the bottom corner of the desk, or `Cmd+F` to fit, instead.

Files & editing

Shortcut	Action
<code>Cmd+N</code>	New Project (a blank slate, one empty desktop)
<code>Cmd+O</code>	Open... (load a saved project)
<code>Cmd+Shift+O</code>	Recent projects (the same menu a right-click on the toolbar's Open button drops)
<code>Cmd+S</code>	Save the project to its file
<code>Cmd+Shift+S</code>	Save As... (give the project a new file)
<code>Cmd+B</code>	Toggle the build guide (Bill Of Materials) panel
<code>Cmd+A</code>	Select All — every board, part and wire on the desk; in a text field, its text
<code>Cmd+Z</code>	Undo
<code>Cmd+Shift+Z</code>	Redo

App

Shortcut	Action
<code>Cmd+,</code>	Open Settings
<code>Cmd+K</code>	Open the quick Keyboard Shortcuts popup
<code>Cmd+/<code></code></code>	Open the Chip Hippo User Guide (this guide)
<code>Alt+Cmd+I</code>	Toggle Developer Tools